

Welcome to the ISC Flux Tutorial!



 **Lawrence Livermore
National Laboratory**



flux





flux



Agenda

1. Introduction to Flux Framework
 - Hands-on Flux Demonstration
 - ☕ coffee break
2. Flux in Different Environments
3. Community and Questions



Part I

Introduction to Flux Framework



What is Flux Framework?

Flux Framework "Flux" is a system level **workload manager** developed by Lawrence Livermore National Laboratory that combines hierarchical **job management** with graph-based **scheduling**.

Did you know!

Flux has 15+ contributors, including principal engineers behind Slurm, and developers with expertise in container technologies, scheduling, and cloud computing.



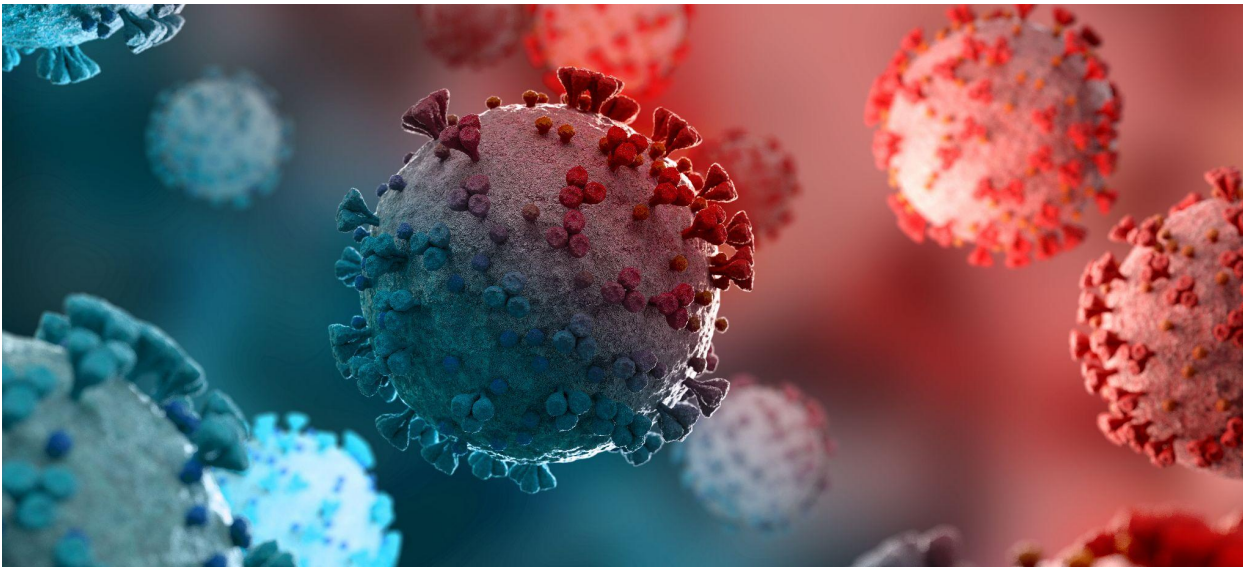
Flux is the system scheduler on El Capitan



- 1. El Capitan (LLNL)
- 11. Tuolumne (LLNL)
- 28. El Dorado (SNL)
- 63. rzAdams (LLNL)
- 257. Tioga (LLNL)
- 337. Tenaya (LLNL)

Why do we need workload managers?

Emerging scientific workflows are complex, integrating simulation with AI/ML and services that use heterogeneous resources (machine architectures, accelerators / GPUs, and devices).



Glossary of terms

A **workload manager** is a computer program capable of scheduling and launching large parallel jobs while managing user priorities, queues, limits, and scheduling.

Resource management is the process by which we manage the time and compute resources on supercomputers.

A **workflow** is a set of related jobs that are orchestrated to accomplish a goal. A **workflow manager** performs this orchestration. Flux is *not* a workflow manager, but provides workflow capabilities and integrates with many workflow managers.

Glossary of terms

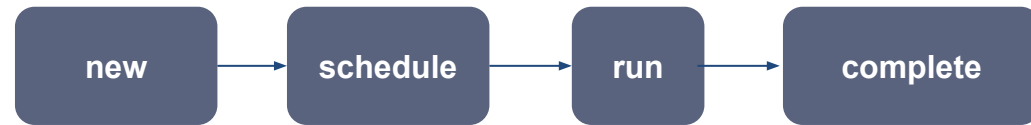
Scheduling is the process of determining which resources are going to serve which users in what order.

Queueing is the process of recording the order in which jobs were submitted on a cluster.

Accounting for time on compute clusters is accomplished by determining user and “bank” (“project”) priorities.

Job Management

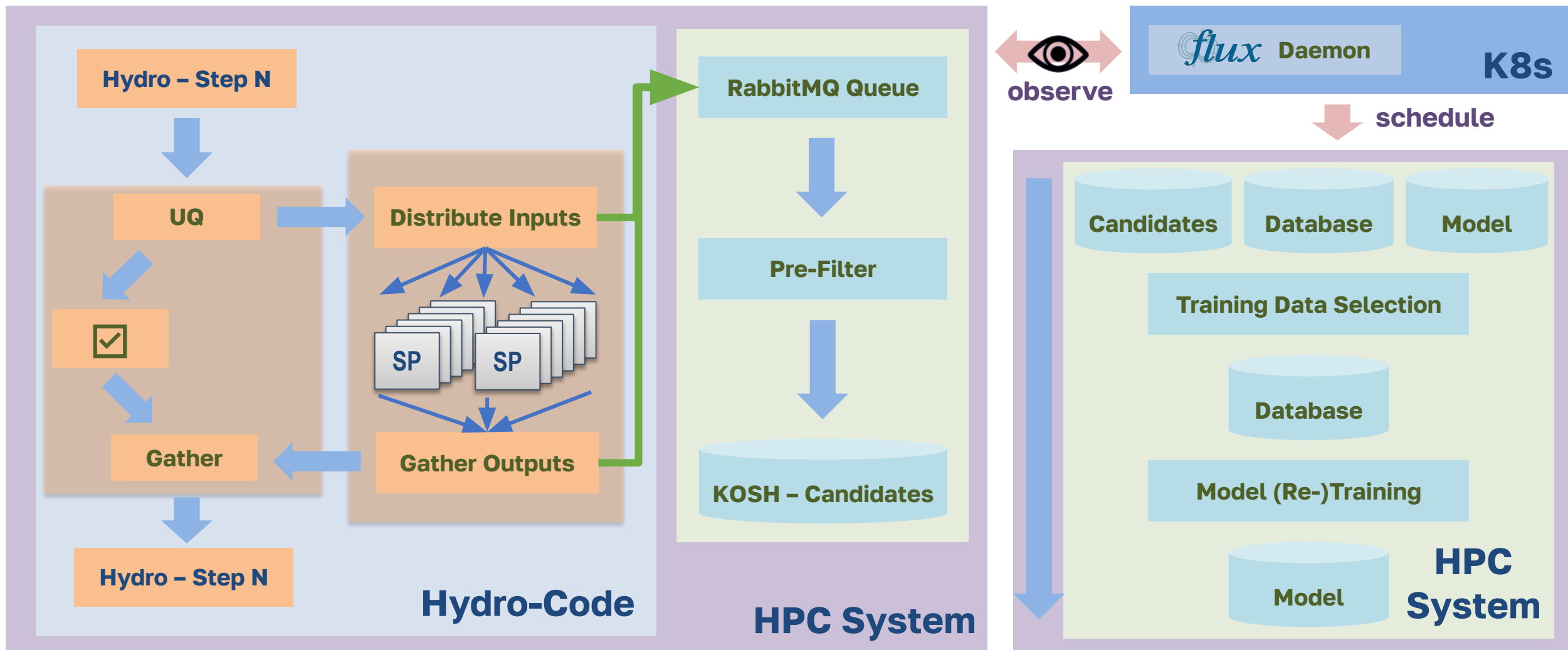
What is the lifecycle of my job?



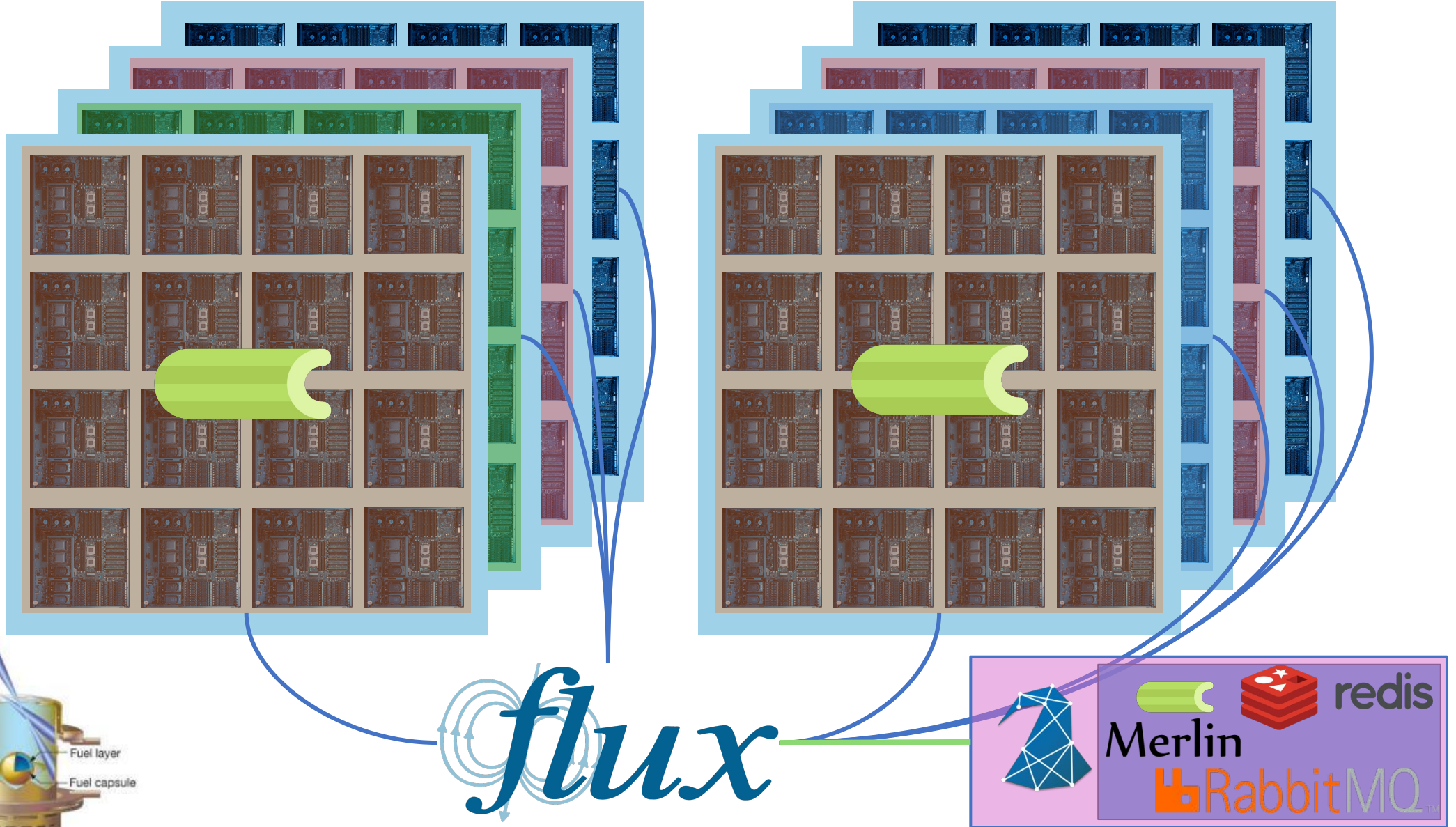
*"Hey guys, why are we
building another one?"*



Autonomous Multiscale (AMS) Workflow

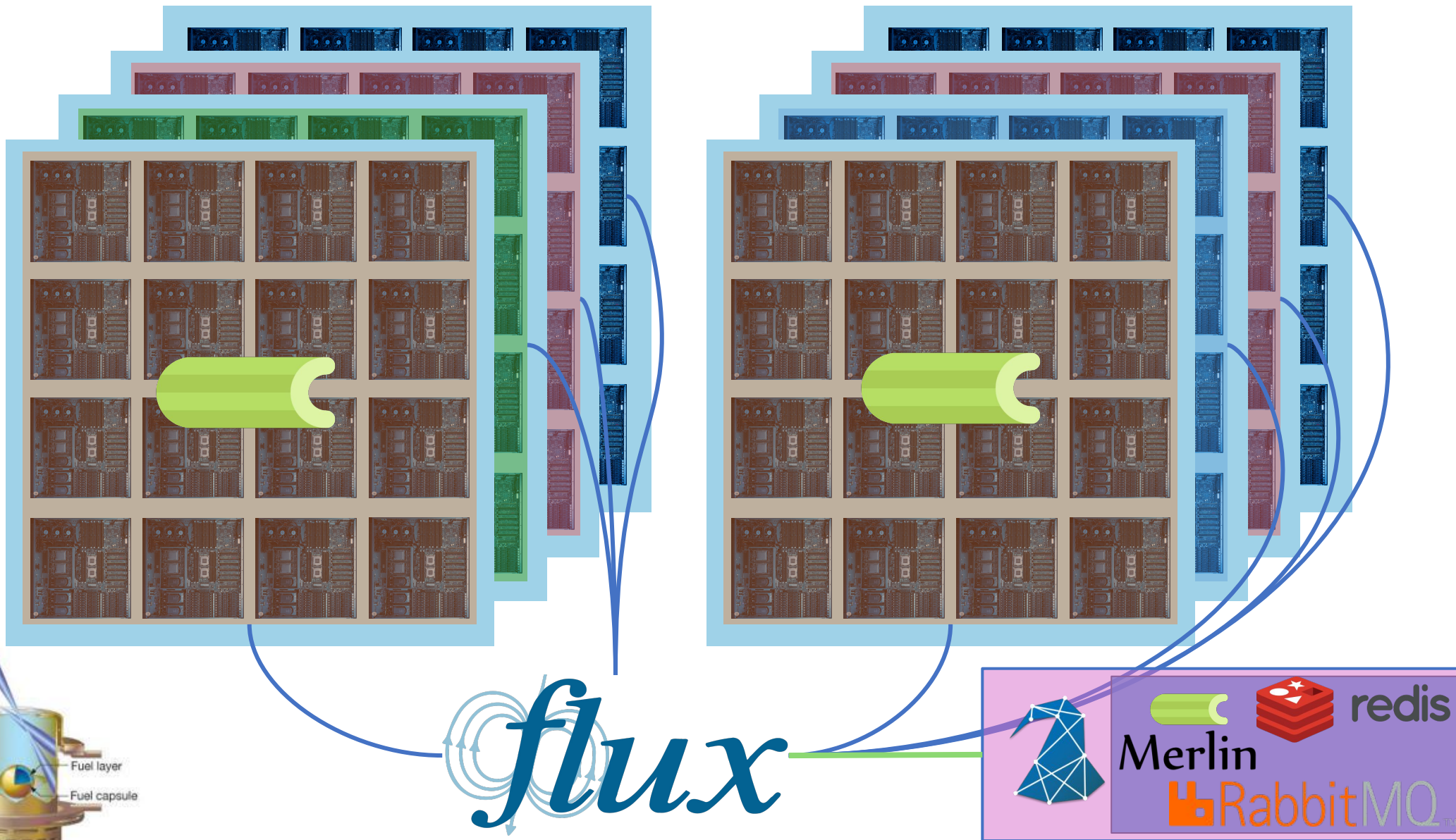


<https://www.osti.gov/servlets/purl/1630805>



Combination of 1024- and 512-node jobs (blue subsquare is 32 nodes), 61,440 concurrent workers

1M simulations per wallclock hour, 100M total.



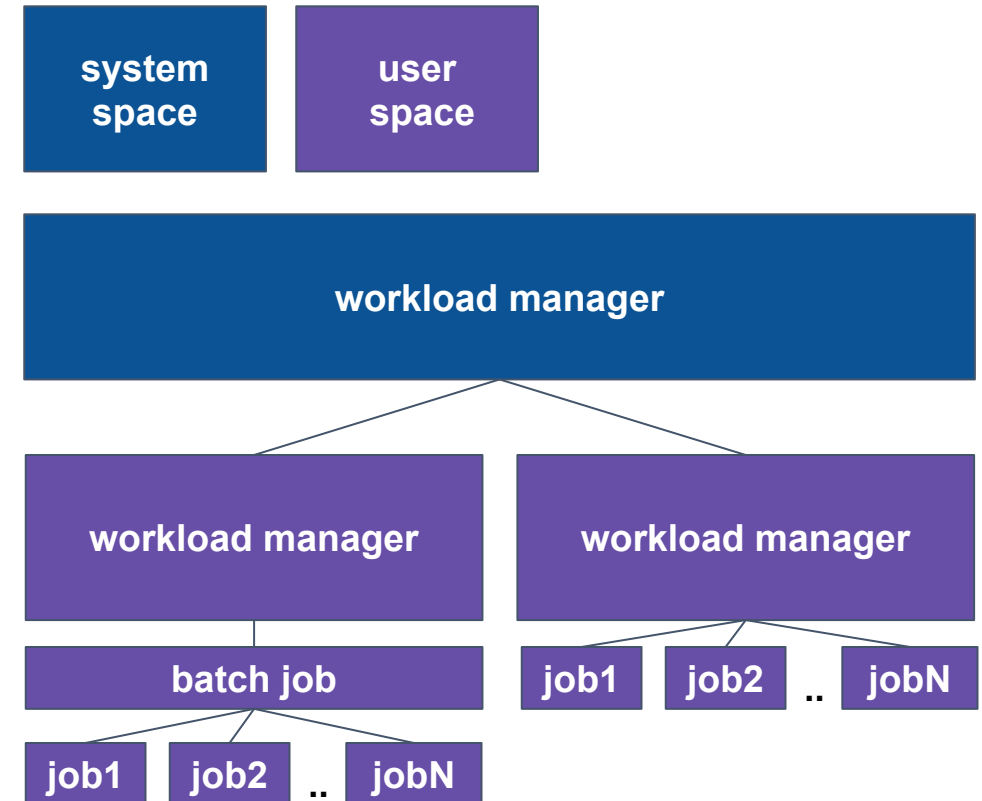
1. Emerging workflows and systems need a flexible resource model

- Resources like network, power, storage, and **<insert future resource>** are not easily integrated into existing schedulers.
- A *flexible resource model* can express arbitrary resources, including those that don't exist yet.



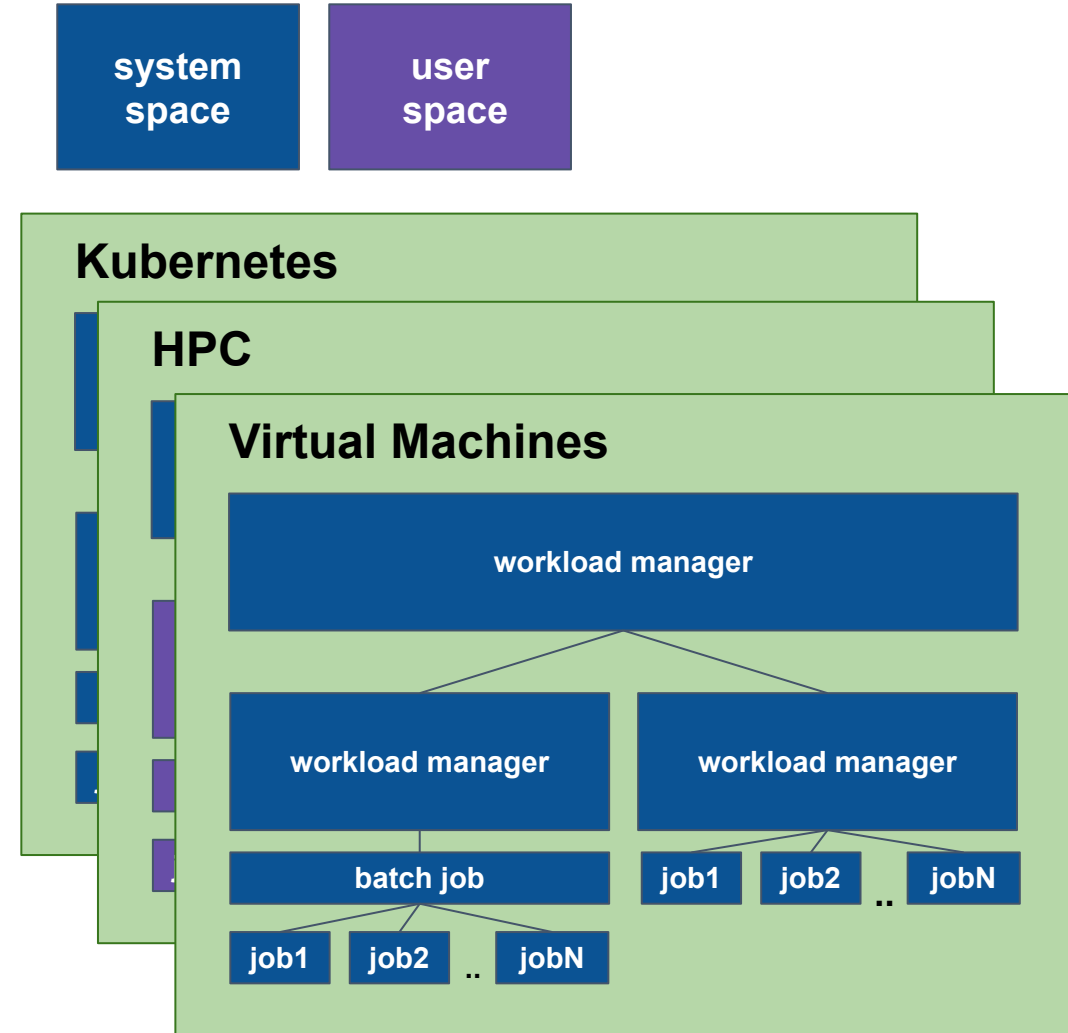
2. The hierarchical design facilitates portability

- A resource manager can be instantiated *as a job* under another resource manager
- The resource manager itself can be a portability layer.



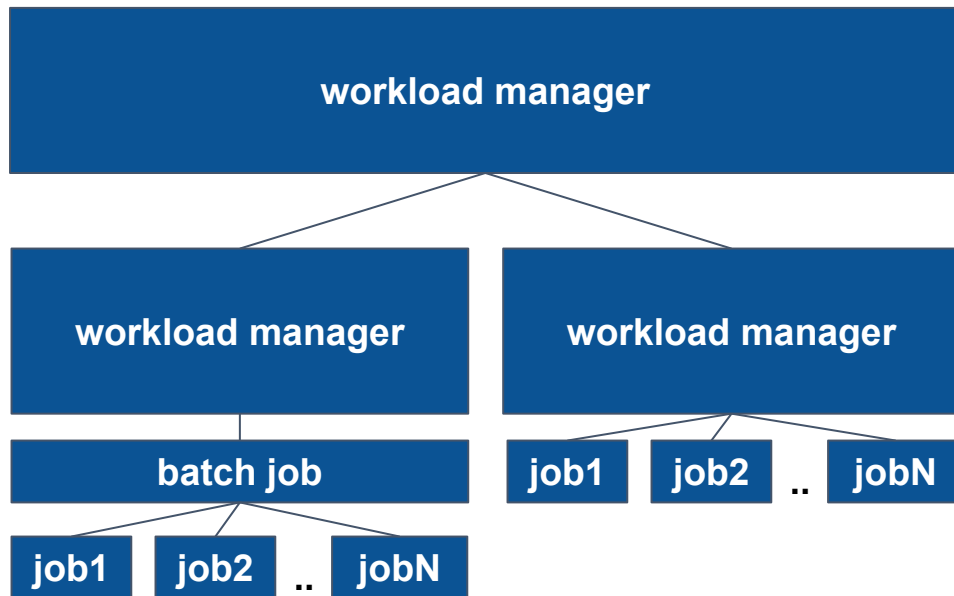
2. The hierarchical design facilitates portability

- A resource manager can be instantiated *as a job* under another resource manager
- The resource manager itself can be a portability layer.



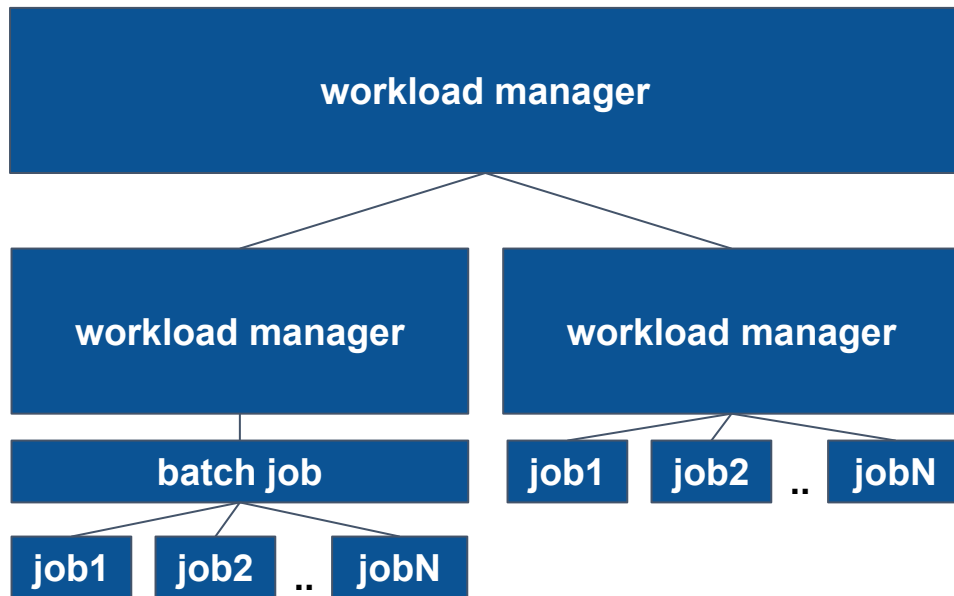
3. The hierarchical, modular design enables scheduler configurability

One scheduling algorithm to rule them all (what about queue)?



3. The hierarchical, modular design enables scheduler configurability

One scheduling algorithm to rule them all (what about queue)?

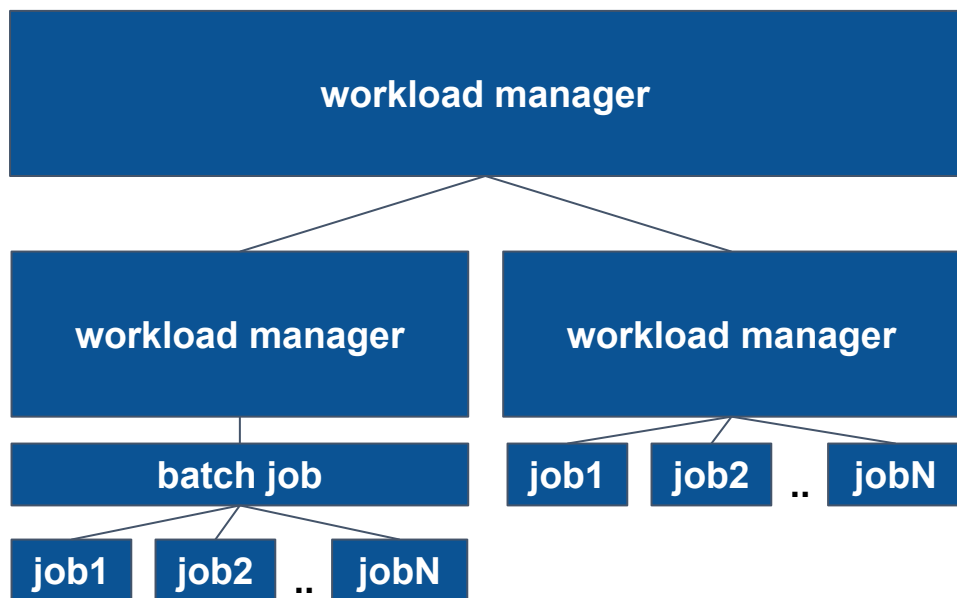


Did you know!

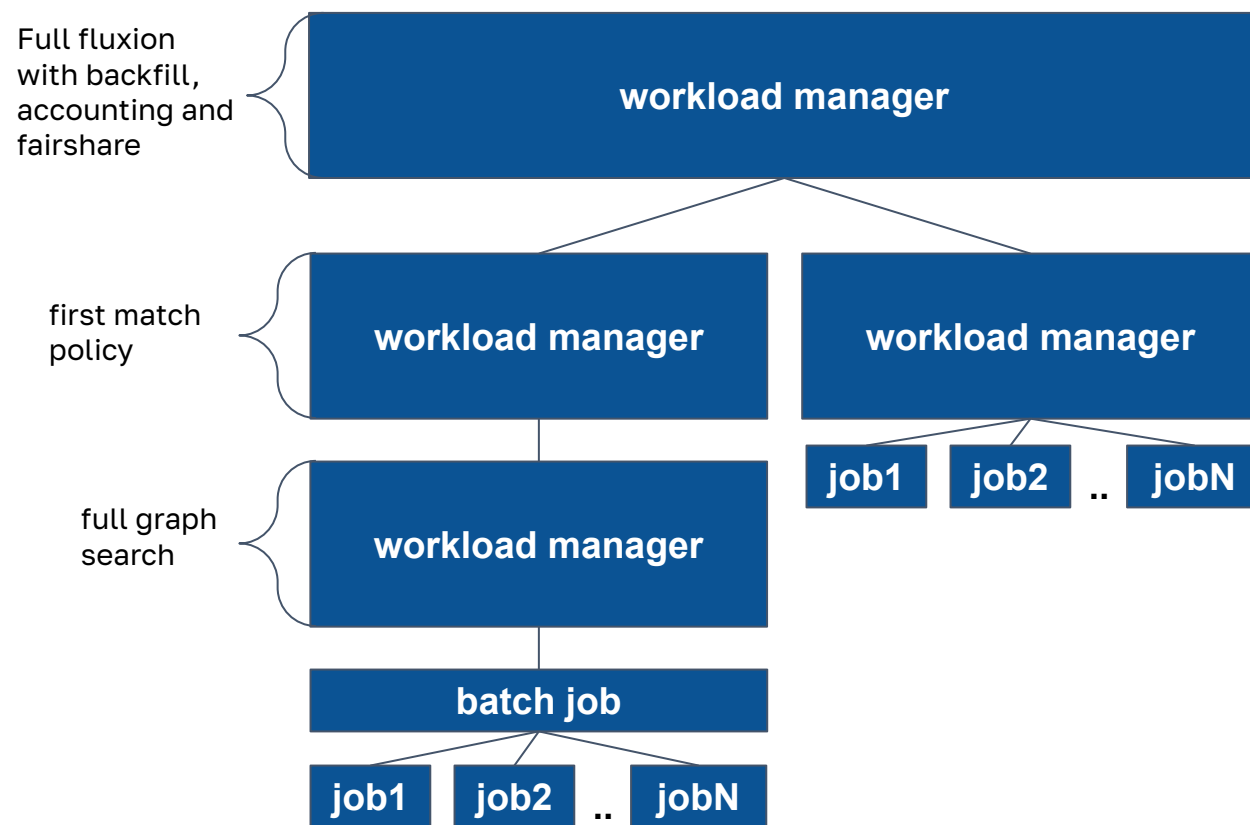
A **policy** can dictate how to schedule. Should we stop at the first match? Continue until we find all matches? Use a different algorithm?

3. The hierarchical, modular design enables scheduler configurability

One scheduling algorithm to rule them all (what about queue)?

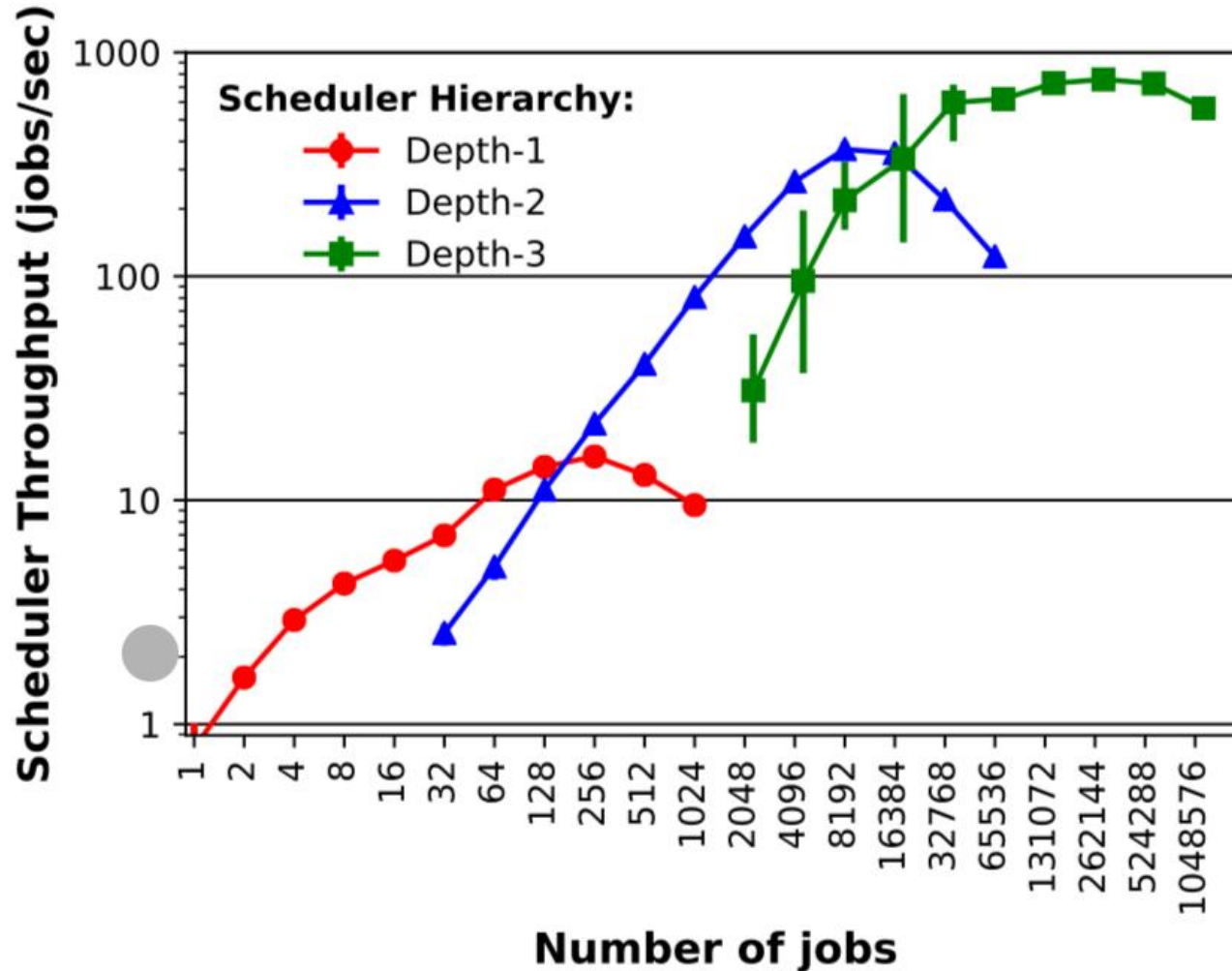


Choose an algorithm for each level (instance, or sub-graph) in the hierarchy.



4. The design enables increased scalability and throughput

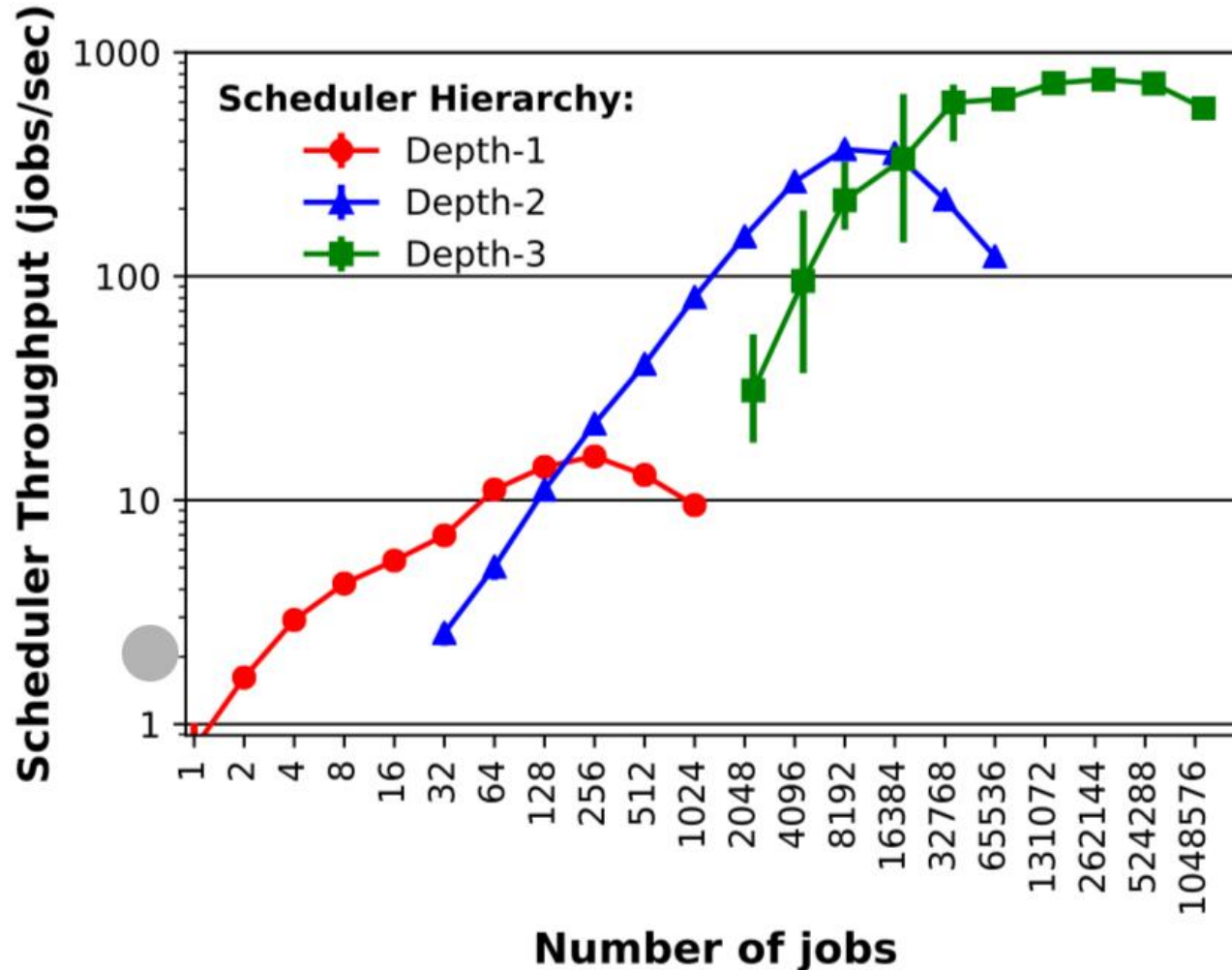
Fractal scheduling enables high throughput



- Flux addresses demand for high throughput

4. The design enables increased scalability and throughput

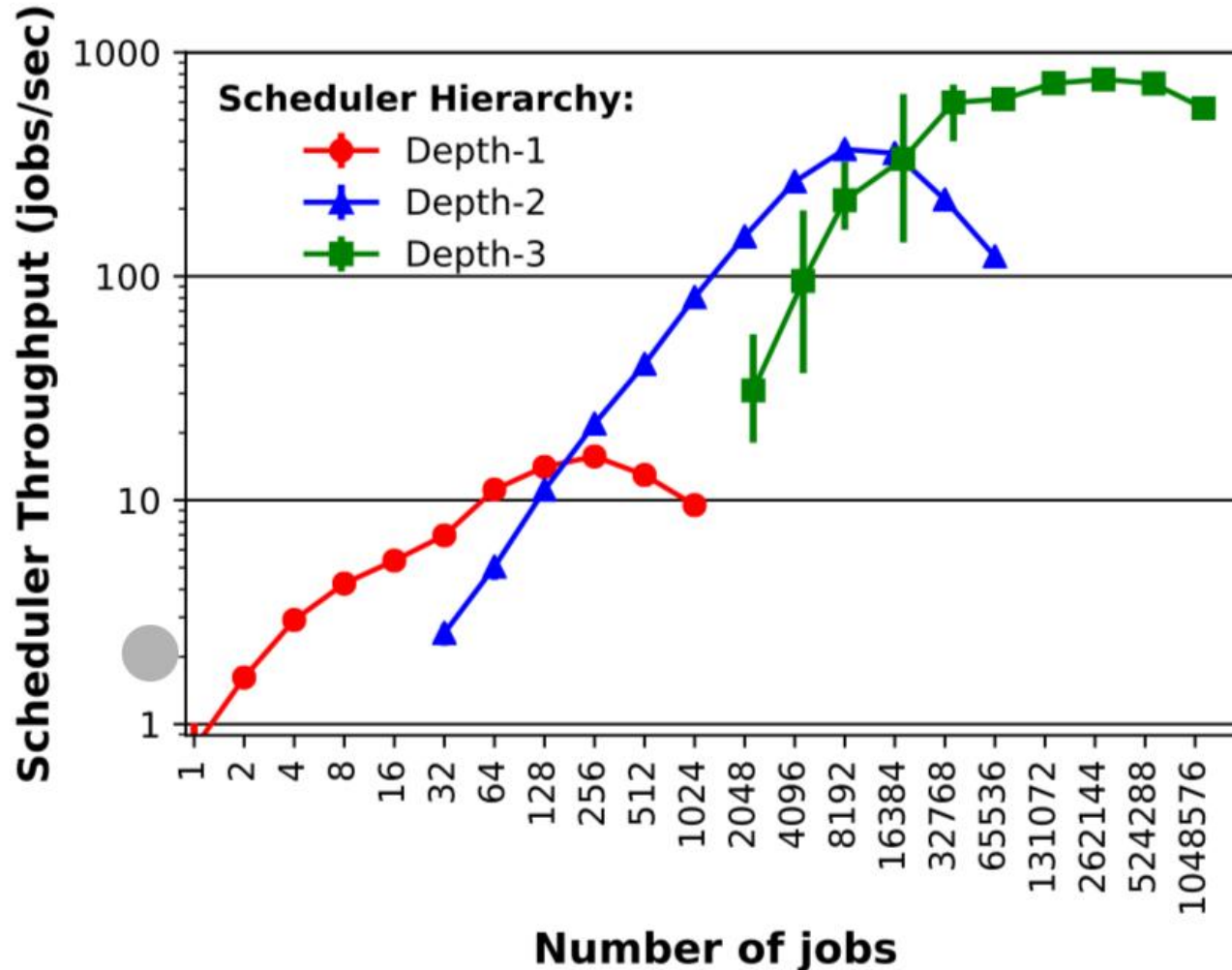
Fractal scheduling enables high throughput



- Flux addresses demand for high throughput
- A hierarchy of Flux instances managing jobs increases throughput.

4. The design enables increased scalability and throughput

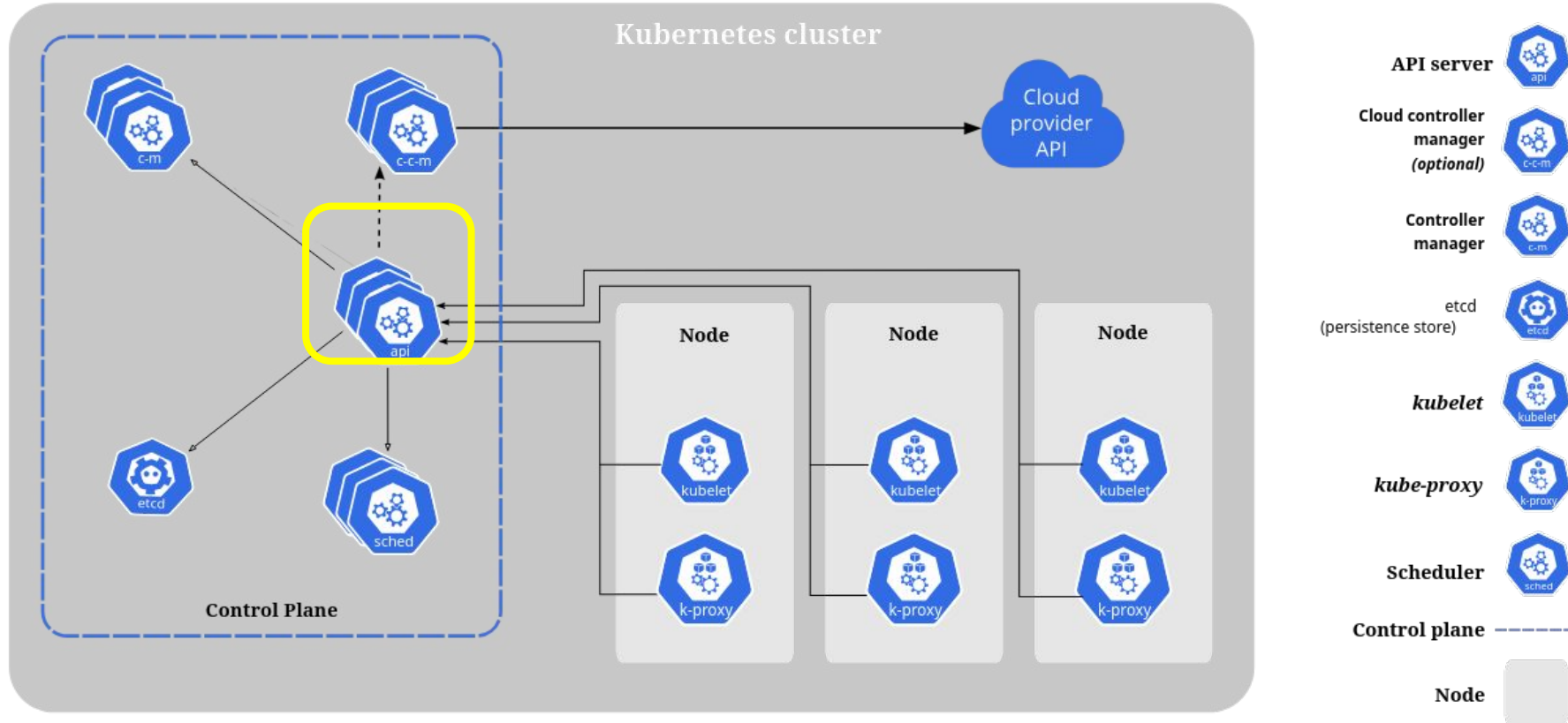
Fractal scheduling enables high throughput



- Flux addresses demand for high throughput
- A hierarchy of Flux instances managing jobs increases throughput.
- We don't face the bottleneck of one, central scheduler

4. The design enables increased scalability and throughput

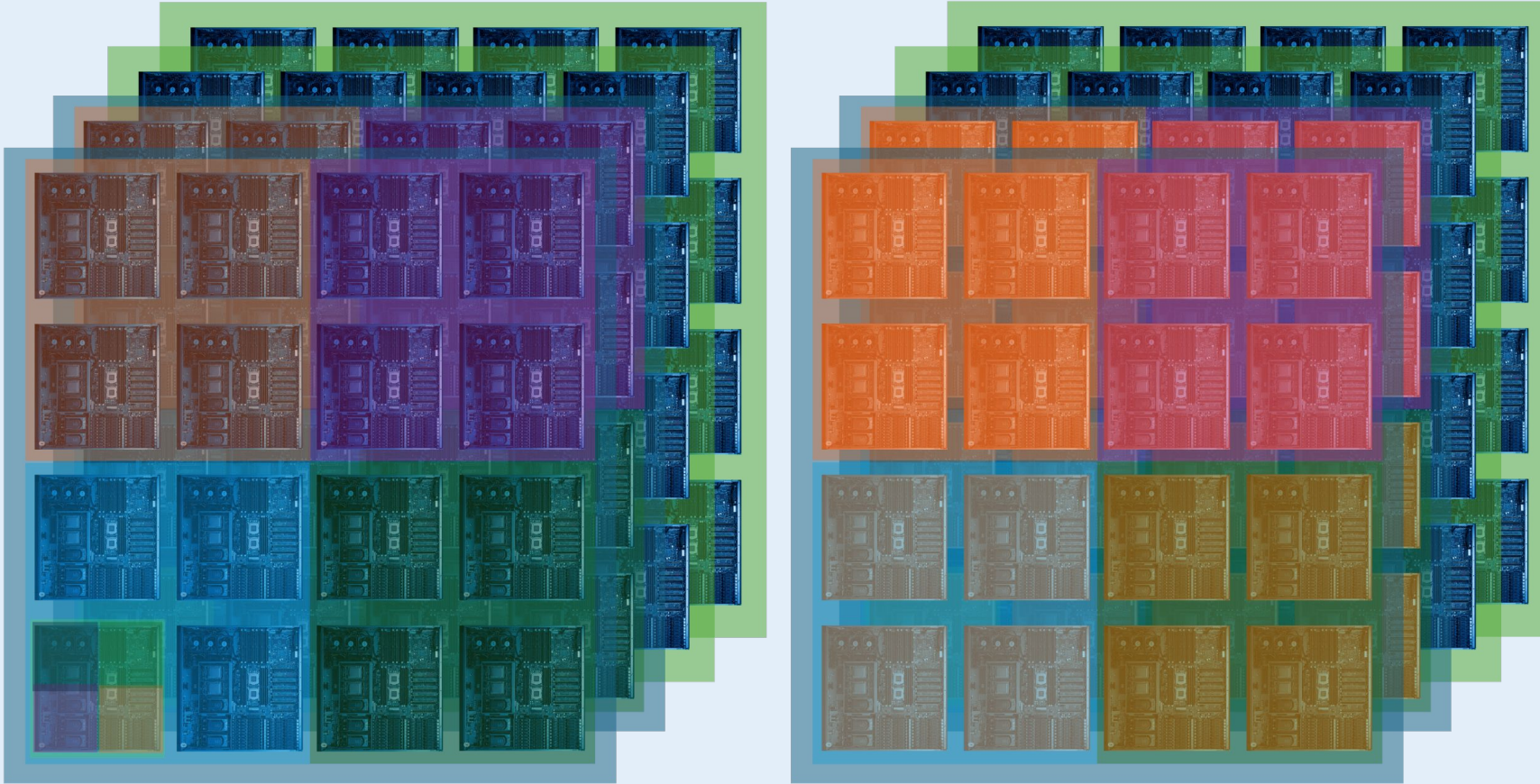
Fractal scheduling enables high throughput



Deploy Flux in Kubernetes - jobs avoid Kubernetes scheduler – *no etcd or API bottlenecks*

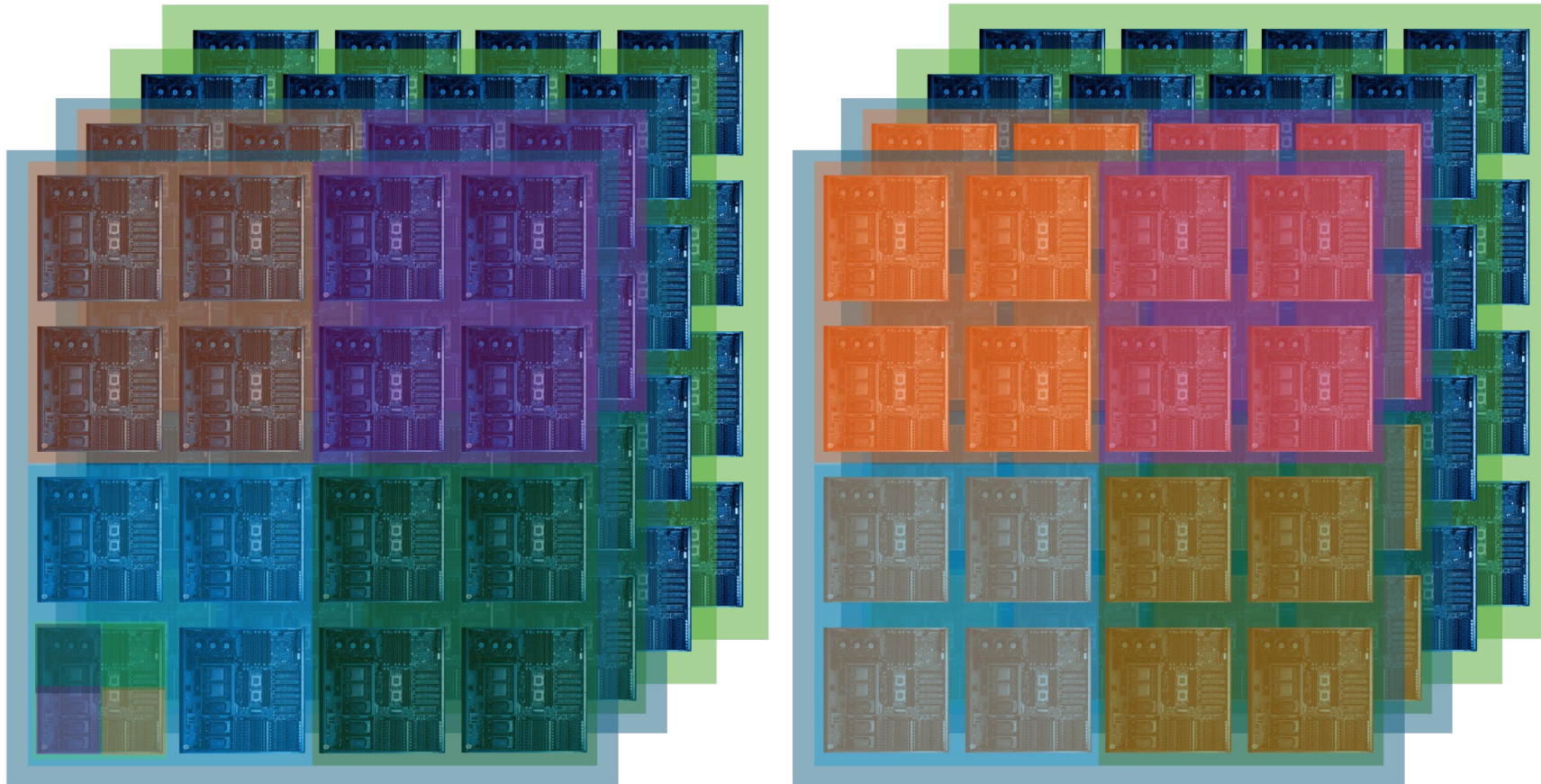
Exploration of a Flux hierarchy

Flux level 0 (system)



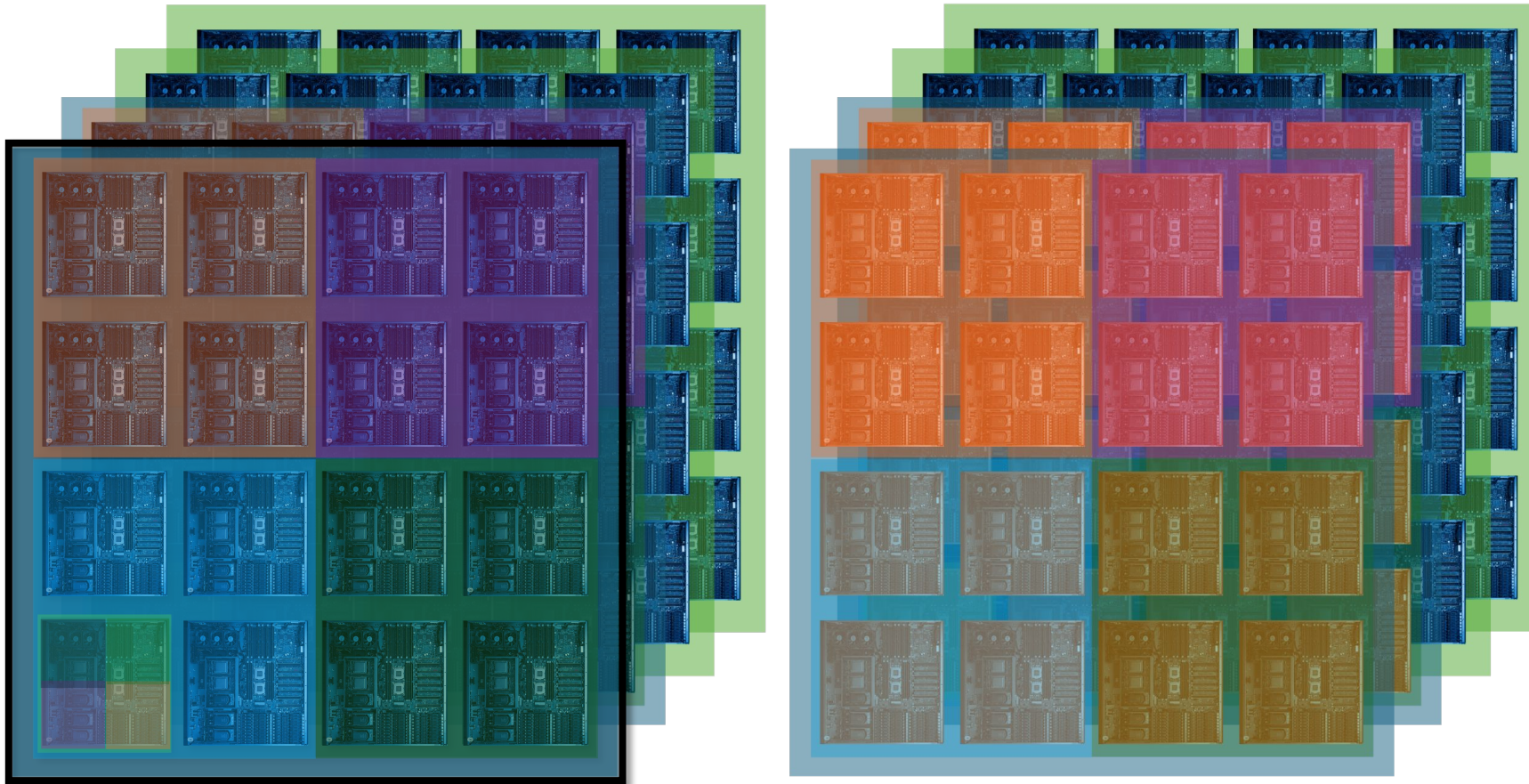
match-policy = "first"
queue-policy = "easy"

Flux level 1



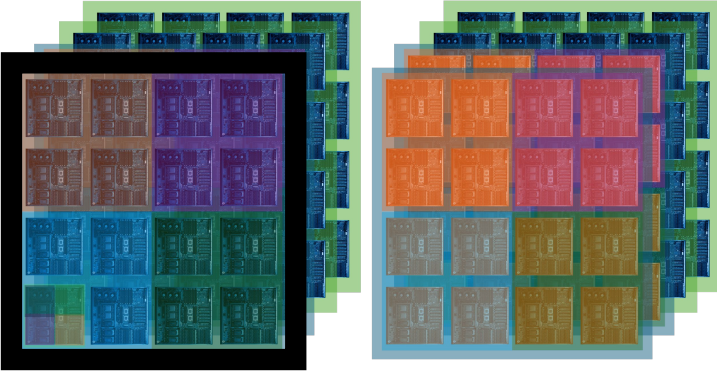
```
flux batch -N 64 -x level1.sh  
match-policy = "first"  
queue-policy = "easy"
```


Flux level 1



```
flux batch -N 64 -x level1.sh  
match-policy = "first"  
queue-policy = "easy"
```

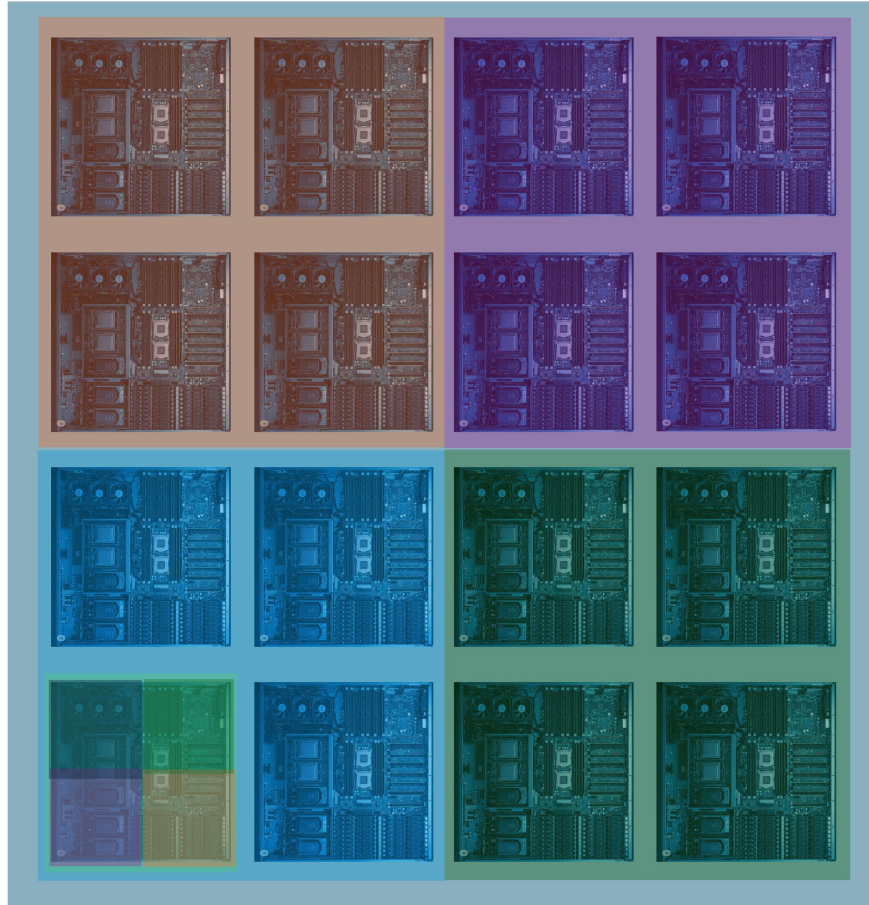
Flux level 1



Level 0: 128 nodes

You are
here

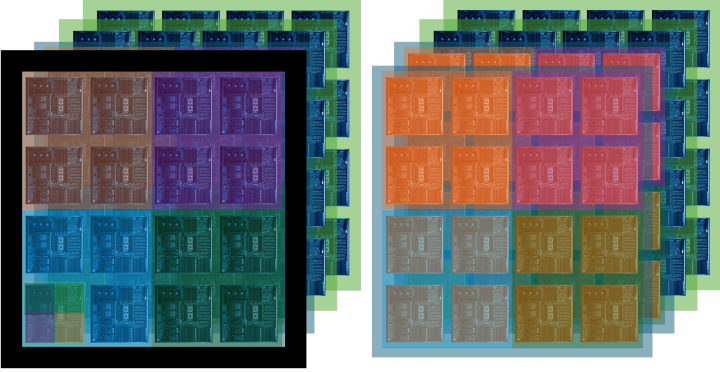
Level 1



match-policy = "first"
queue-policy = "easy"

```
for i in {1..16}; do  
    flux batch -N 4  
    level2.sh  
done
```

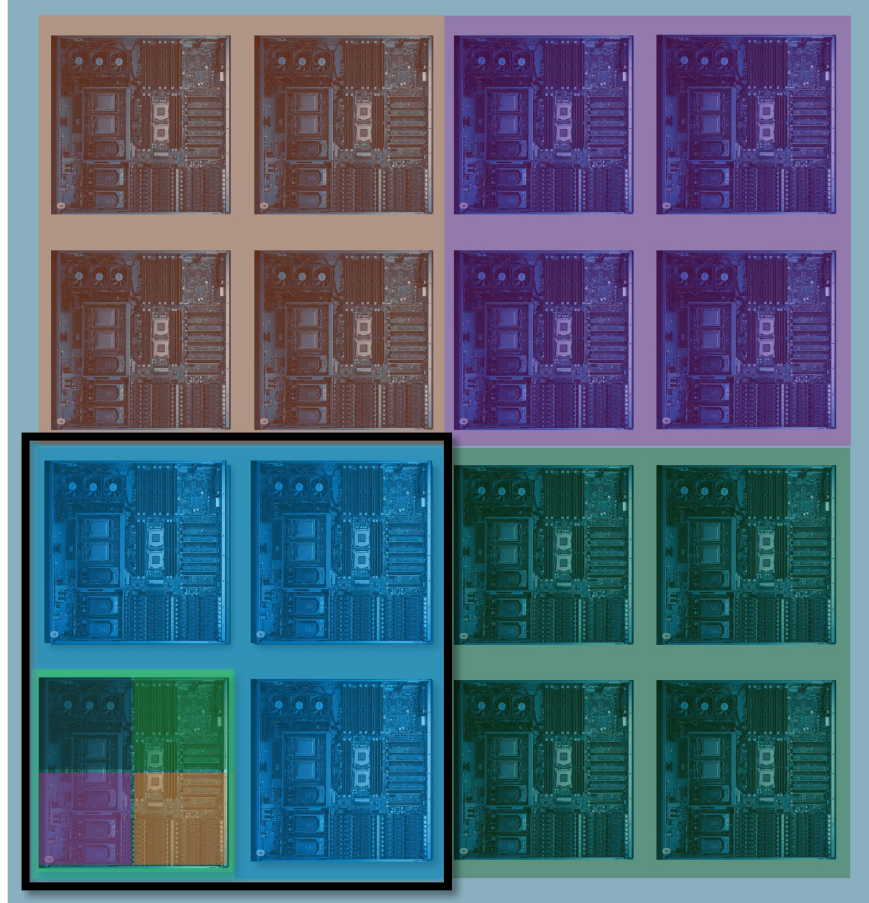

Flux level 1



Level 0: 128 nodes

You are
here

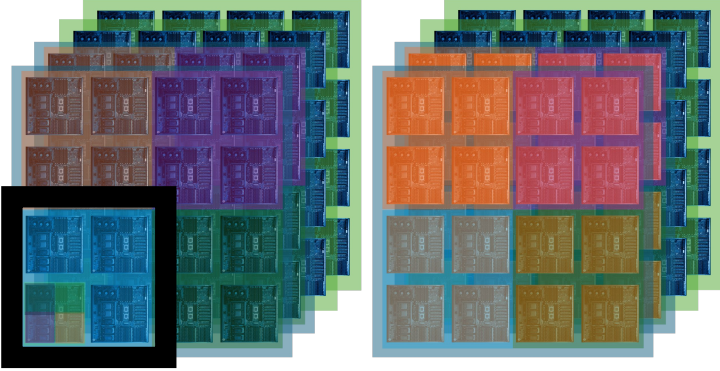
Level 1



match-policy = "first"
queue-policy = "easy"

```
for i in {1..16}; do  
    flux batch -N 4  
    level2.sh  
done
```

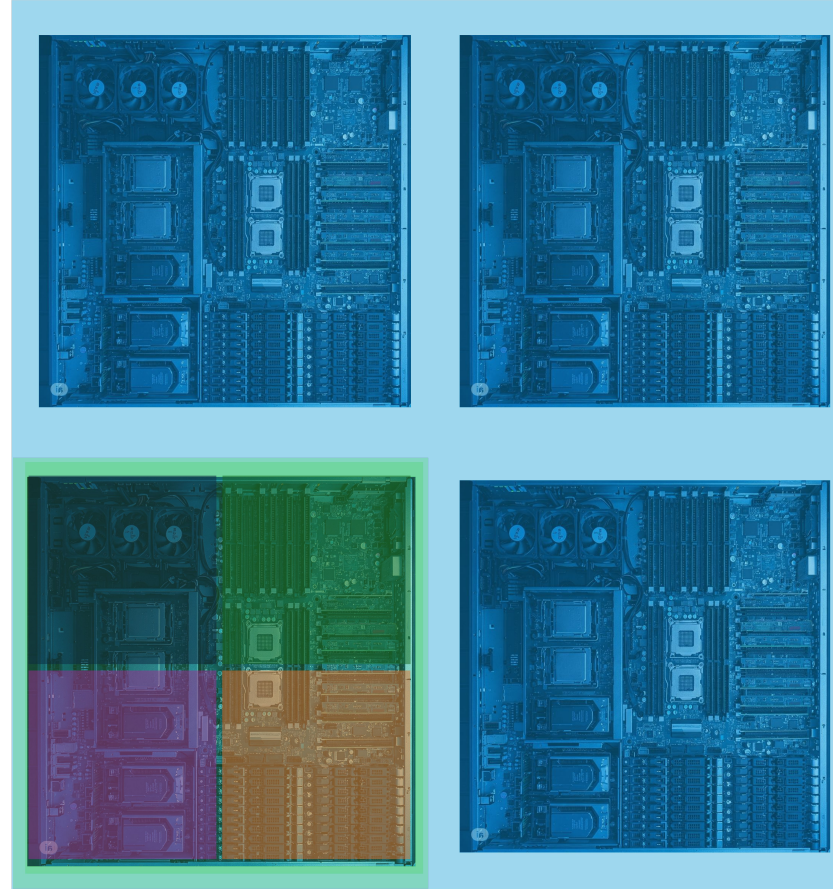
Flux level 2



Level 0: 128 nodes

Level 1: 16 nodes

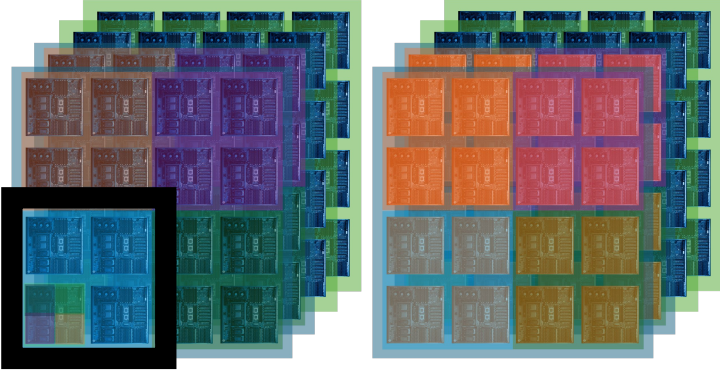
You are here Level 2



```
for i in {1..4}; do  
    flux batch -N 1  
    level13.sh  
done
```

```
match-policy = "lonodex"  
queue-policy = "fcfs"
```

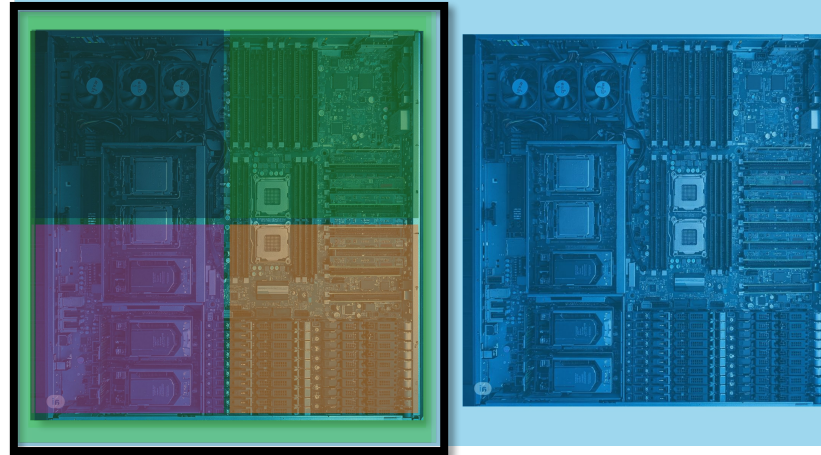
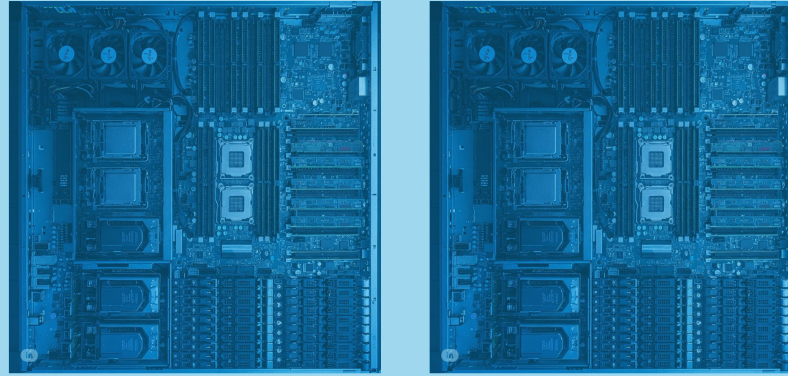

Flux level 2



Level 0: 128 nodes

Level 1: 16 nodes

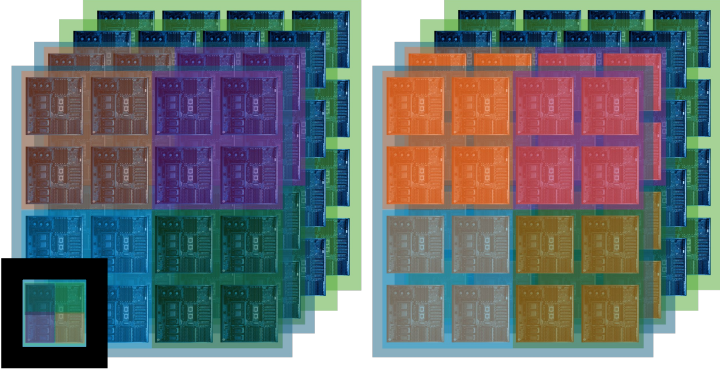
You are here Level 2



```
for i in {1..4}; do  
    flux batch -N 1  
    level13.sh  
done
```

```
match-policy = "lonodex"  
queue-policy = "fcfs"
```

Flux level 3

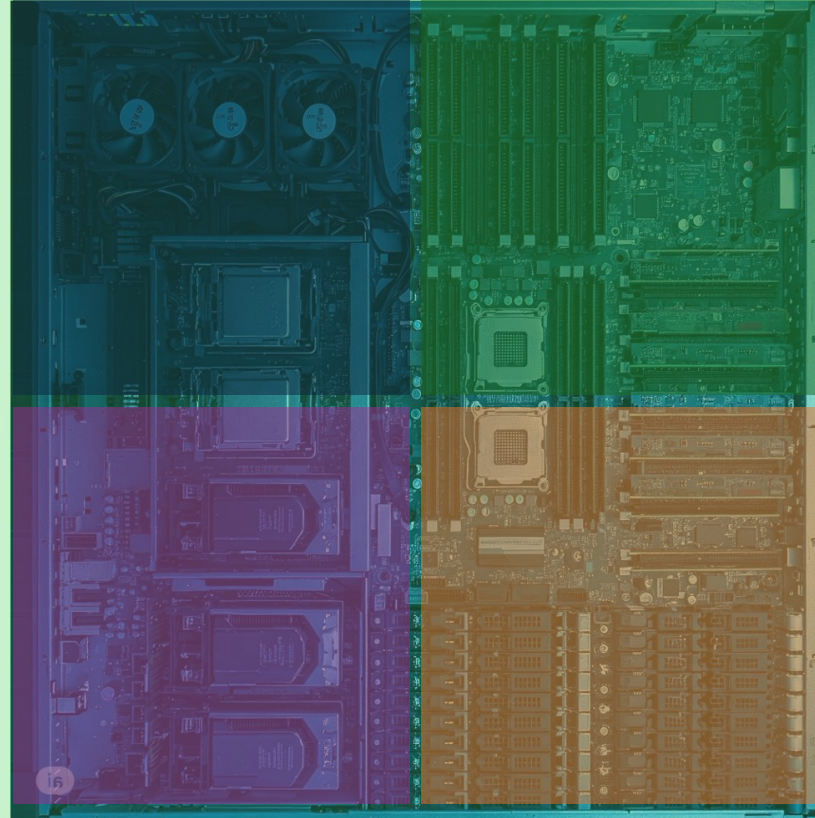


Level 0: 128 nodes

Level 1: 16 nodes

Level 2: 4 nodes

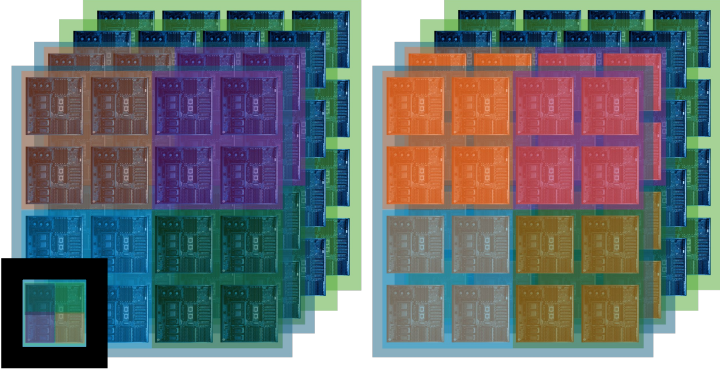
You are
here Level 3



```
for i in {1..4}; do  
flux batch -n 16 -g 2  
level4.sh  
done
```

sched-simple

Flux level 3

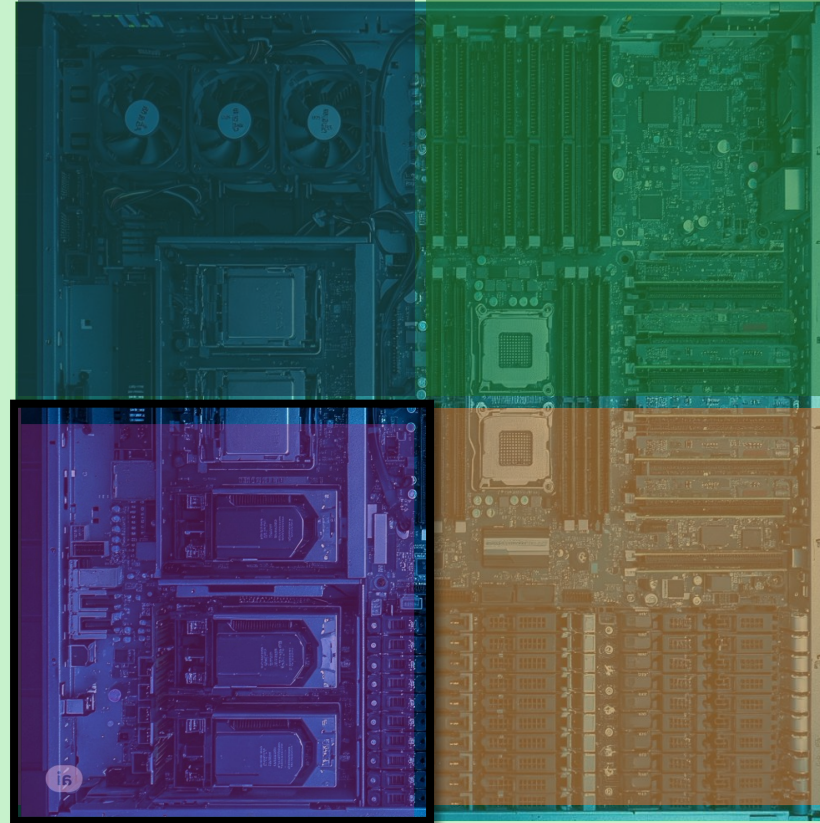


Level 0: 128 nodes

Level 1: 16 nodes

Level 2: 4 nodes

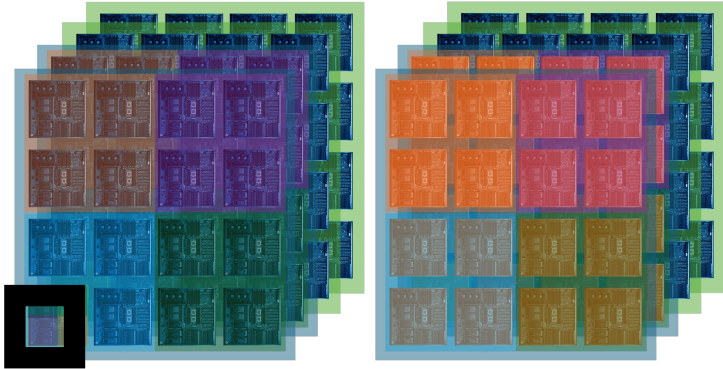
You are here
Level 3



sched-simple

```
for i in {1..4}; do  
flux batch -n 16 -g 2  
level4.sh  
done
```

Flux level 4



Level 0: 128 nodes

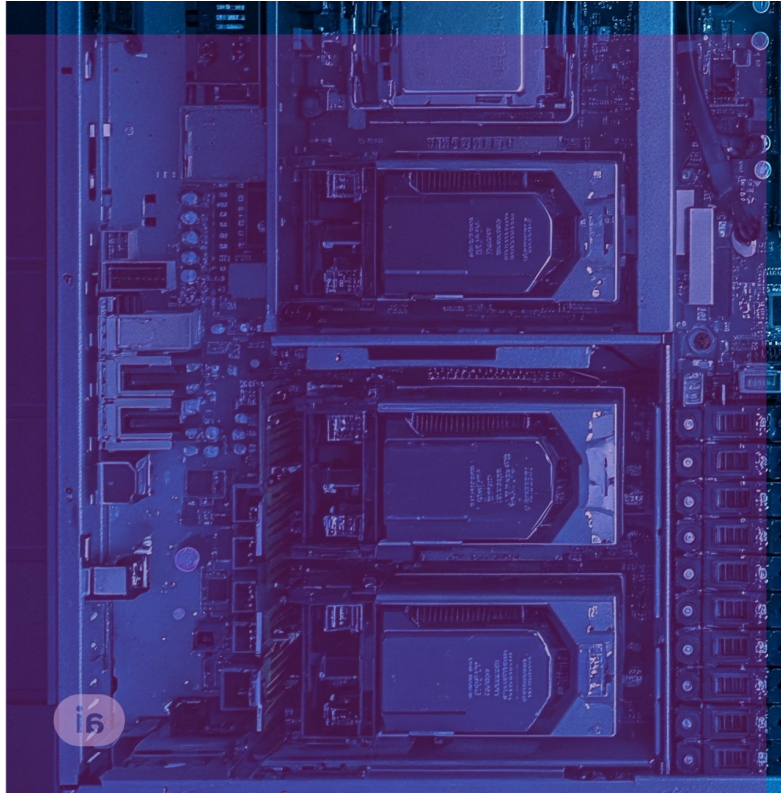
Level 1: 16 nodes

Level 2: 4 nodes

Level 3: 1 node

You are
here

Level 4



```
flux submit -cc=1-16384 -n 4 -g 1  
app.exe
```

sched-simple

How did Flux come to be?

Project Established

Initial project funding
proposal and staffing



How did Flux come to be?

Project Established

Initial project funding
proposal and staffing



How did Flux come to be?

Project Established

Initial project funding
proposal and staffing

Instance Launch

Prototype Flux
instance that could
be launched by Slurm



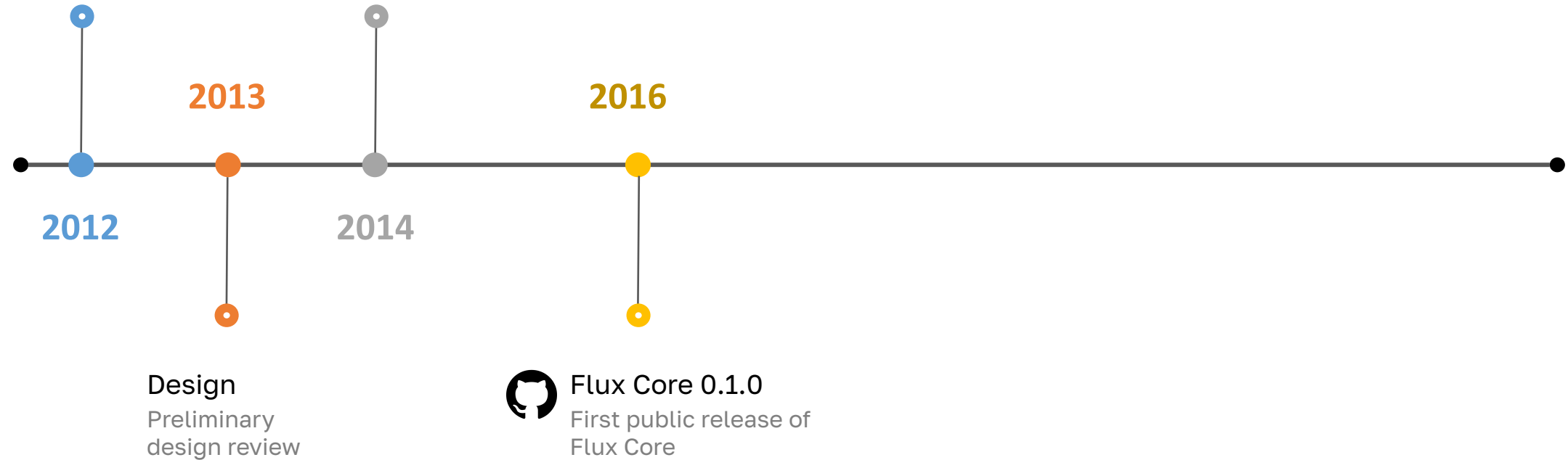
How did Flux come to be?

Project Established

Initial project funding
proposal and staffing

Instance Launch

Prototype Flux
instance that could
be launched by Slurm



How did Flux come to be?

Project Established

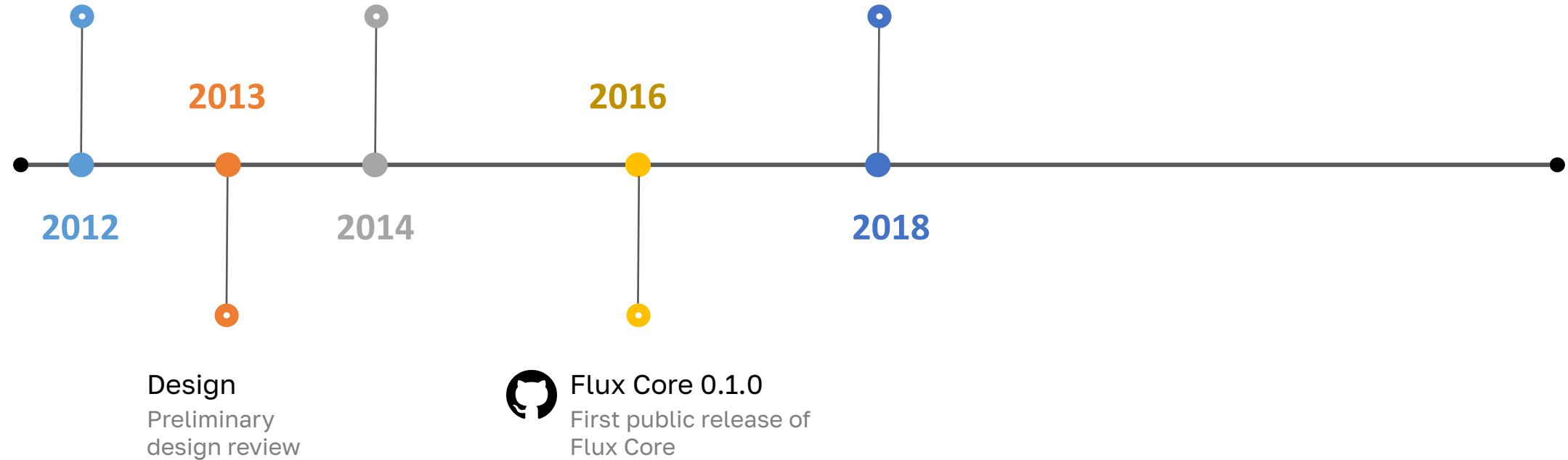
Initial project funding
proposal and staffing

Instance Launch

Prototype Flux
instance that could
be launched by Slurm

Science

Flux used by Pilot2
cancer moonshot
ensemble



How did Flux come to be?

Project Established

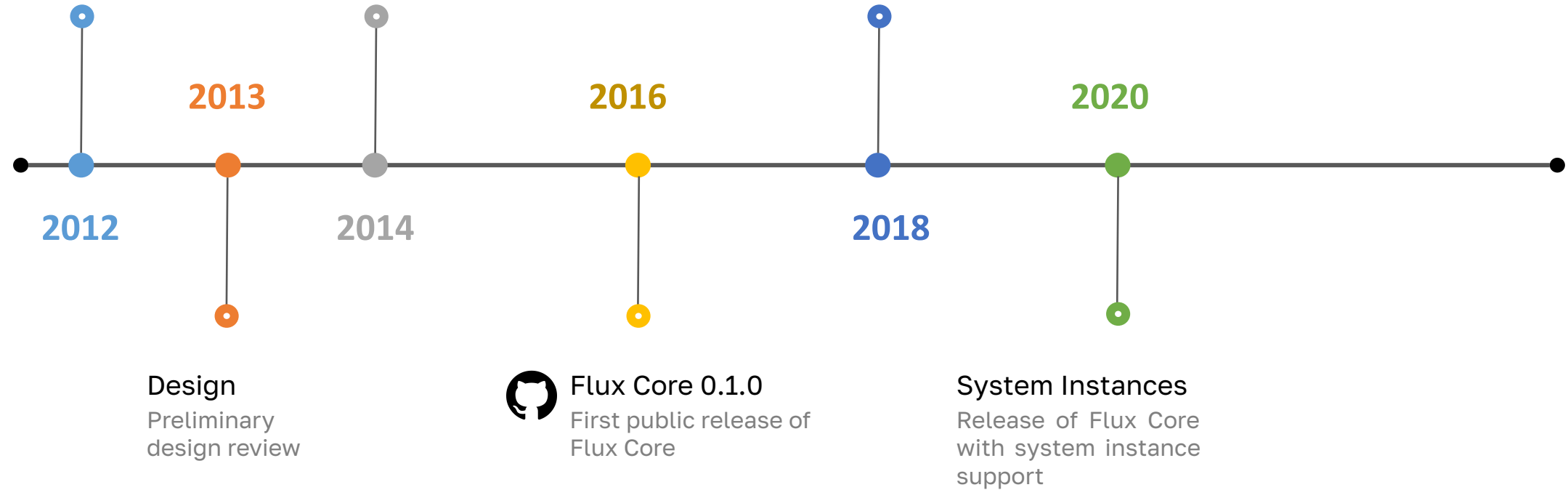
Initial project funding
proposal and staffing

Instance Launch

Prototype Flux
instance that could
be launched by Slurm

Science

Flux used by Pilot2
cancer moonshot
ensemble



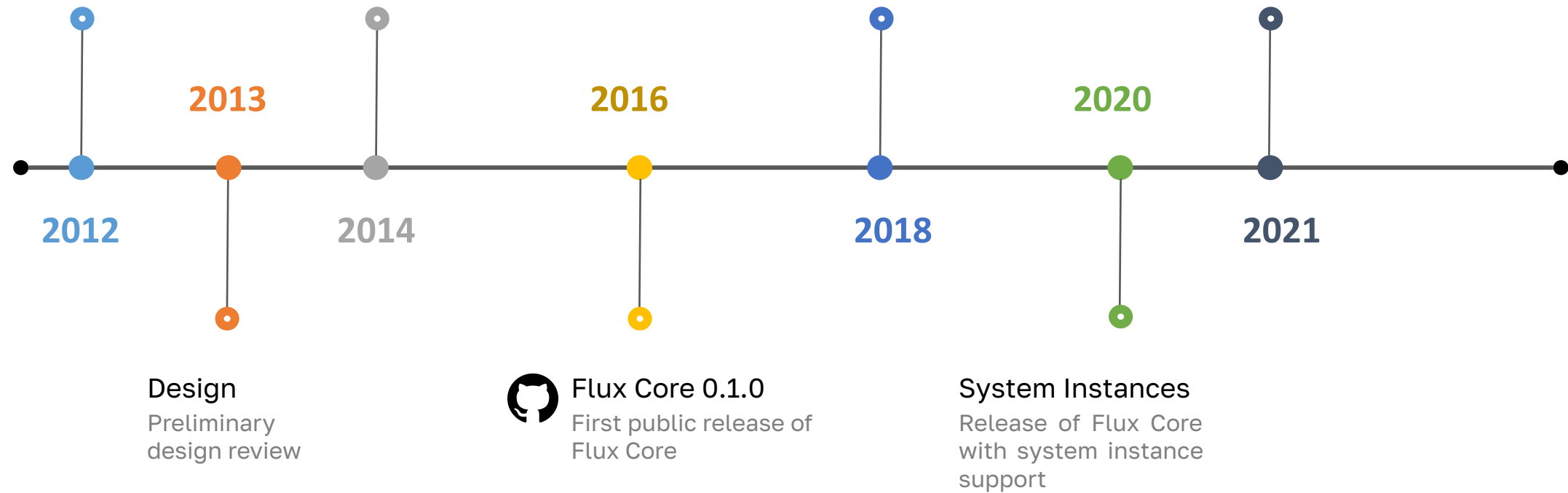
How did Flux come to be?

Project Established
Initial project funding
proposal and staffing

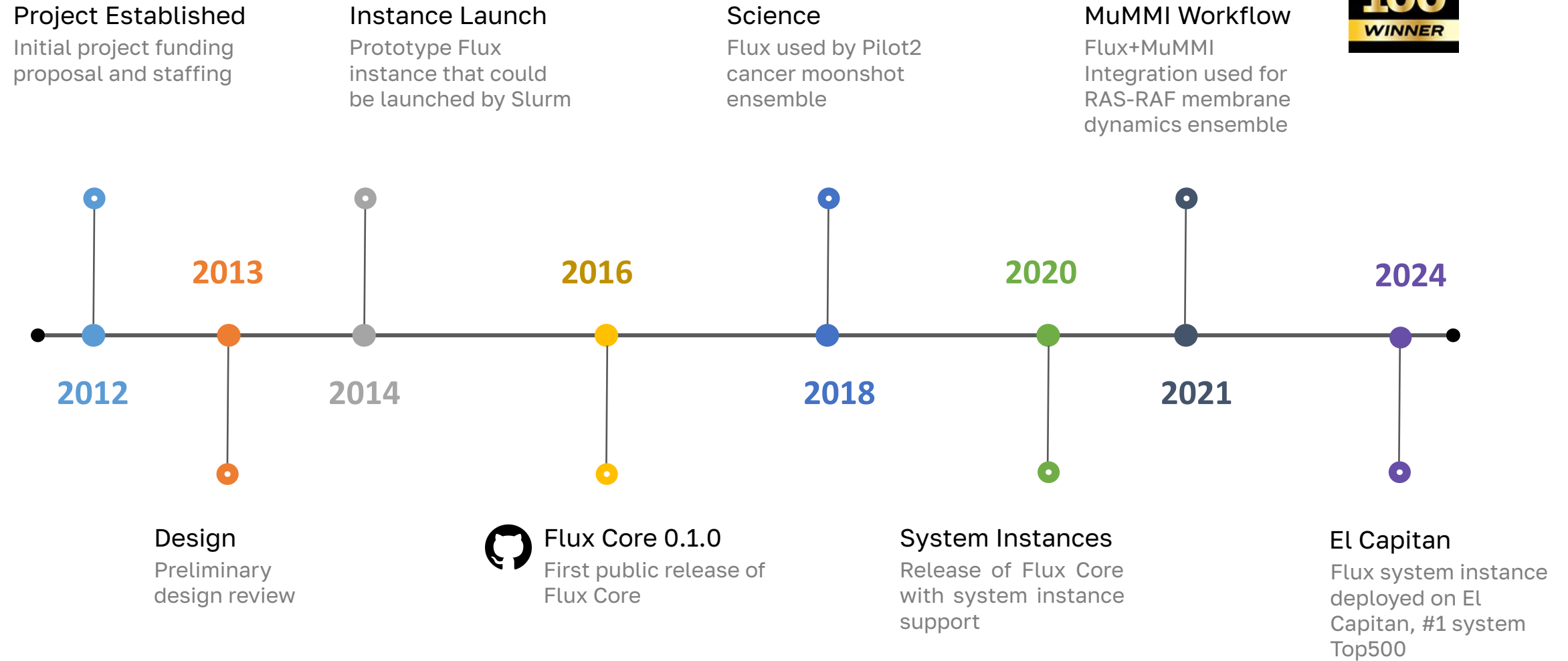
Instance Launch
Prototype Flux
instance that could
be launched by Slurm

Science
Flux used by Pilot2
cancer moonshot
ensemble

MuMMI Workflow
Flux+MuMMI
Integration used for
RAS-RAF membrane
dynamics ensemble



How did Flux come to be?



What features make Flux unique?



Start an instance in user-space, under another manager

1. Install Flux (source, conda, container, etc.)
2. Then:

```
salloc -N2 --exclusive  
srun -v --mpi=pmi2 -N2 --pty /opt/conda/bin/flux -v start
```

Video tutorial: [Running Flux Under Slurm](#)

Drain and undrain a node as a user (no admin)

```
$ flux resource list
```

STATE	NNODES	NCORES	NGPUS	ODELIST
free	1	10	0	flux-node
allocated	0	0	0	
down	0	0	0	

```
$ flux resource drain flux-node "I am demonstrating drain"
```

```
$ flux resource list
```

STATE	NNODES	NCORES	NGPUS	ODELIST
free	0	0	0	
allocated	0	0	0	
down	1	10	0	flux-node

```
$ flux resource undrain flux-node "I am ready to use it again"
```

```
$ flux resource list
```

STATE	NNODES	NCORES	NGPUS	ODELIST
free	1	10	0	flux-node
allocated	0	0	0	
down	0	0	0	

Move files between nodes without a shared filesystem

```
singularity pull docker://vsoch/really-big-container:latest  
flux archive create --name container --dir $(pwd) really-big-container_latest.sif  
flux exec --exclude 0 --rank all flux archive extract --name container
```


Subscribe to Job events

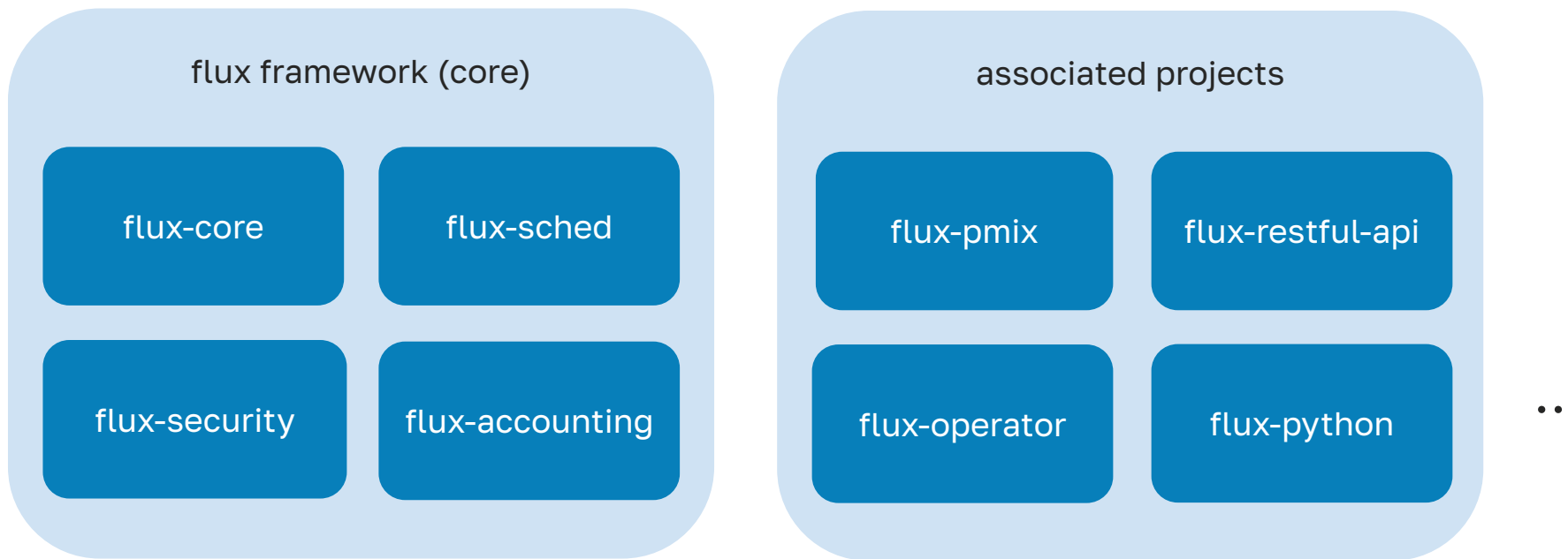
```
import flux

handle = flux.Flux()
consumer = flux.job.JournalConsumer(handle).start()
while True:
    event = consumer.poll(timeout=-1)
    print(event)
```

Flux Architecture and job lifecycle overview

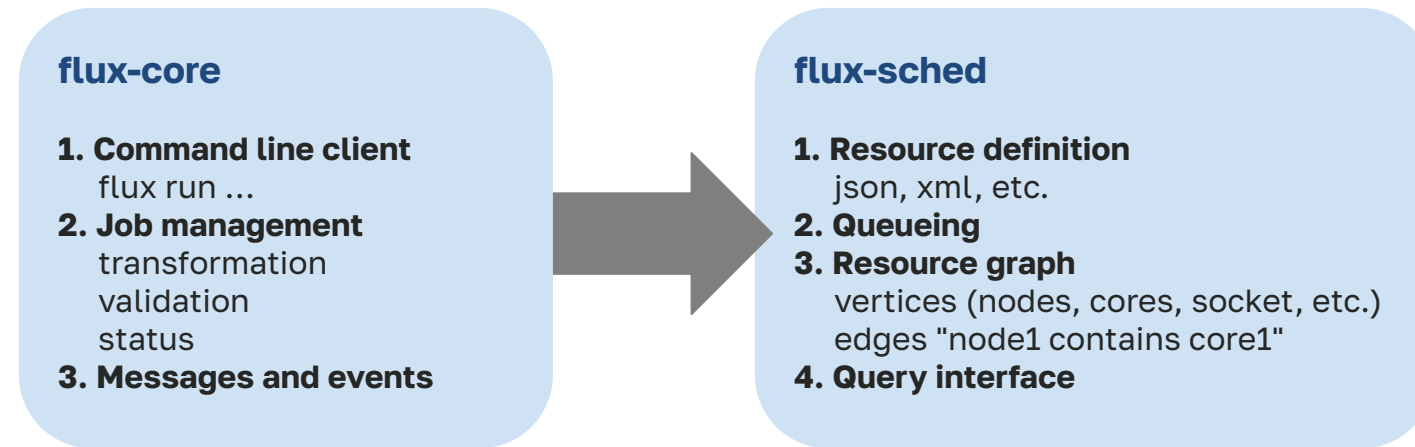


Flux Framework is a suite of projects



We will show some of these more in detail later in the presentation!

The projects work together to encompass "Flux"



flux-core is what you interact with to manage jobs

flux-core

1. Command line client

flux run ...

2. Job management

transformation

validation

status

3. Messages and events

"I want to run a job with these resources"

"I want to watch and respond to job events"

"I want to cancel a job"

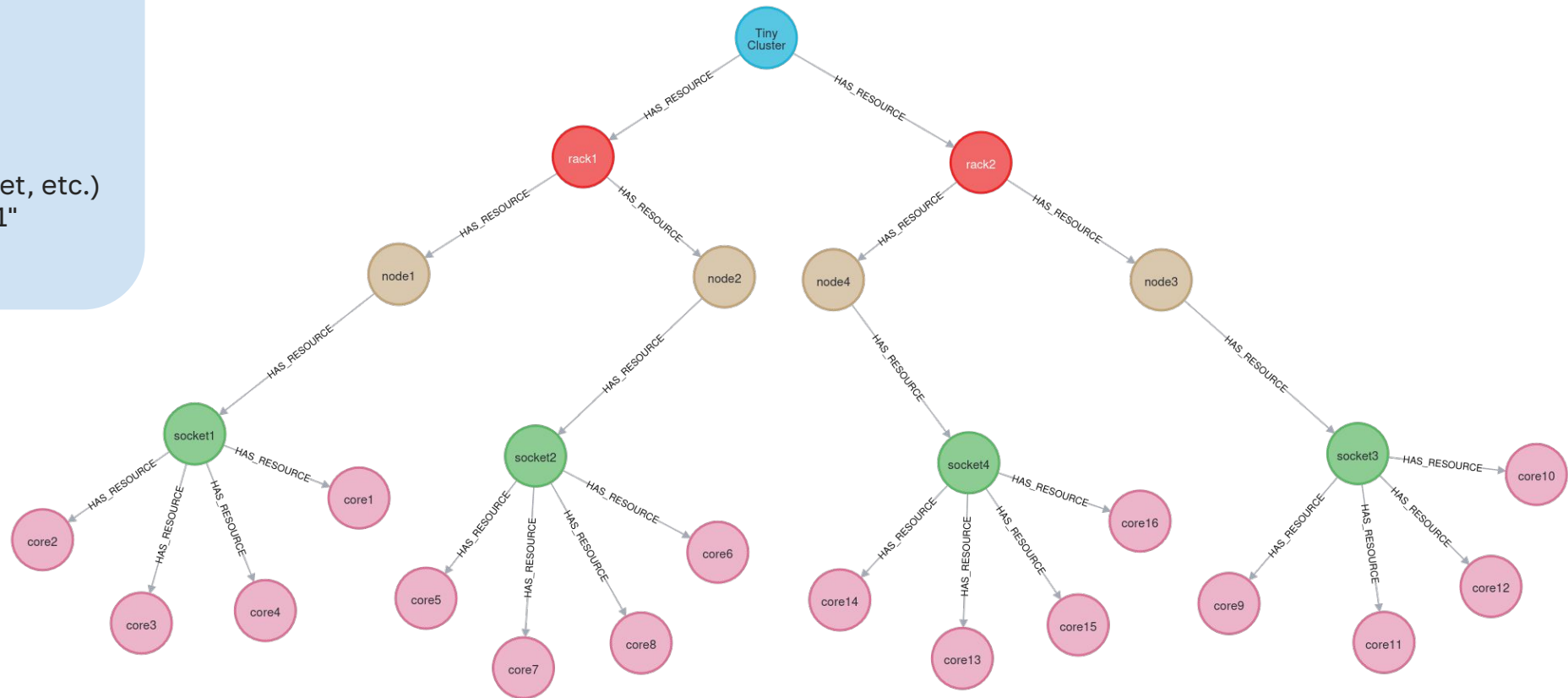
"I want to check my job status"

"I want to write a plugin to validate jobs"

flux-sched is modeling your system as a graph

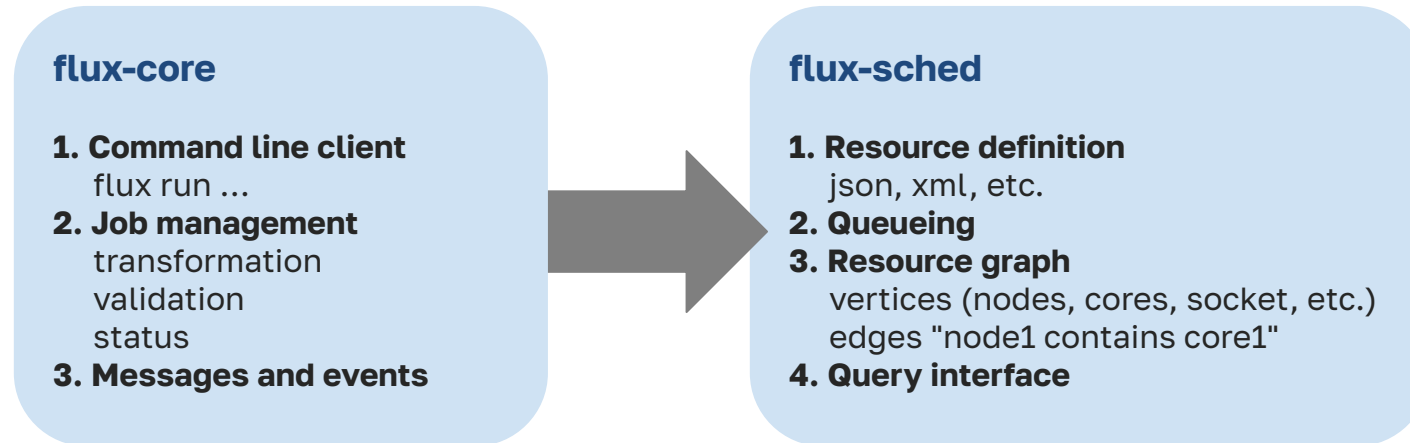
flux-sched

1. Resource definition
json, xml, etc.
2. Queueing
3. Resource graph
vertices (nodes, cores, socket, etc.)
edges "node1 contains core1"
4. Query interface

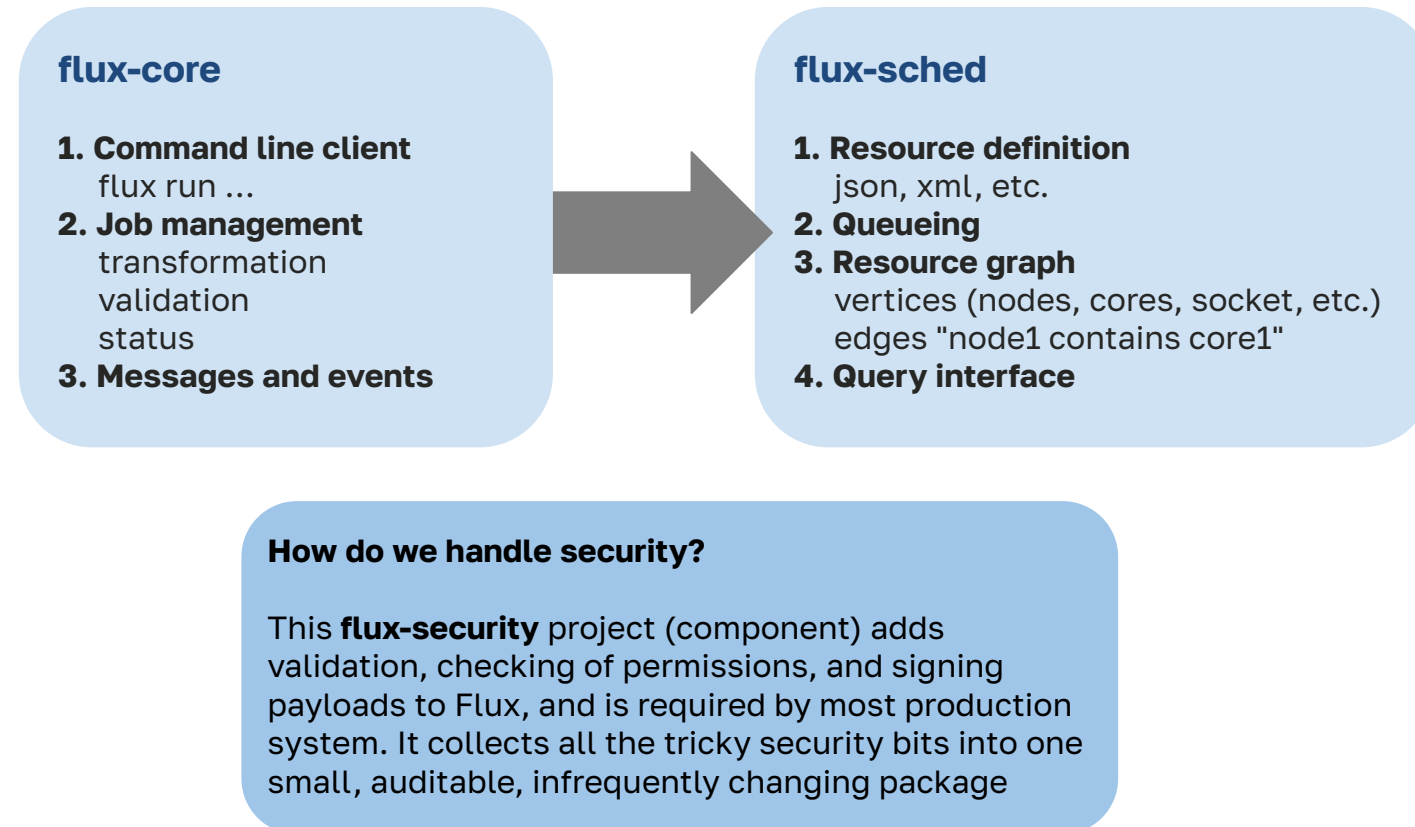


The projects work together to encompass "Flux"

When can we
allocate this job?



The projects work together to encompass "Flux"



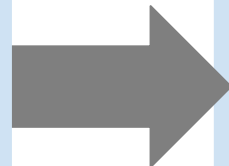
The projects work together to encompass "Flux"

How do we handle priority among many users?

This **flux-accounting** project (component) adds a database and algorithms to Flux to ensure that scheduling is fair on a shared resource.

flux-core

1. **Command line client**
flux run ...
2. **Job management**
transformation
validation
status
3. **Messages and events**



flux-sched

1. **Resource definition**
json, xml, etc.
2. **Queueing**
3. **Resource graph**
vertices (nodes, cores, socket, etc.)
edges "node1 contains core1"
4. **Query interface**

How do we handle security?

This **flux-security** project (component) adds validation, checking of permissions, and signing payloads to Flux, and is required by most production system. It collects all the tricky security bits into one small, auditable, infrequently changing package

What is the lifecycle of a job?

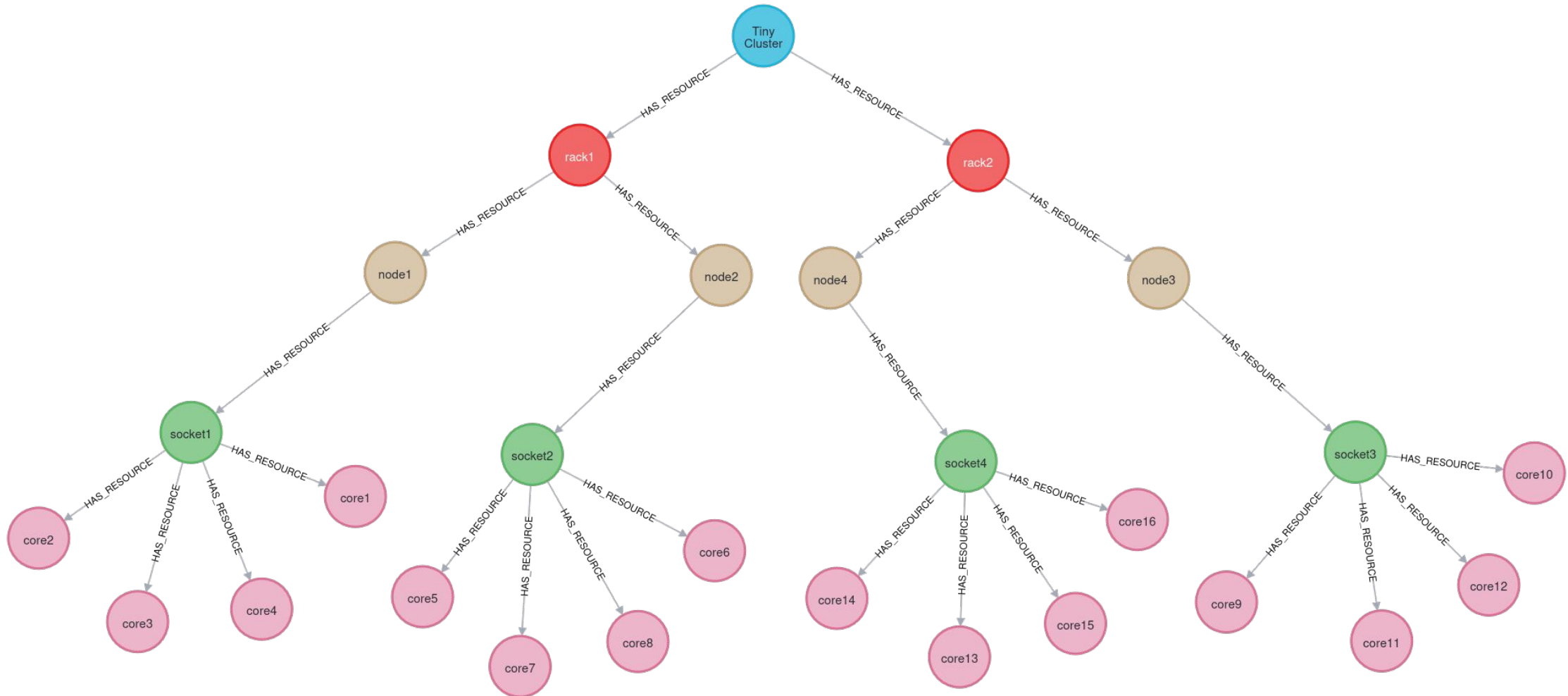
My HPC Job



The Science!

The system is represented as a graph

- ❑ Express the resource requirements of a program to the scheduler
- ❑ Provide attributes like arguments, run time, and task layout, to be considered
- ❑ Represents relationships between resources



1. I request my application to run

"I want 4 nodes, 384 cores"

```
flux run -N 4 -n 384 ./app
```

"Block terminal and stream output"

1. I request my application to run

"I want 4 nodes, 384 cores"

```
flux run -N 4 -n 384 ./app
```

"Block terminal and stream output"

```
flux submit -N 4 -n 384 ./app
```

"The same, but not blocking"

1. I request my application to run

"I want 4 nodes, 384 cores"

```
flux run -N 4 -n 384 ./app
```

"Block terminal and stream output"

```
flux submit -N 4 -n 384 ./app
```

"The same, but not blocking"

```
flux alloc -N 4 -n 384 bash
```

"Make me an interactive allocation"

```
flux batch -N 4 -n 384 --wrap bash
```

"Make me a new instance, run script"

2. My request is transformed into a "jobspec"

A graph-based specification for work

```
flux run -N 1 -n 18 ./app
```



```
1  version: 1
2  resources:
3    - type: cluster
4      count: 1
5      with:
6        - type: rack
7          count: 2
8          with:
9            - type: slot
10              label: myslot
11              count: 3
12              with:
13                - type: node
14                  count: 1
15                  with:
16                    - type: socket
17                      count: 2
18                      with:
19                        - type: core
20                          count: 18
21
22  # a comment
23  attributes:
24    system:
25      duration: 3600
26  tasks:
27    - command: app
28      slot: myslot
29      count:
30        per_slot: 1
```

2. My request is transformed into a "jobspec"

The jobspec has resources and tasks

```
flux run -N 1 -n 18 ./app
```



- **slot** is the only non-physical resource type
 - Represent a schedulable place where program process or processes will be spawned and contained

```
1  version: 1
2  resources:
3    - type: cluster
4      count: 1
5      with:
6        - type: rack
7          count: 2
8          with:
9            - type: slot
10              label: myslot
11              count: 3
12              with:
13                - type: node
14                  count: 1
15                  with:
16                    - type: socket
17                      count: 2
18                      with:
19                        - type: core
20                          count: 18
21
22  # a comment
23  attributes:
24    system:
25      duration: 3600
26  tasks:
27    - command: app
28      slot: myslot
29      count:
30        per_slot: 1
```

2. My request is transformed into a "jobspec"

The jobspec has resources and tasks

```
flux run -N 1 -n 18 ./app
```



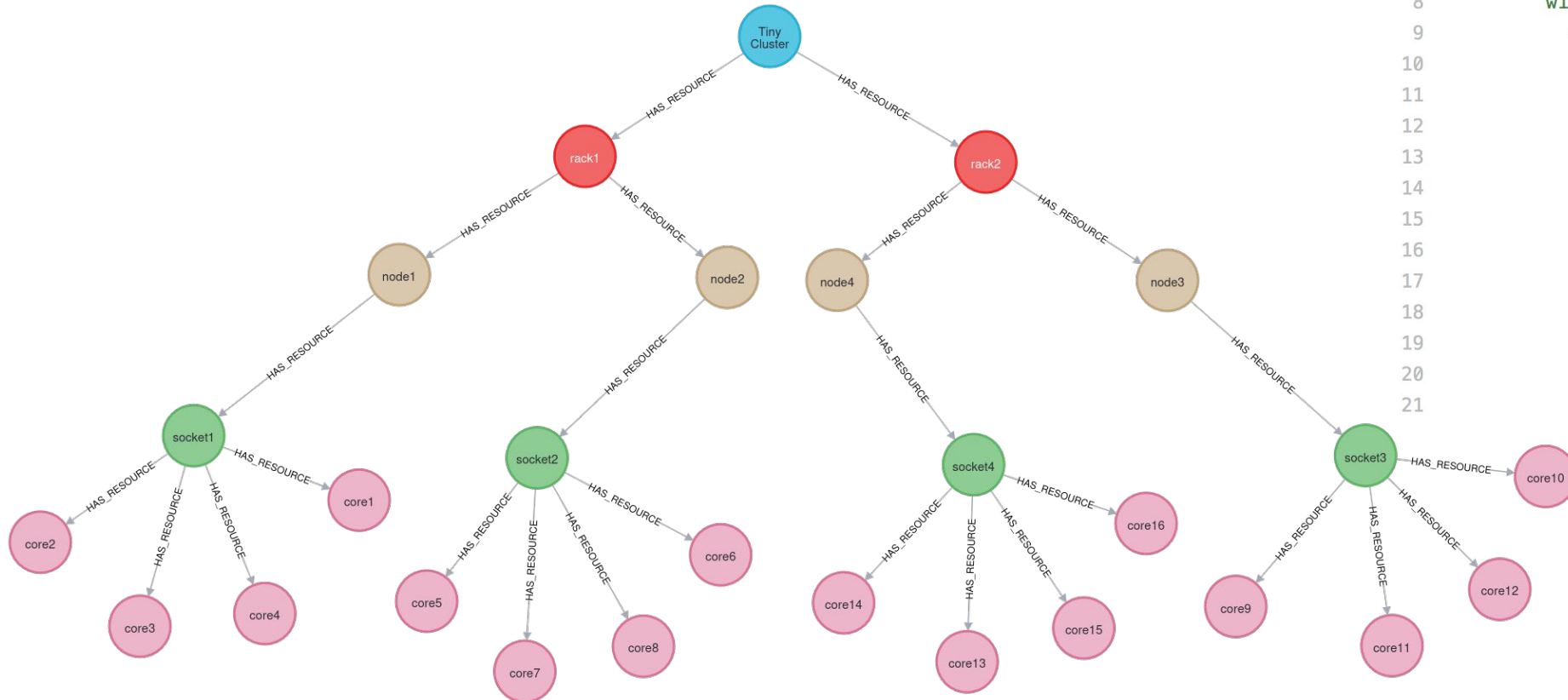
- **slot** is the only non-physical resource type
 - Represent a schedulable place where program process or processes will be spawned and contained
- Referenced from the **tasks** section

```
1  version: 1
2  resources:
3    - type: cluster
4      count: 1
5      with:
6        - type: rack
7          count: 2
8          with:
9            - type: slot
10              label: myslot
11              count: 3
12              with:
13                - type: node
14                  count: 1
15                  with:
16                    - type: socket
17                      count: 2
18                      with:
19                        - type: core
20                          count: 18
21
22  # a comment
23  attributes:
24    system:
25      duration: 3600
26  tasks:
27    - command: app
28      slot: myslot
29      count:
30        per_slot: 1
```

3. The jobspec is compared to the system graph

When can you allocate this target subgraph in the system graph?

cluster->racks[2]->slot[3]->node[1]->sockets[2]->core[16]



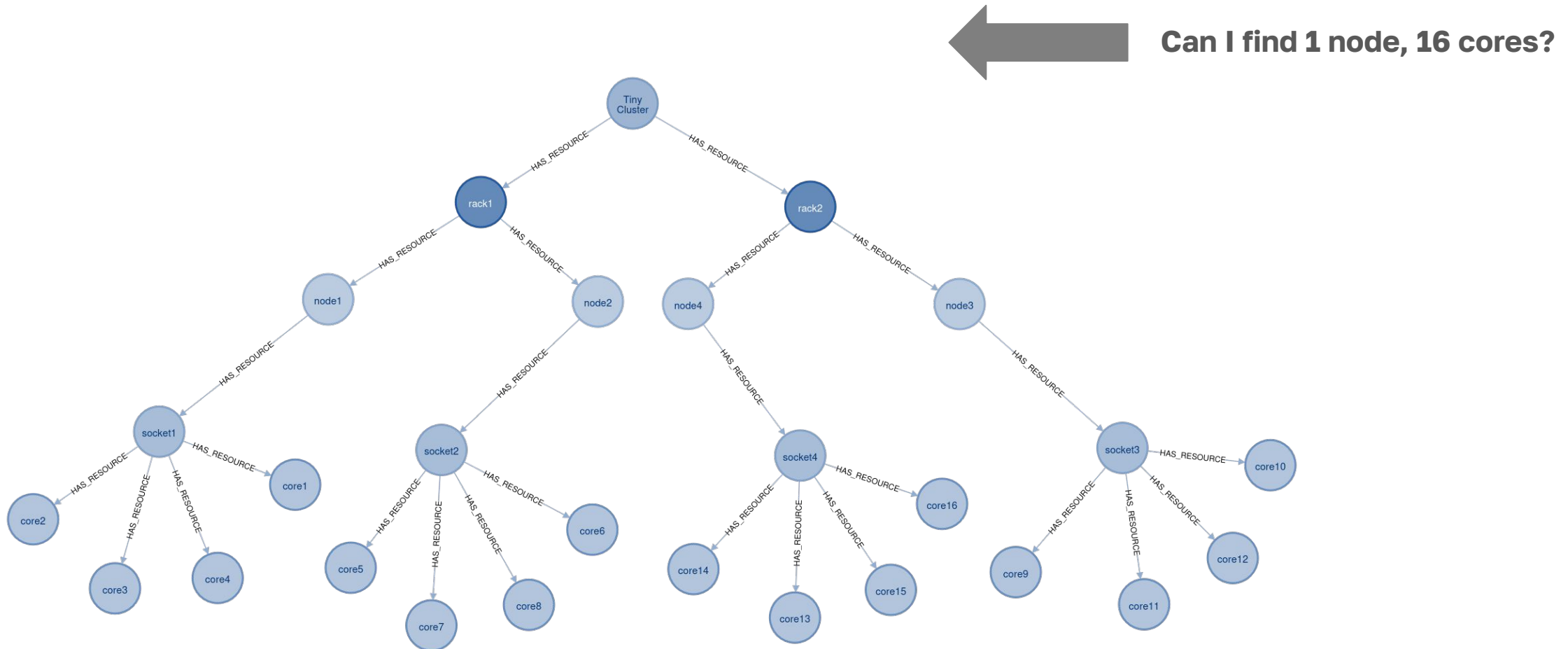
```

1  version: 1
2  resources:
3    - type: cluster
4      count: 1
5      with:
6        - type: rack
7          count: 2
8          with:
9            - type: slot
10              label: myslot
11              count: 3
12              with:
13                - type: node
14                  count: 1
15                  with:
16                    - type: socket
17                      count: 2
18                      with:
19                        - type: core
20                          count: 18
21

```

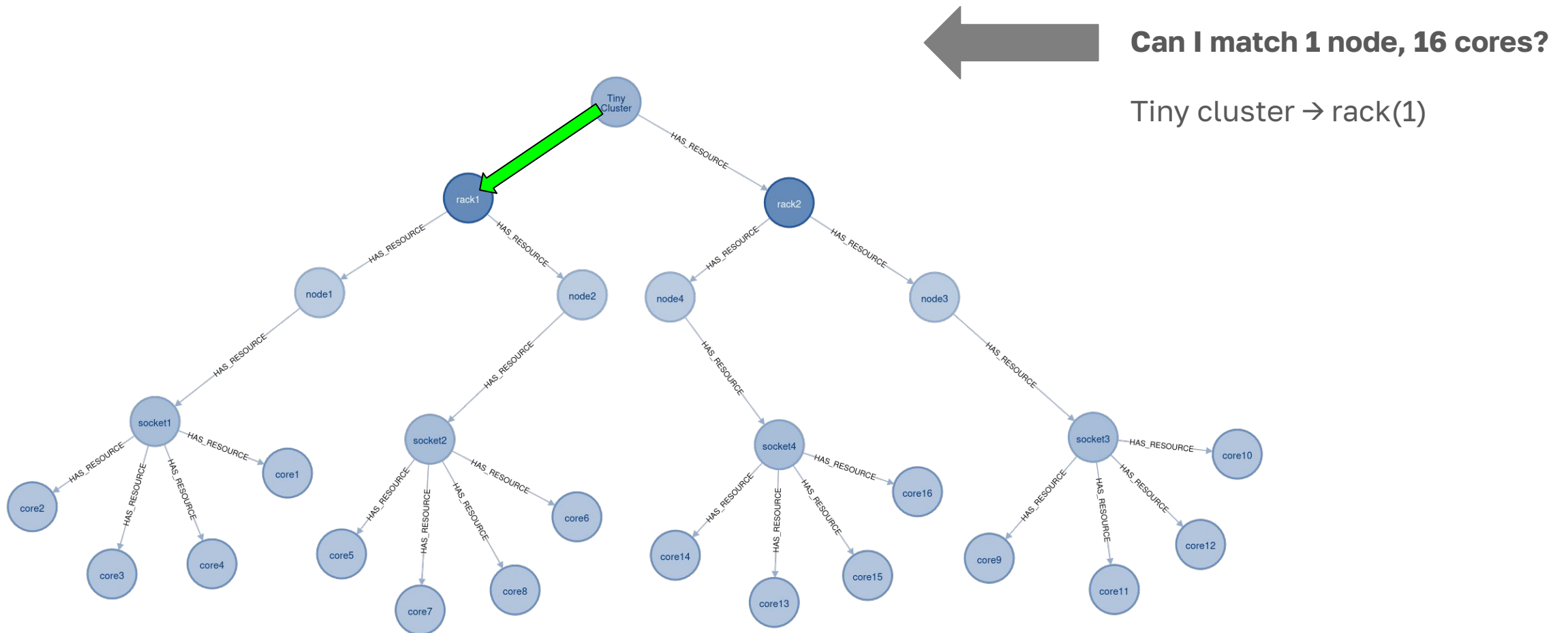
3. The jobspec is compared to the system graph

Traversal means exploring the graph for matching resource sets



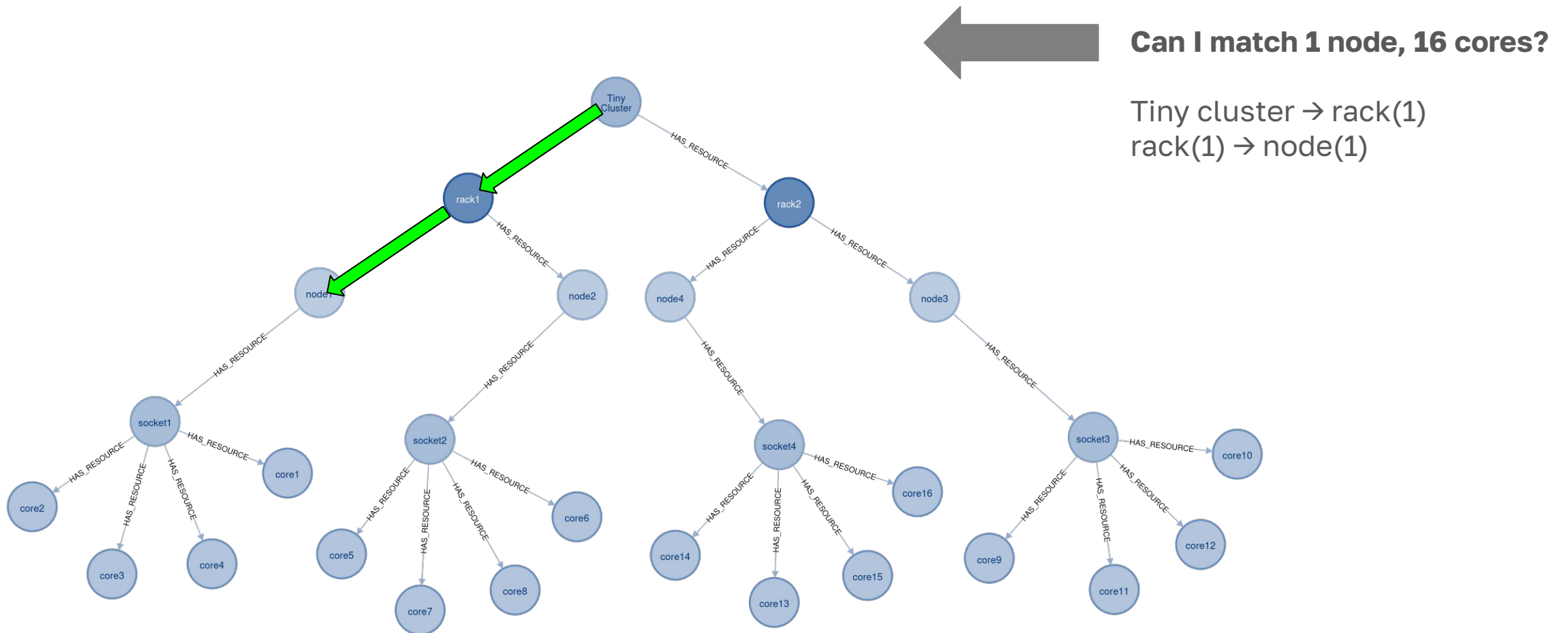
3. The jobspec is compared to the system graph

Traversal means exploring the graph for matching resource sets



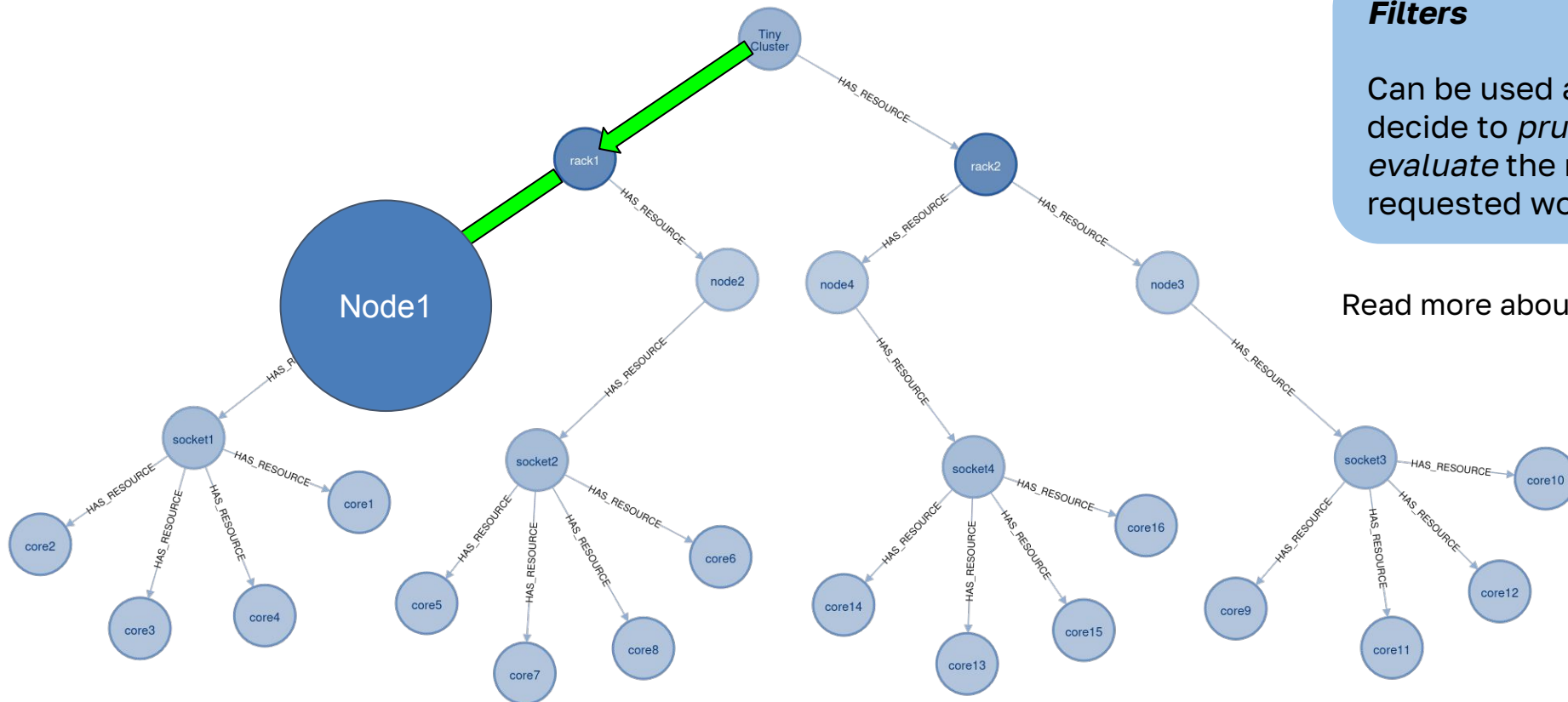
3. The jobspec is compared to the system graph

Traversal means exploring the graph for matching resource sets



3. The jobspec is compared to the system graph

Filters at the node level can quickly assess total counts



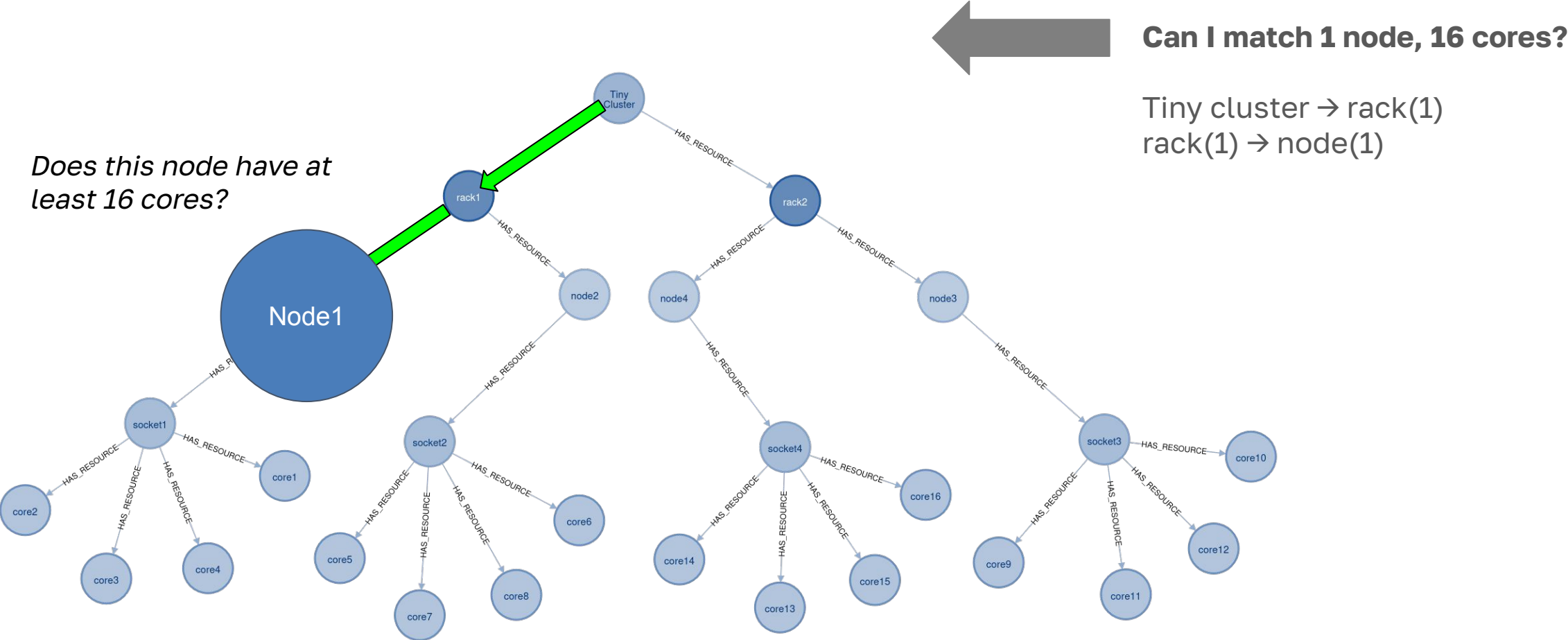
Filters

Can be used at each vertex (any type) to decide to *prune* it or not (and keep looking) or *evaluate* the resources it provides for the requested work (giving it a score).

Read more about the flux scheduler [here](#).

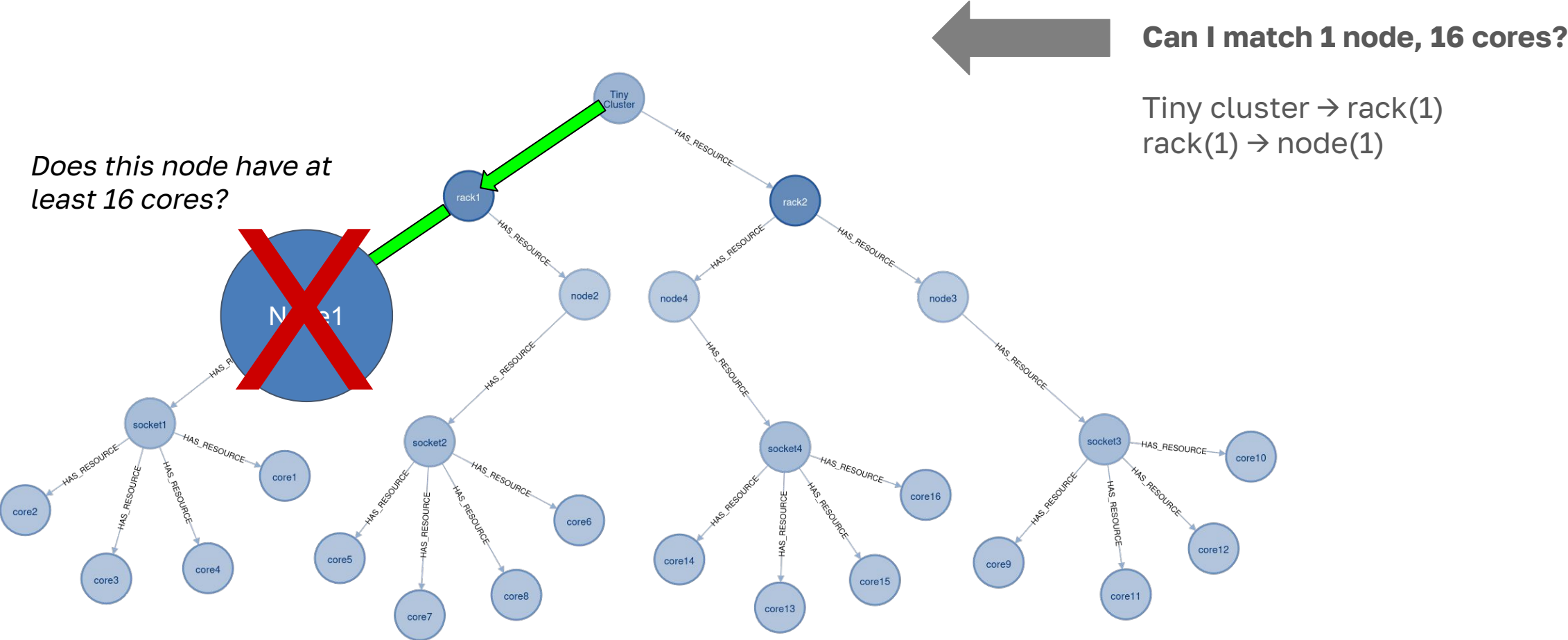
3. The jobspec is compared to the system graph

Filters at the node level can quickly assess total counts



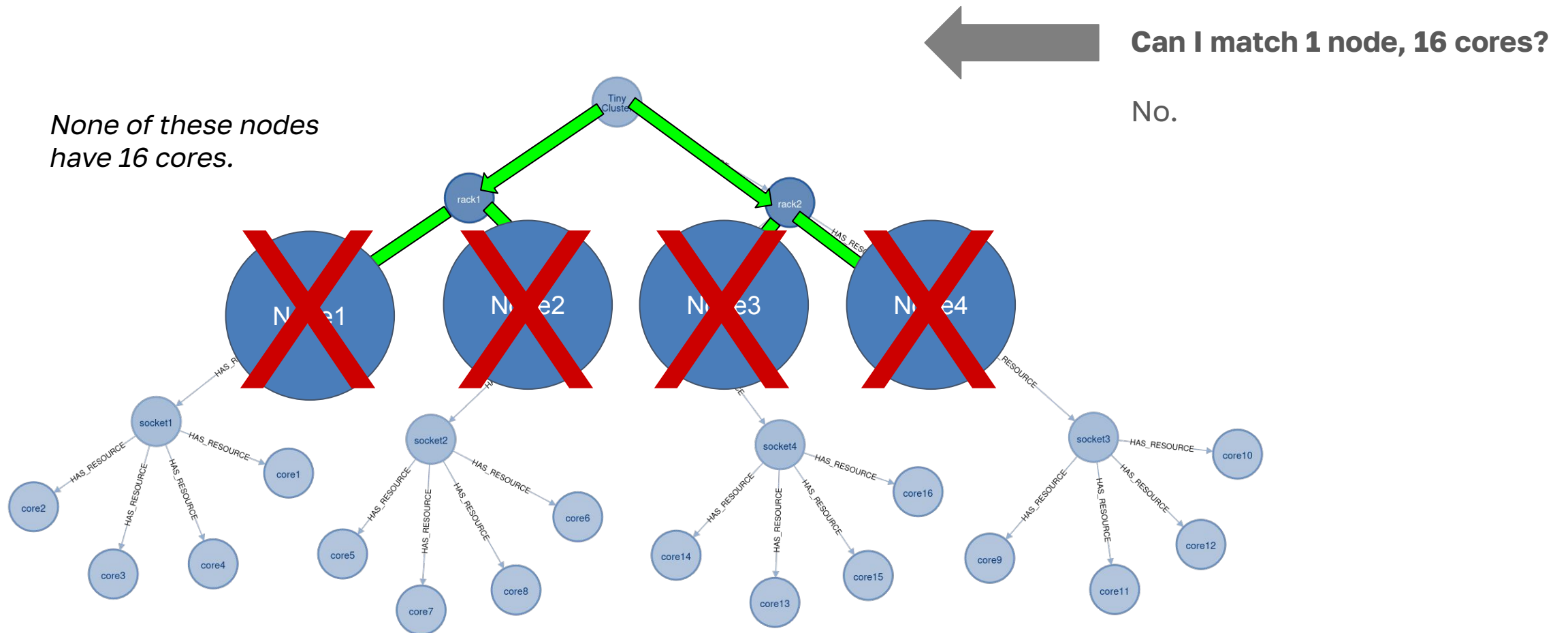
3. The jobspec is compared to the system graph

Filters at the node level can quickly assess total counts



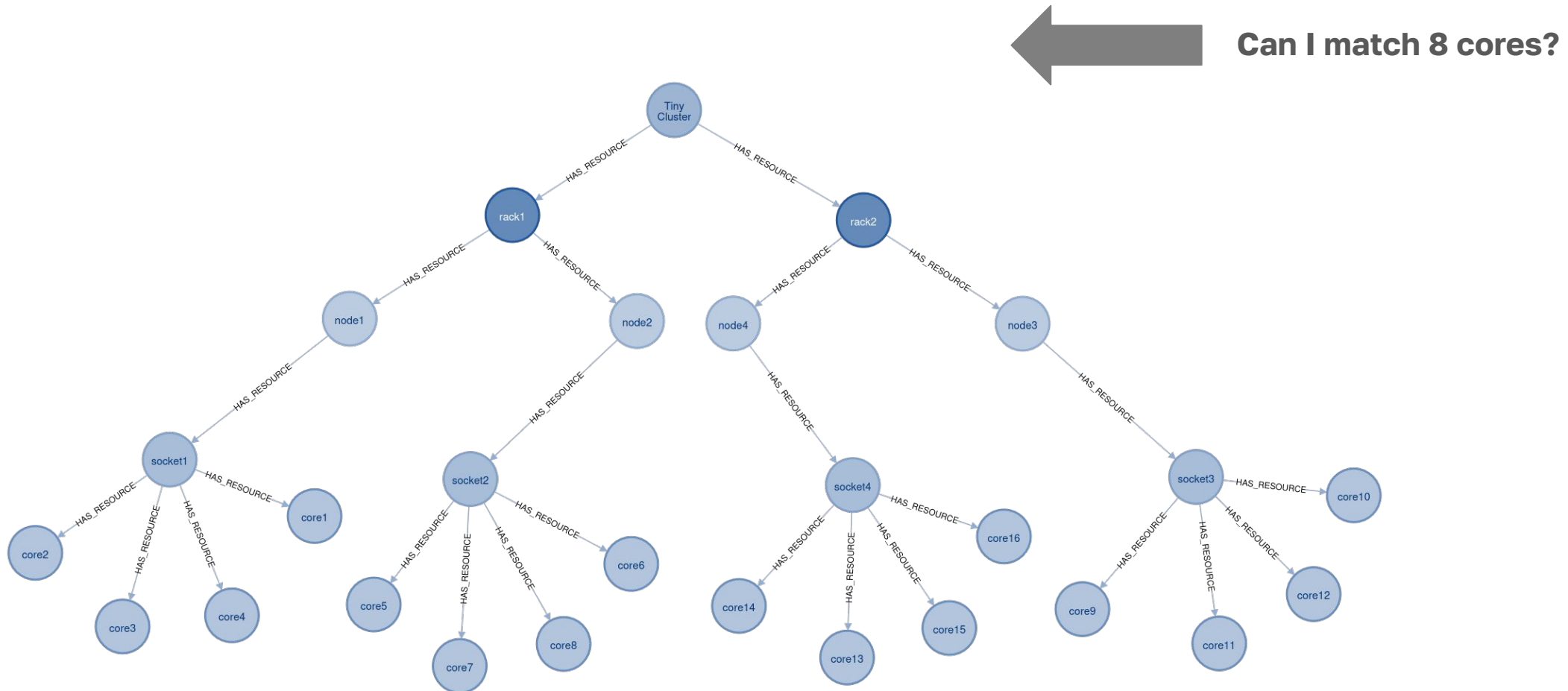
3. The jobspec is compared to the system graph

No matches in the entire graph “forever” means unsatisfiable.



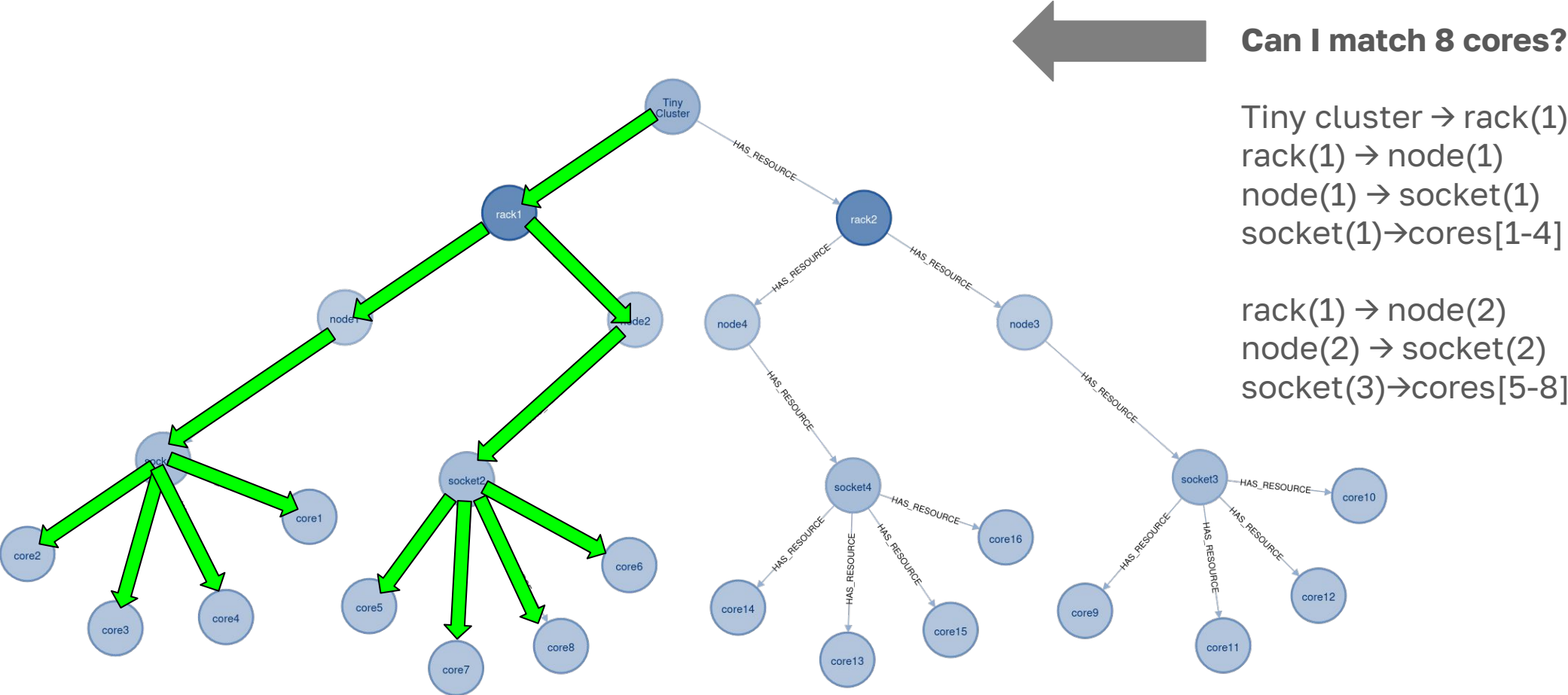
3. The jobspec is compared to the system graph

What if we request cores not limited to number of nodes?



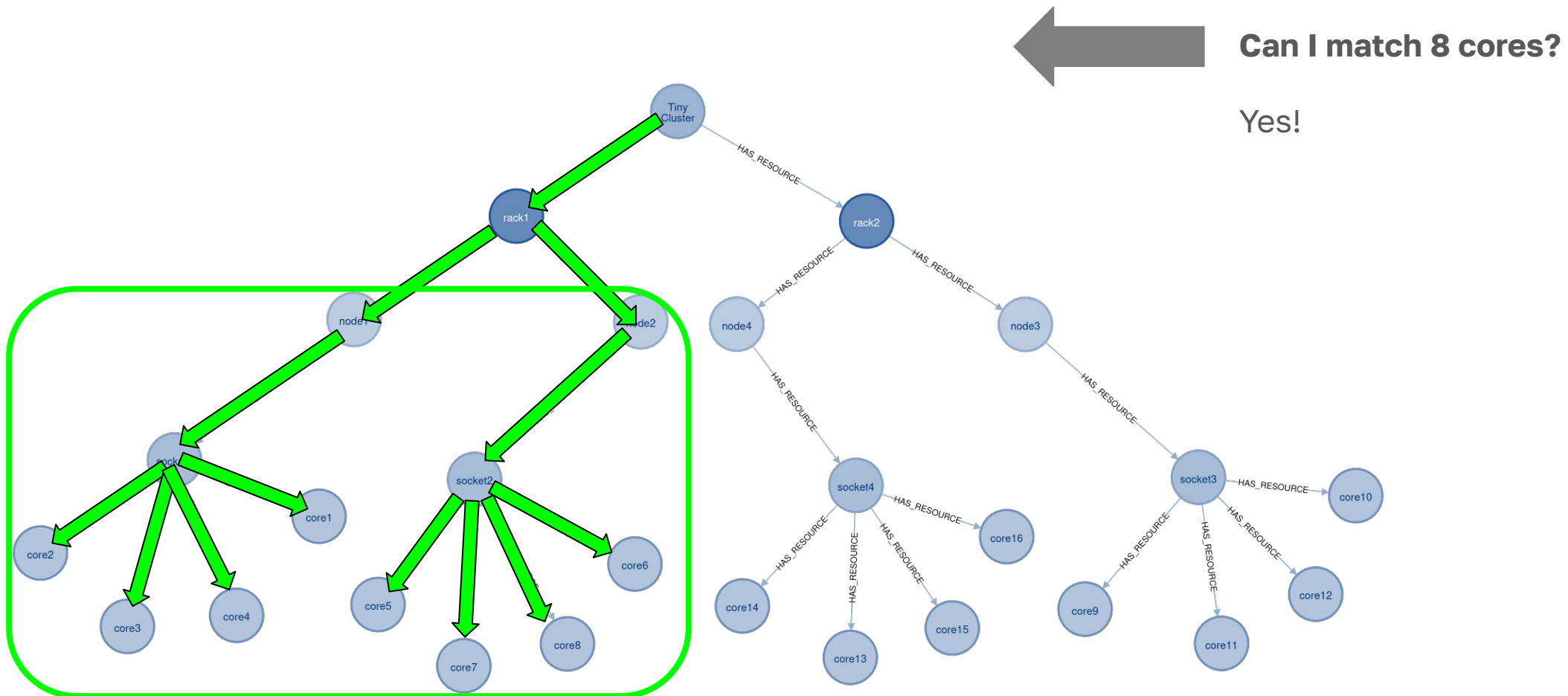
3. The jobspec is compared to the system graph

When we pass filters, we explore sockets and cores



3. The jobspec is compared to the system graph

When we pass filters, we explore sockets and cores

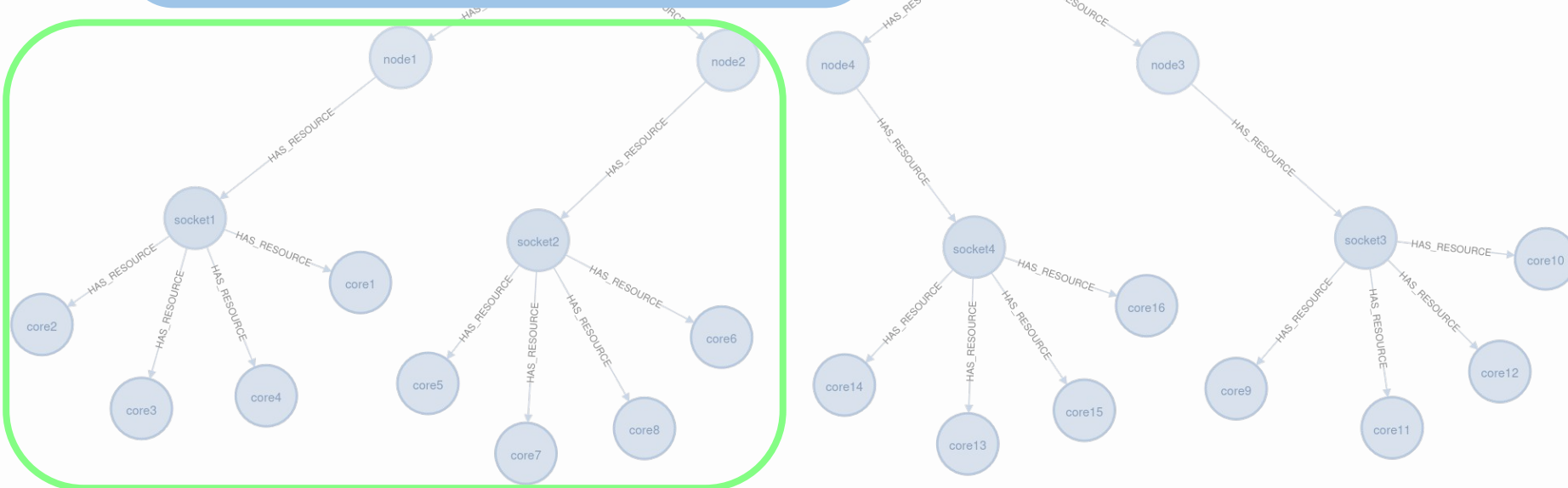


3. The jobspec is compared to the system graph

A match means we have resources for the job

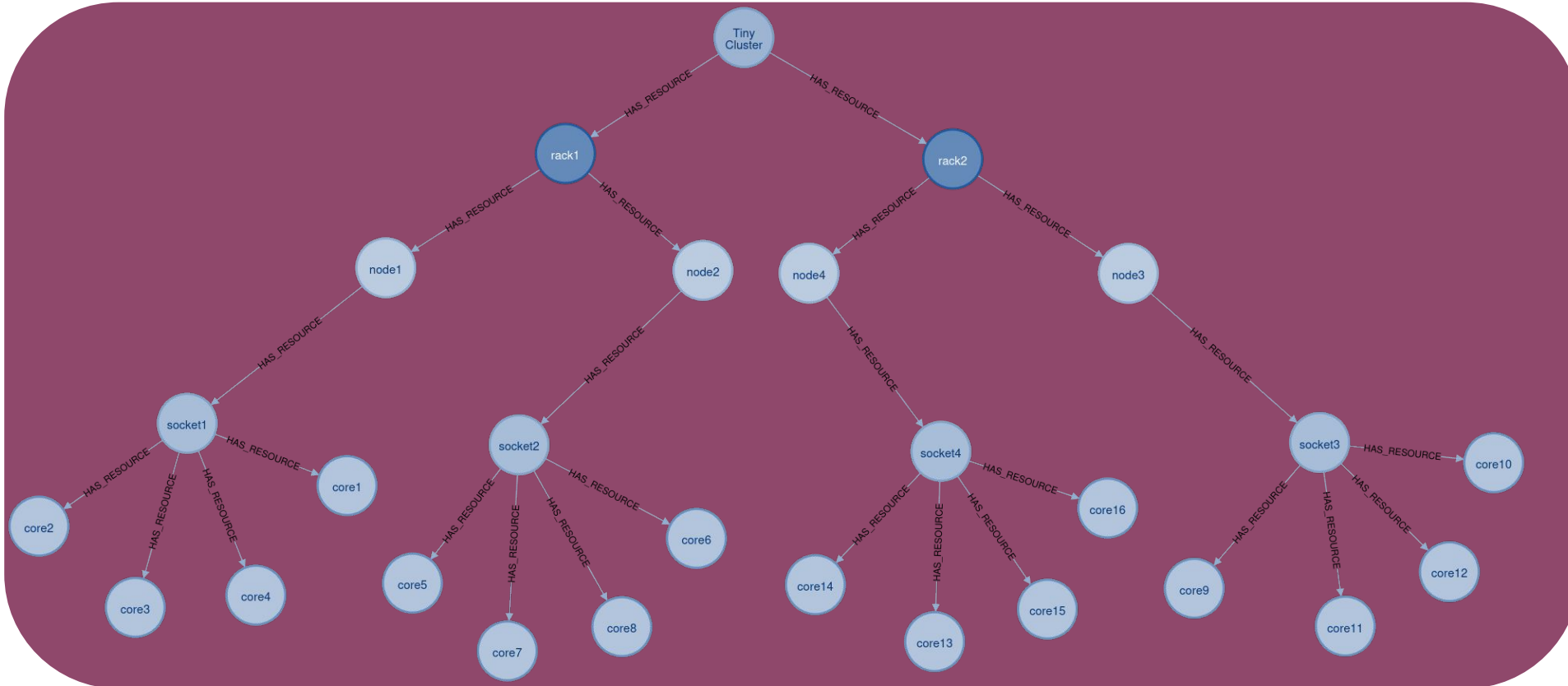
Brokers

Run on each node and work together to constitute a Flux instance. Resources can be subdivided within the instance.



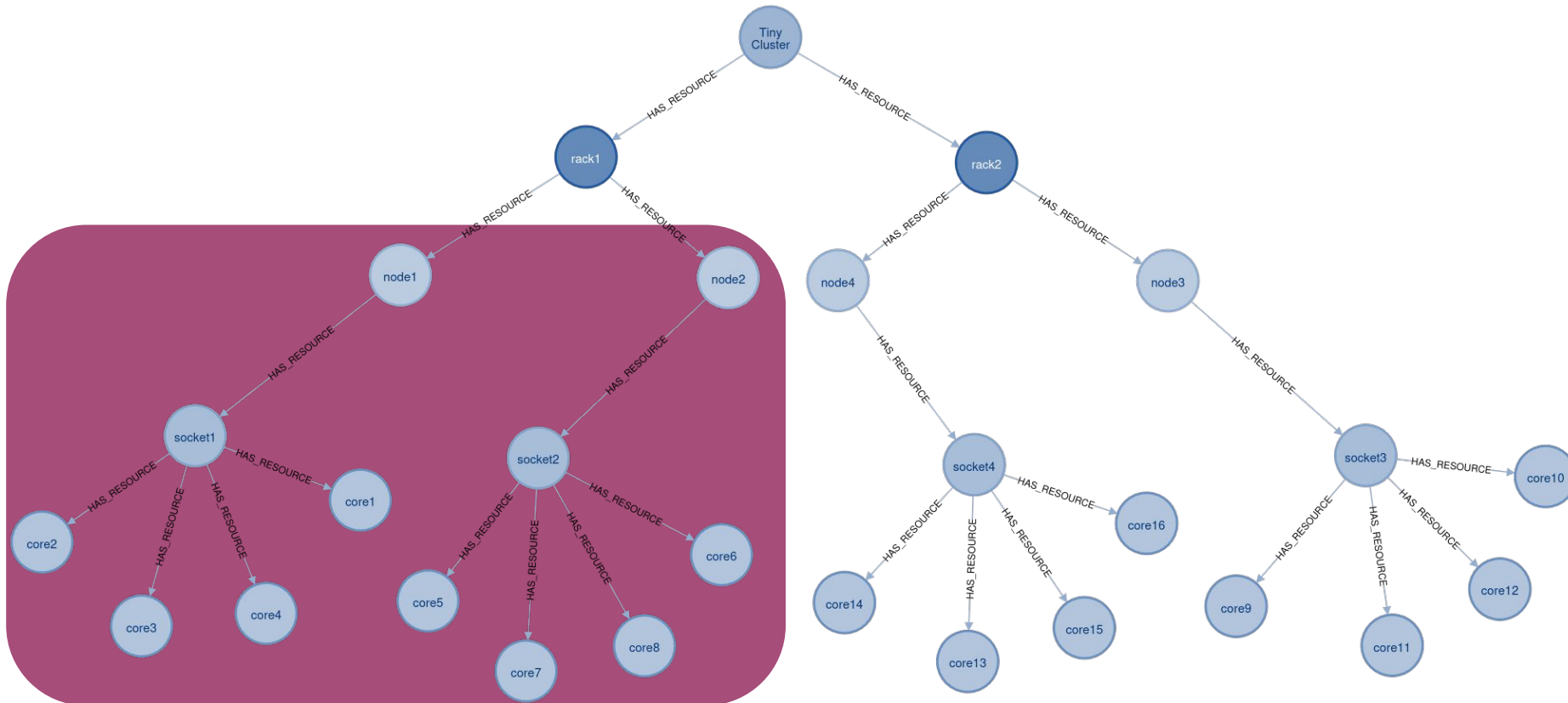
4. An instance is created for your job

This is the system instance



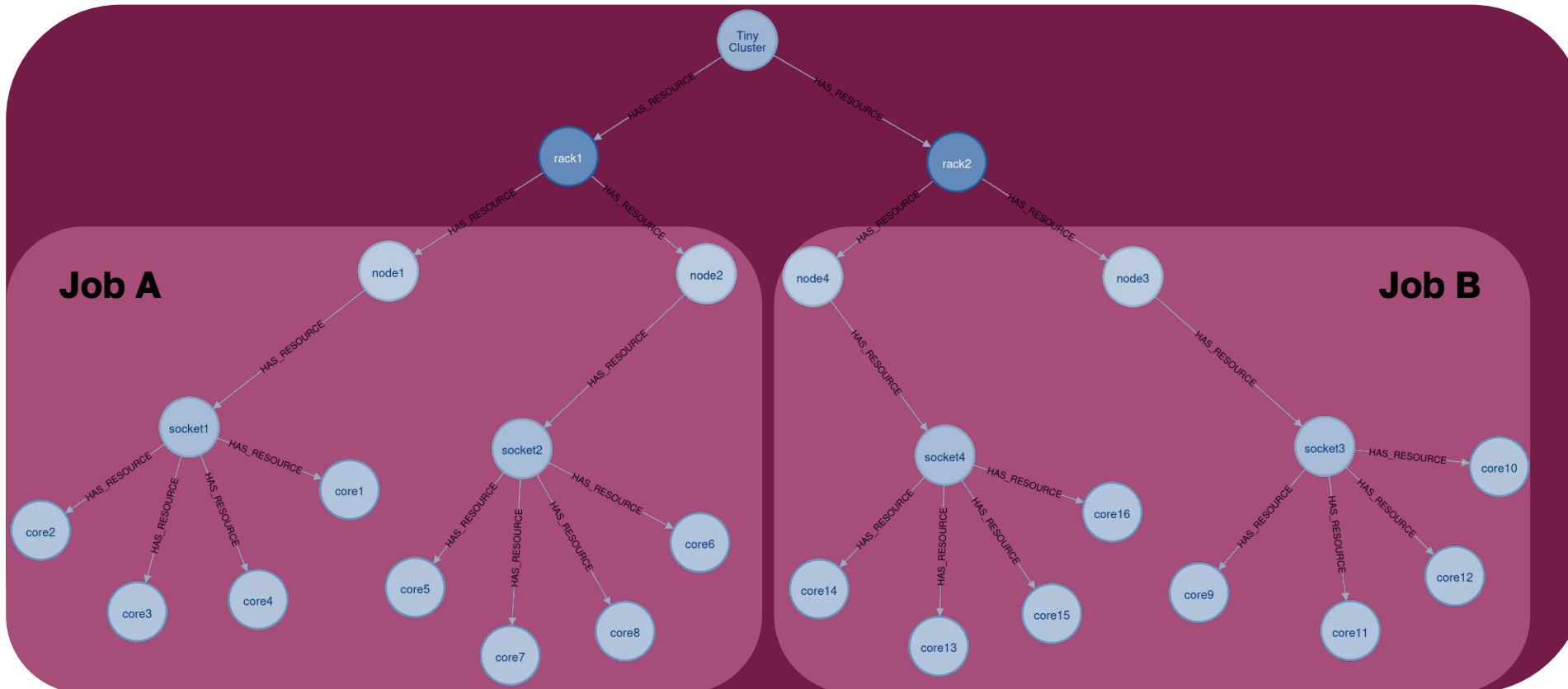
4. An instance is created for your job

This is the job-level instance (sub or child instance)



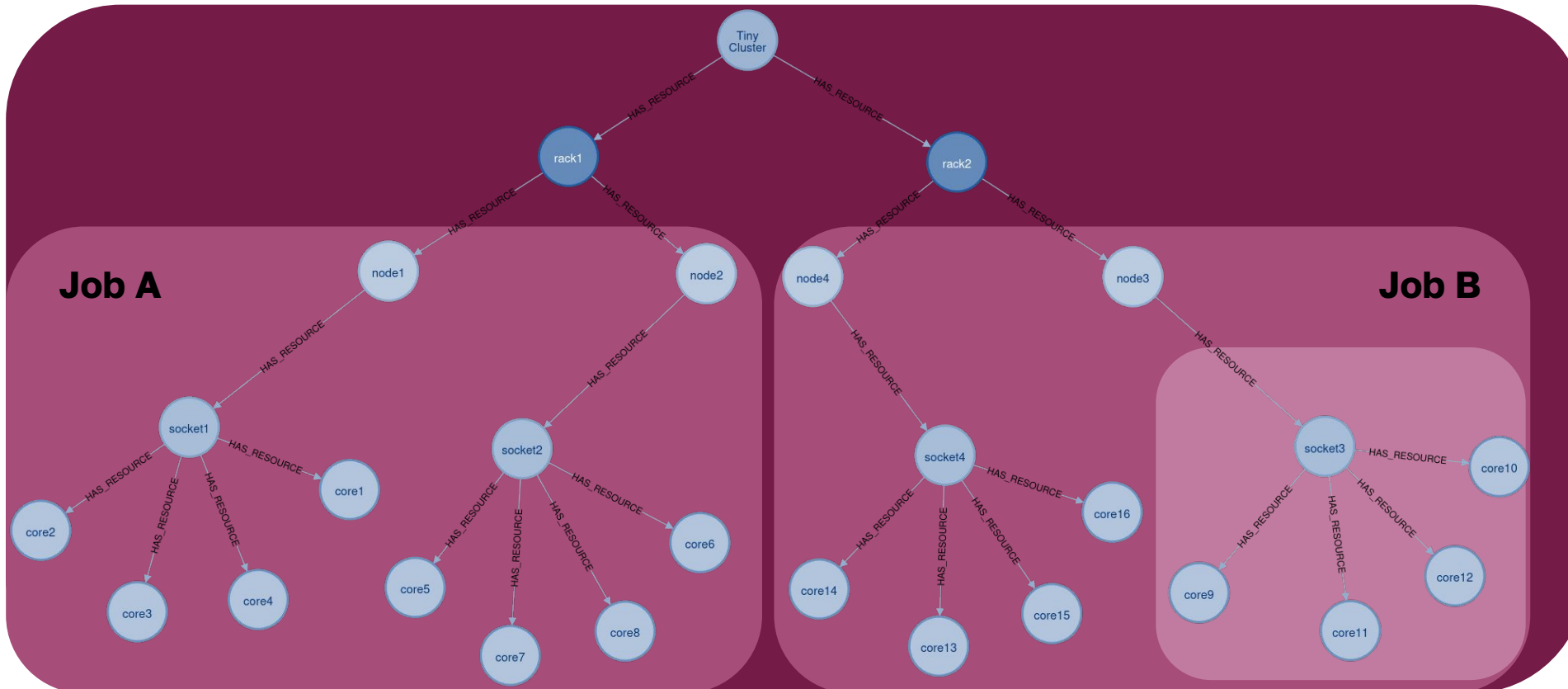
4. An instance is created for your job

All jobs on a Flux cluster are child instances of the system instance



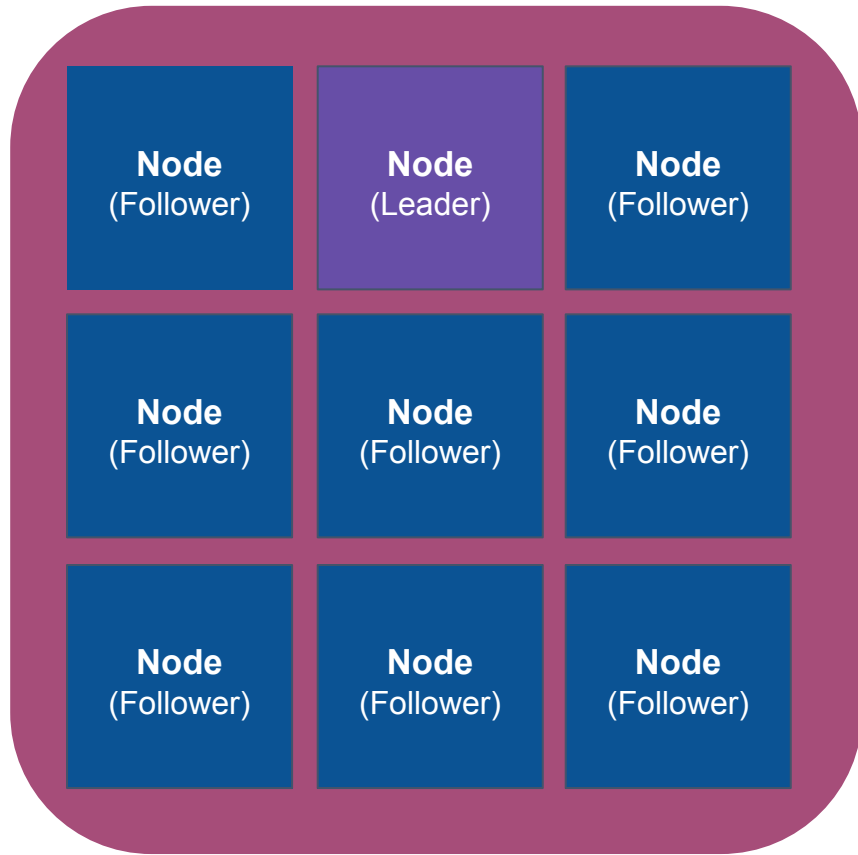
4. An instance is created for your job

You can create sub-instances *within* jobs



4. An instance is created for your job

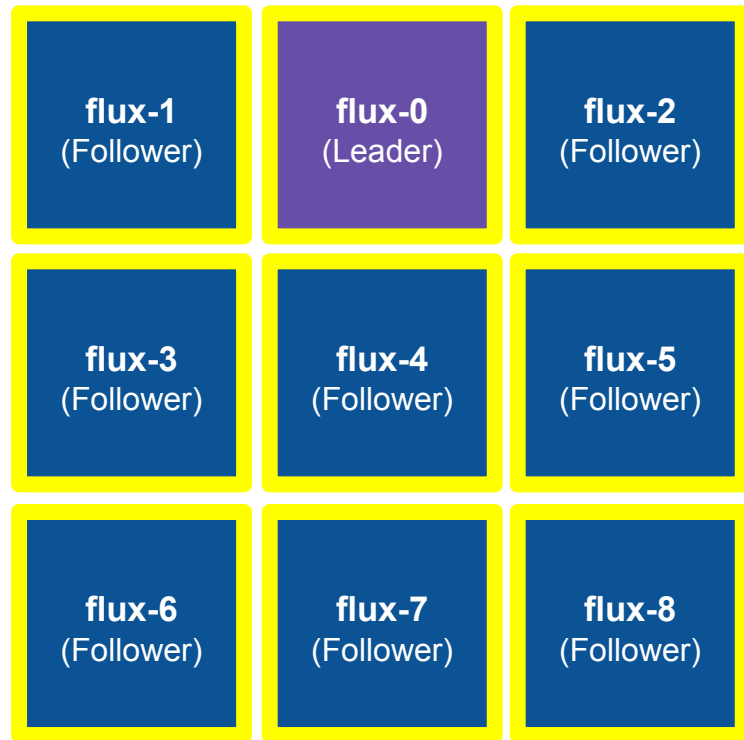
Let's visualize an instance with 9 nodes. There is 1 leader, 8 followers



```
flux batch -N 9 ./batch-job.sh
```

4. An instance is created for your job

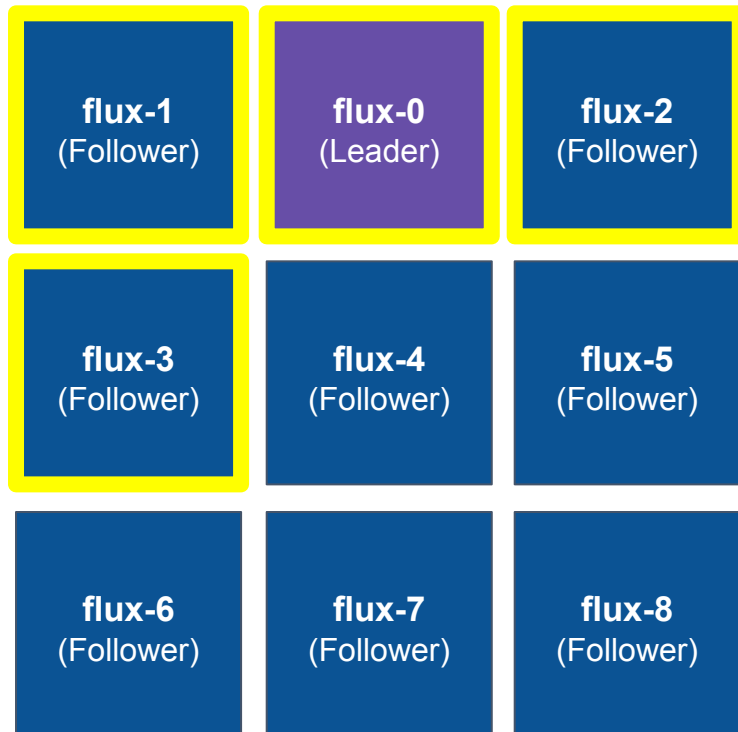
Let's visualize an instance with 9 nodes. There is 1 leader, 8 followers



"I have an allocation with flux[0-8]"

4. An instance is created for your job

Let's visualize an instance with 9 nodes. There is 1 leader, 8 followers



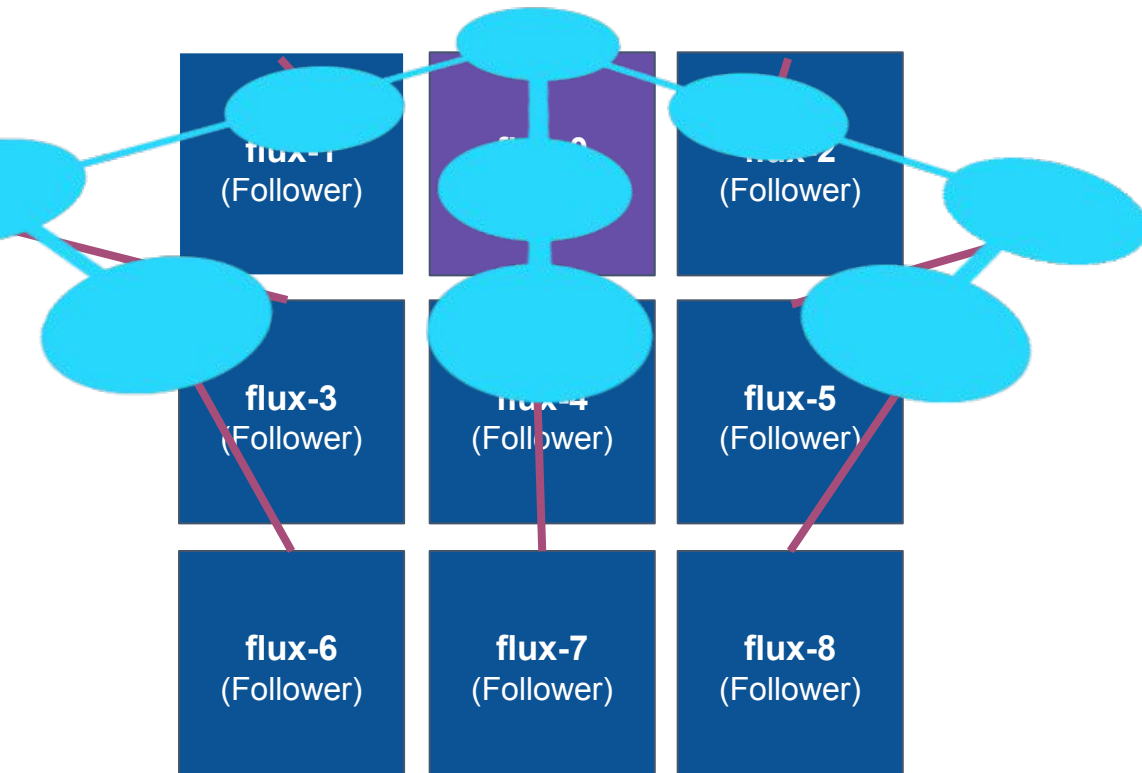
"I have an allocation with flux[0-8]"
"flux[0-3] have 8x GPUs each"

What about the network?



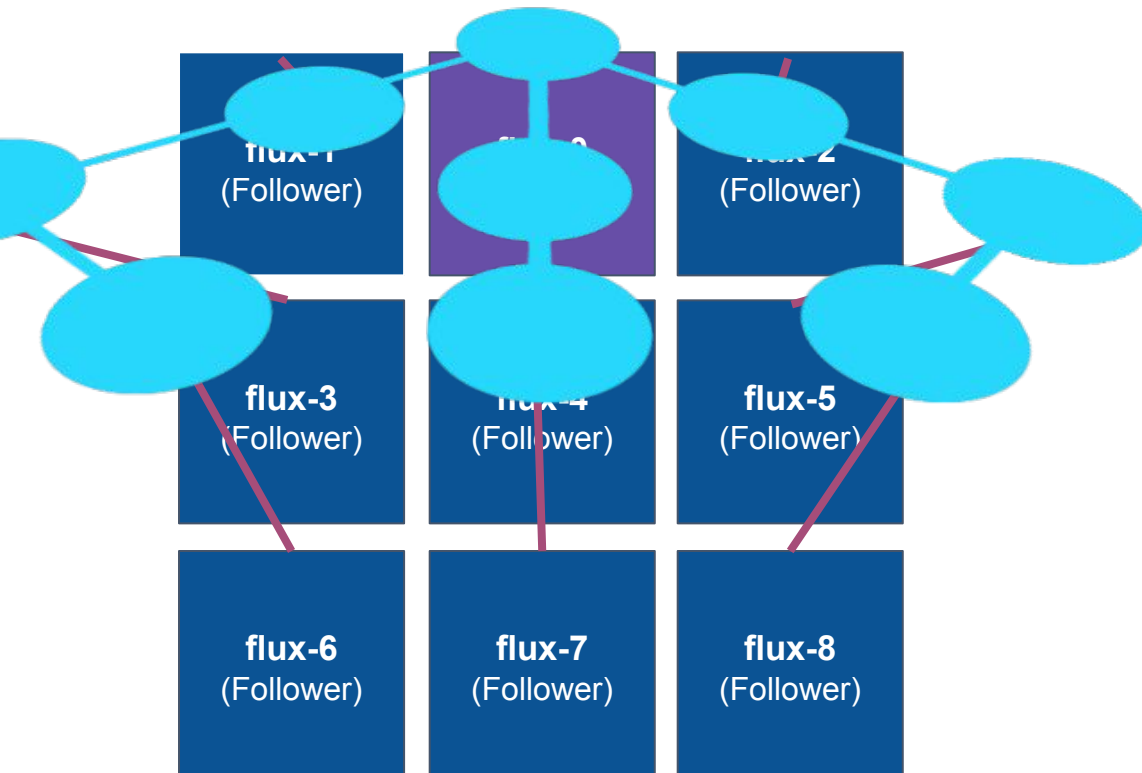
4. An instance is created for your job

Nodes communicate with a tree based overlay network "TBON"



4. An instance is created for your job

Nodes communicate with a tree based overlay network "TBON"

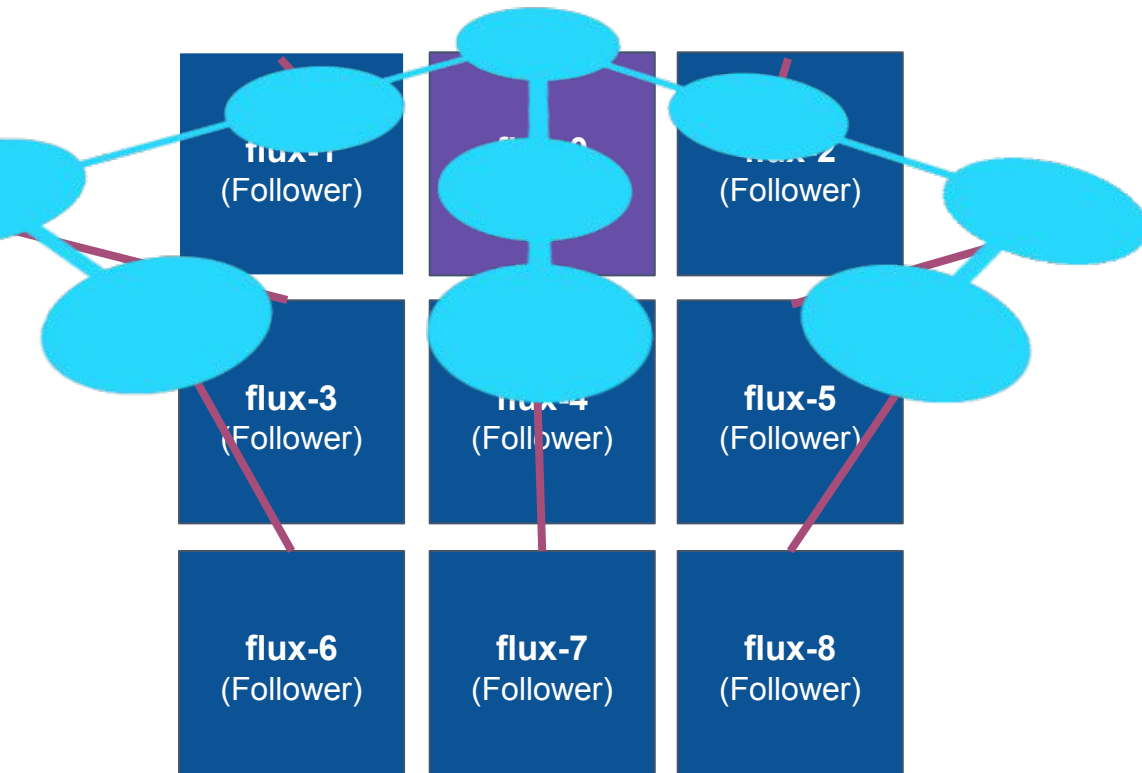


Would you like some T-BON?



4. An instance is created for your job

Nodes communicate with a tree based overlay network "TBON"

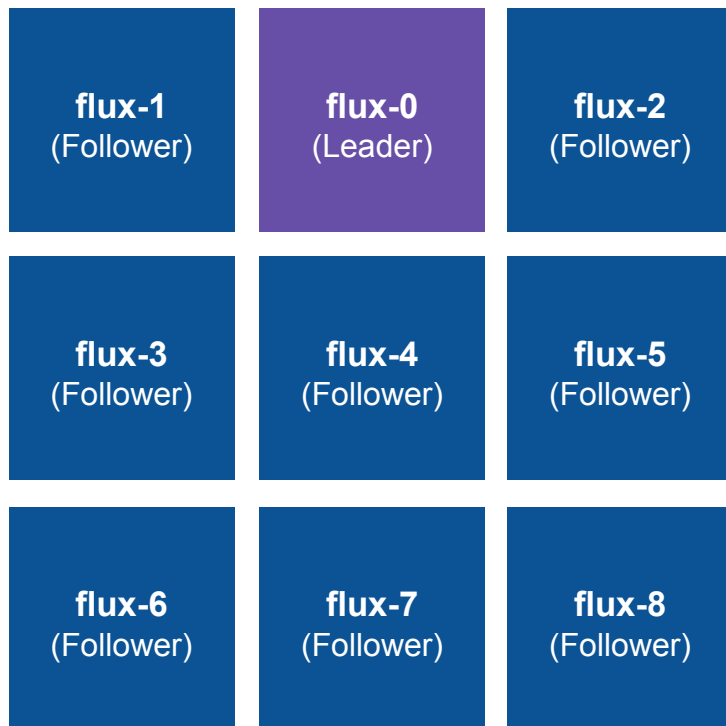


Did you know!

A **topology** of the TBON, or how the nodes are connected (binomial vs. k-ary tree, etc.) can be customized and influence how messages are exchanged.

4. An instance is created for your job

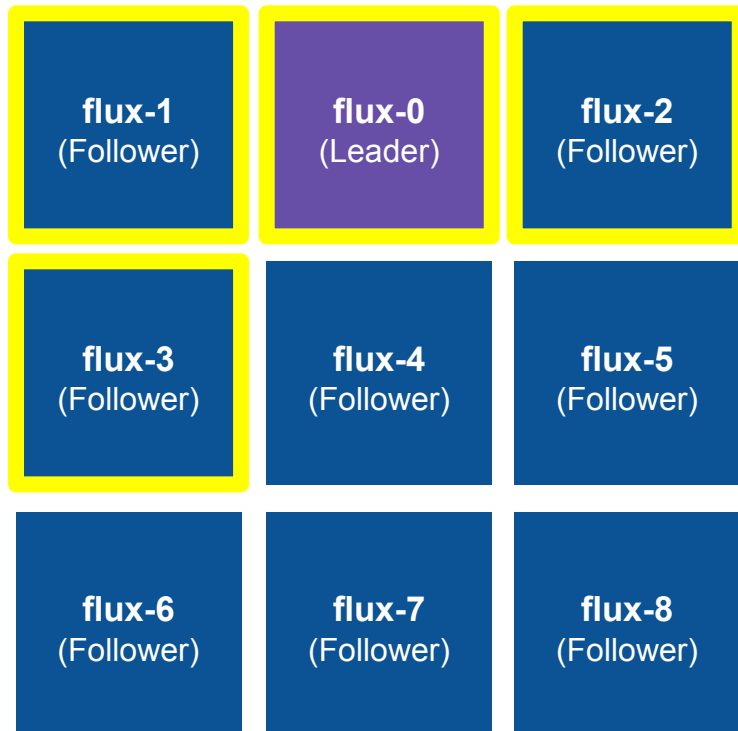
We can sub-allocate resources within our top level instance



```
flux batch -N 9 ./batch-job.sh
```


4. An instance is created for your job

We can sub-allocate resources within our top level instance



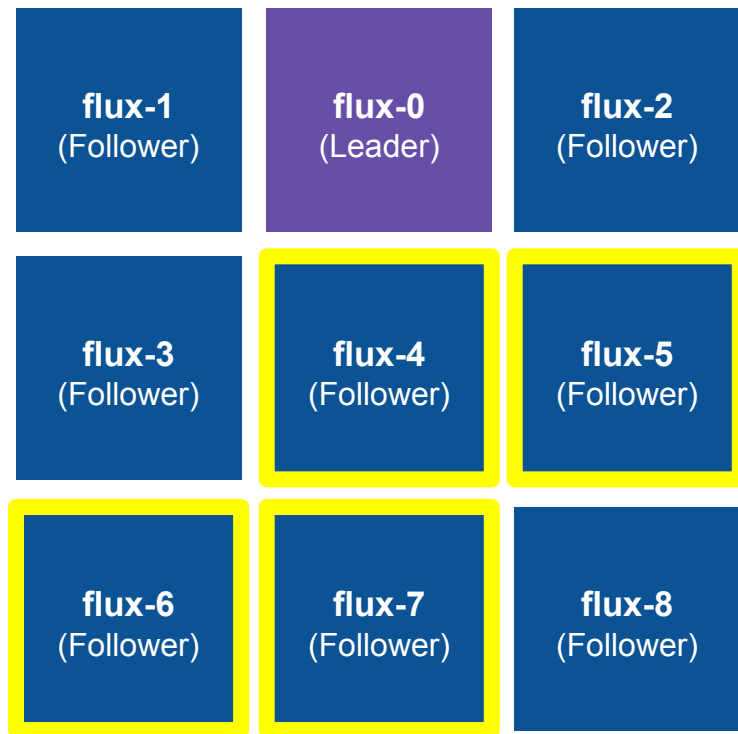
```
flux batch -N 9 ./batch-job.sh
```

I want:

- *flux-[0-3] (with GPUs) to do training*

4. An instance is created for your job

We can sub-allocate resources within our top level instance



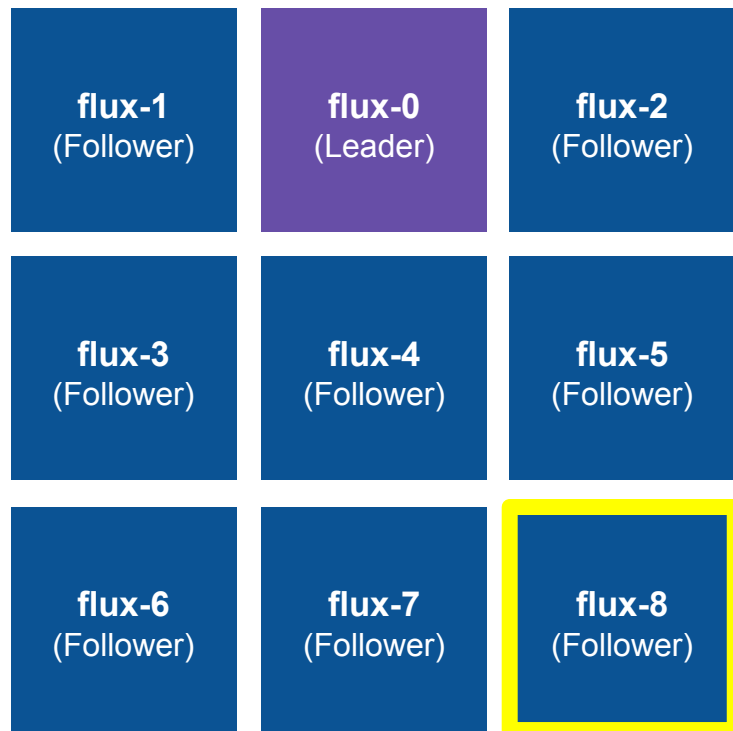
```
flux batch -N 9 ./batch-job.sh
```

I want:

- *flux-[0-3] (with GPUs) to do training*
- *flux-[4-7] to do ensemble simulation*

4. An instance is created for your job

We can sub-allocate resources within our top level instance



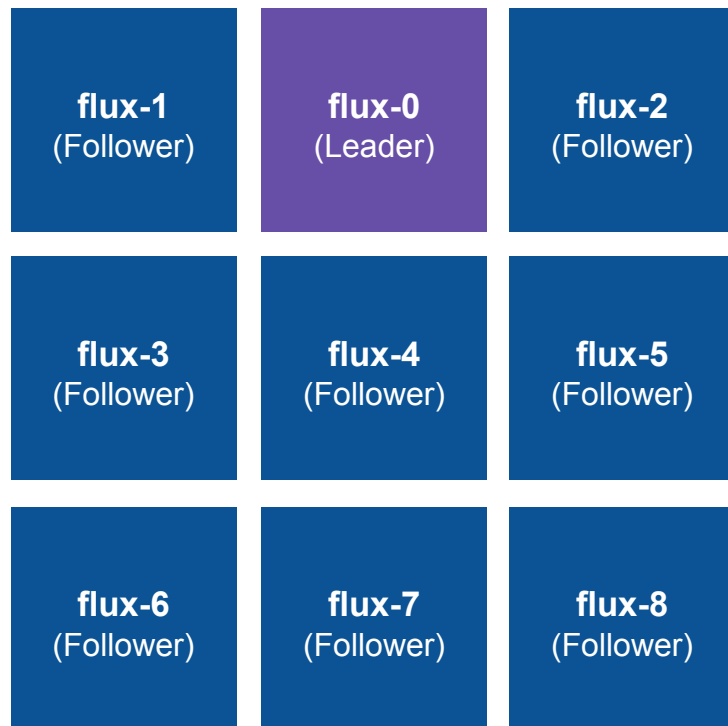
```
flux batch -N 9 ./batch-job.sh
```

I want:

- *flux-[0-3] (with GPUs) to do training*
- *flux-[4-7] to do ensemble simulation*
- *flux-8 to run services*

4. An instance is created for your job

We can sub-allocate resources within our top level instance



./batch-job.sh

```
#!/bin/sh
```

```
#FLUX -N 9
```

```
#FLUX -t 1200m
```

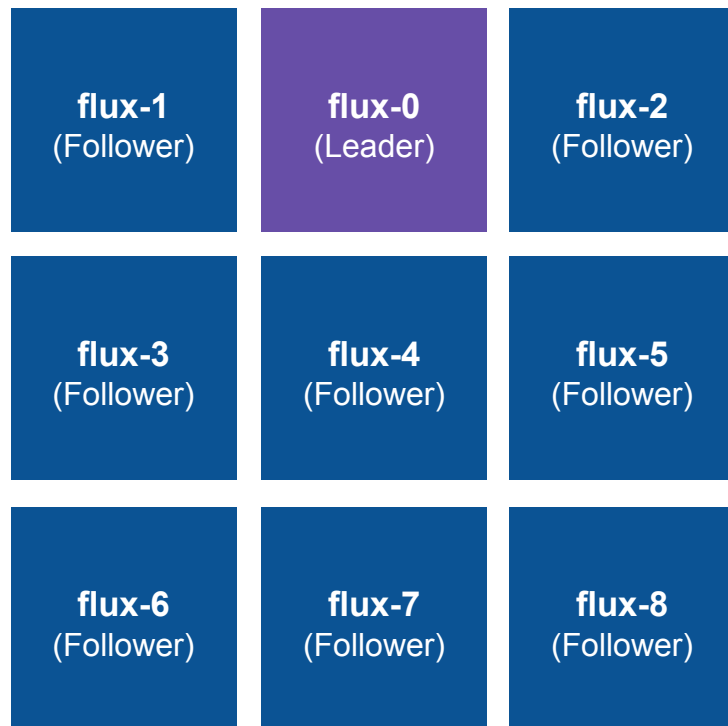
```
flux submit -setattr=usernetes=true sh service.sh
```

```
flux submit -N4 -g1 python train.py
```

```
flux run -N4 python simulation.py
```

4. An instance is created for your job

We can sub-allocate resources within our top level instance



./batch-job.sh

```
#!/bin/sh
```

```
#FLUX -N 9
```

```
#FLUX -t 1200m
```

```
flux submit -setattr=usernetes=true sh service.sh
```

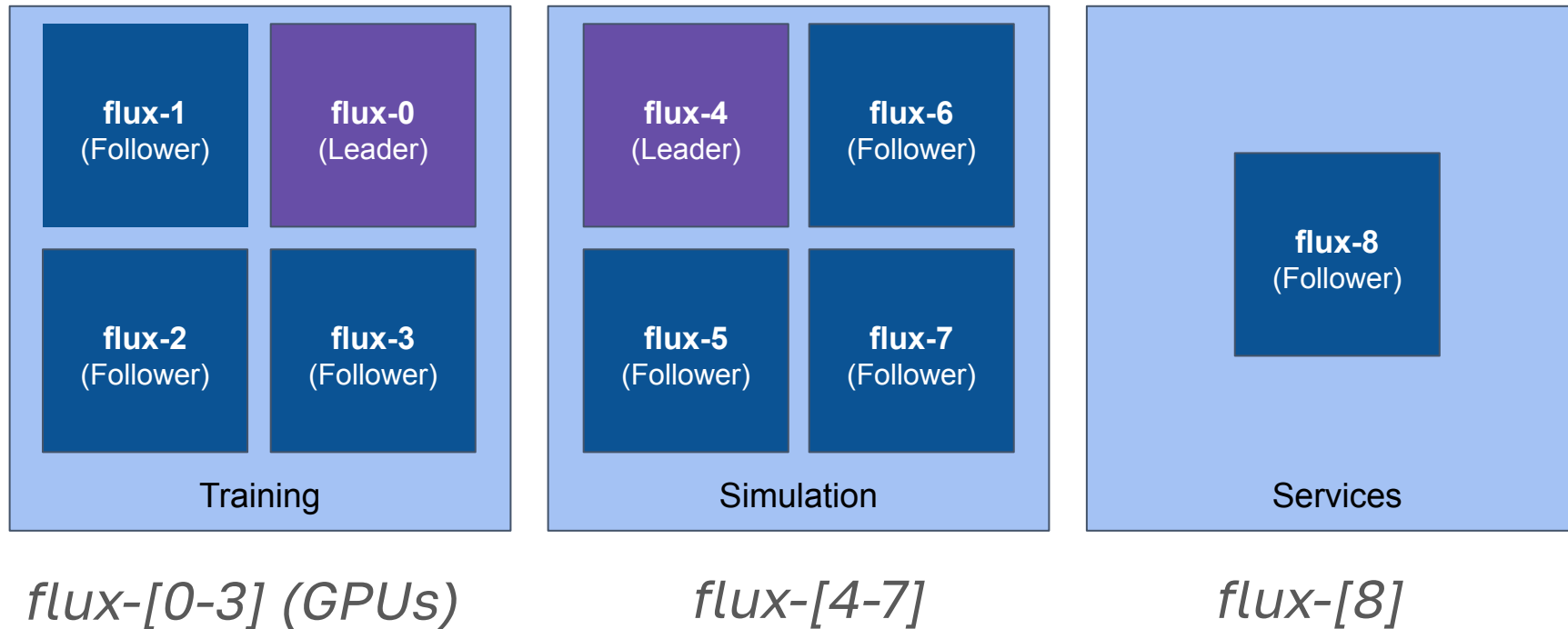
```
flux submit -N4 -g1 python train.py
```

```
flux run -N4 python simulation.py
```

We are prototyping user-space Kubernetes – **Usernetes** to allow for deployment and teardown of a cluster with a job or batch job.

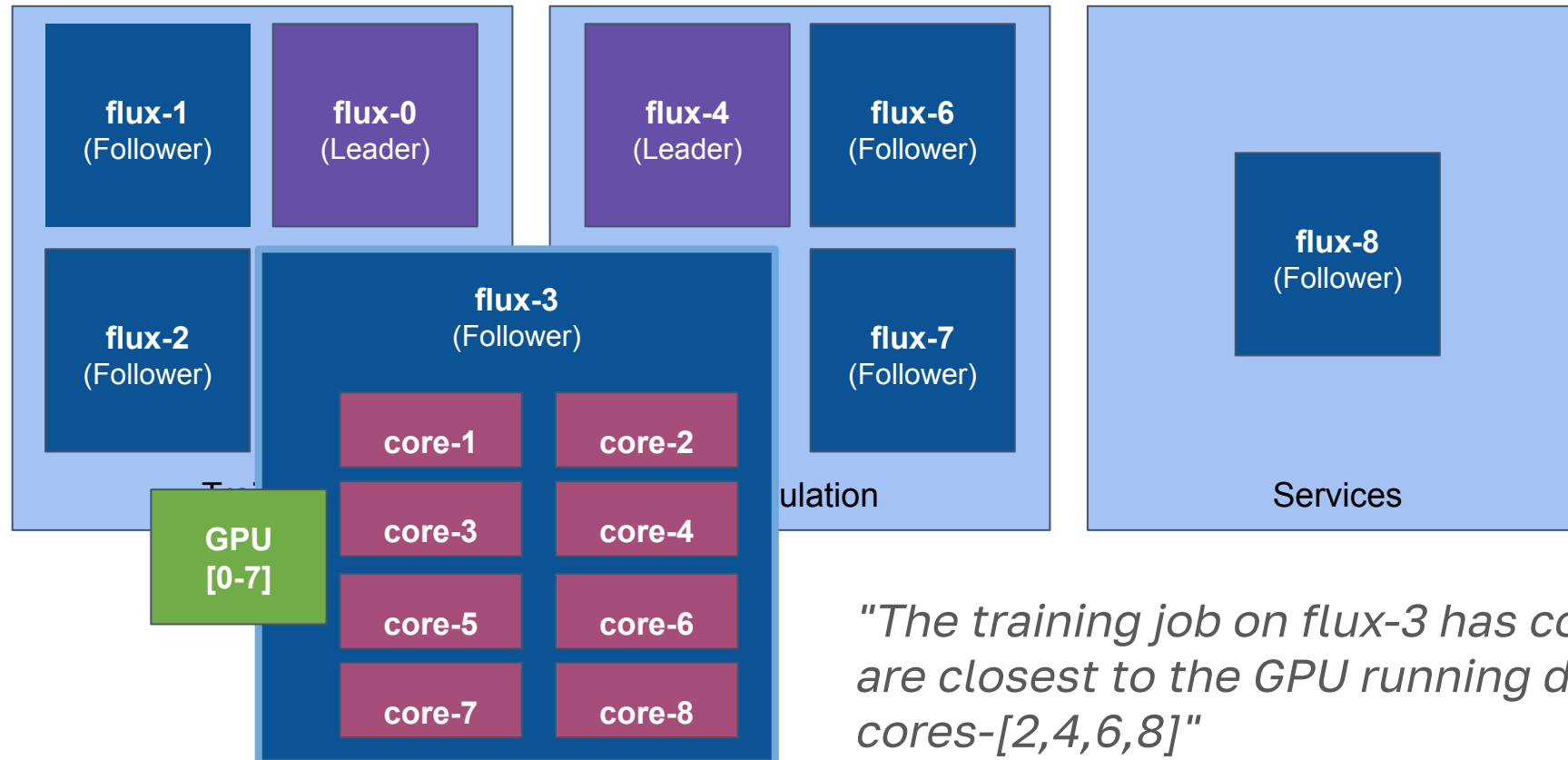
5. Instances can handle nuanced job needs

We can create sub (child) instances *within* our parent instance.



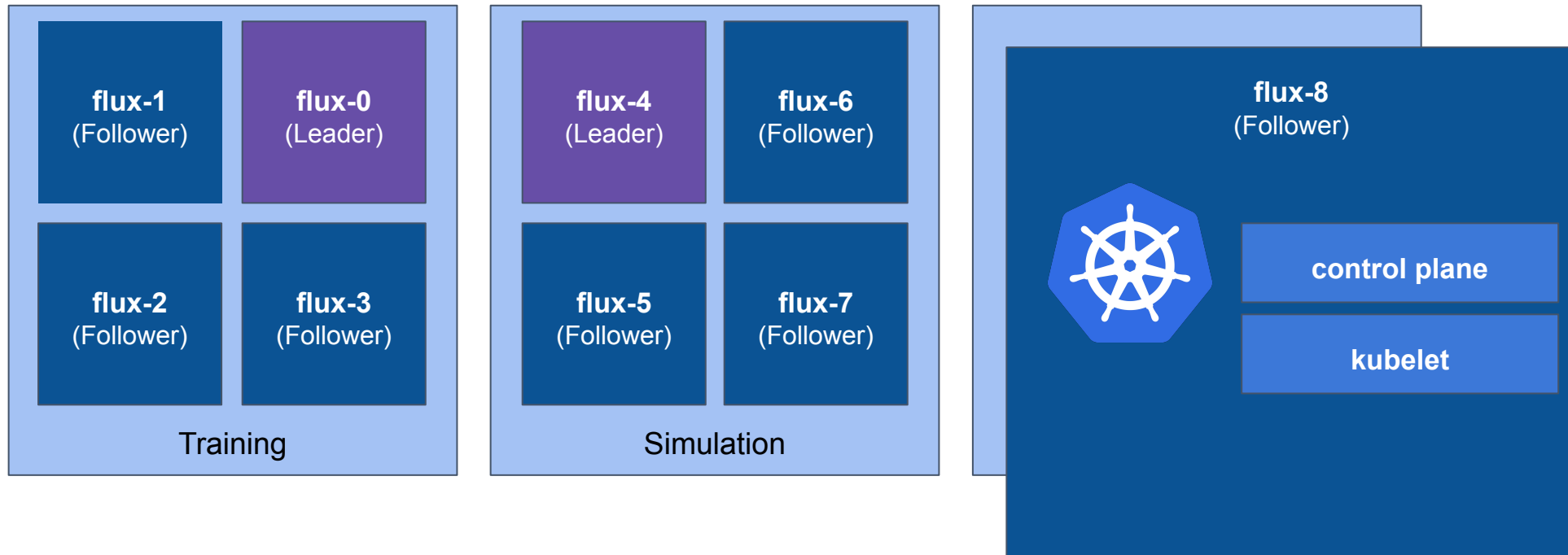
5. Instances can handle nuanced job needs

Brokers on each node can further manage resources



5. Instances can handle nuanced job needs

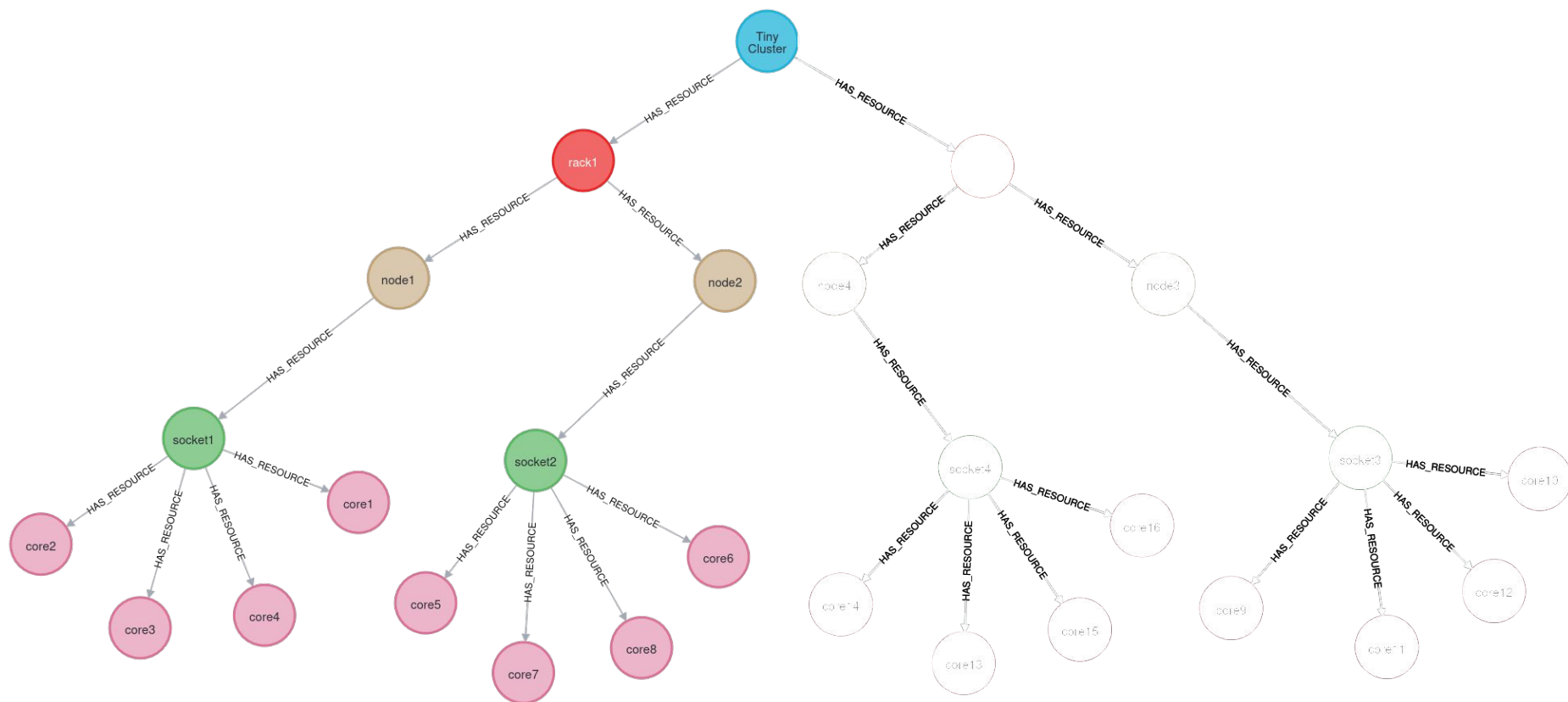
Brokers on each node can further manage resources



```
kubectl apply -f database.yaml  
kubectl apply -f service.yaml
```

6. Graph elasticity means adding or removing subgraphs

"Shrink the system level instance to just one rack."



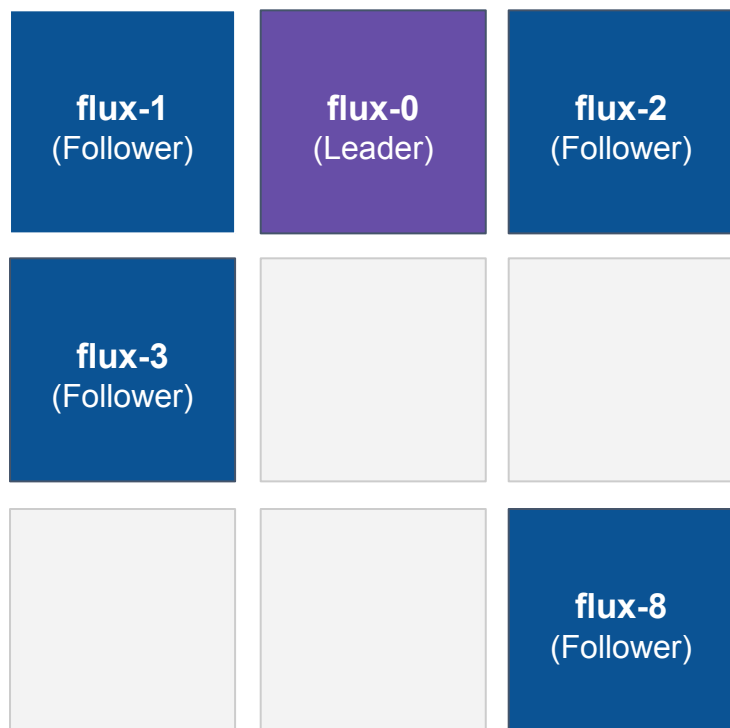
6. Job elasticity means changing an allocation's resources.

"We are done with simulation, deallocate flux[4-7]"



6. Job elasticity means changing an allocation's resources.

"We are done with simulation, deallocate flux[4-7]"



Did you know?

Job elasticity can improve utilization and allow resources to respond to workflow needs.

6. Job elasticity means changing an allocation's resources.

"We are done with simulation, deallocate flux[4-7]"



🚧 Under Development 🚧

flux-sched supports elasticity, integration with flux-core is partially supported (fully supported soon).

What is the lifecycle of a job?

```
flux run -N 1 -n 18 ./app
```



Science!

We need integration with workflow tools!

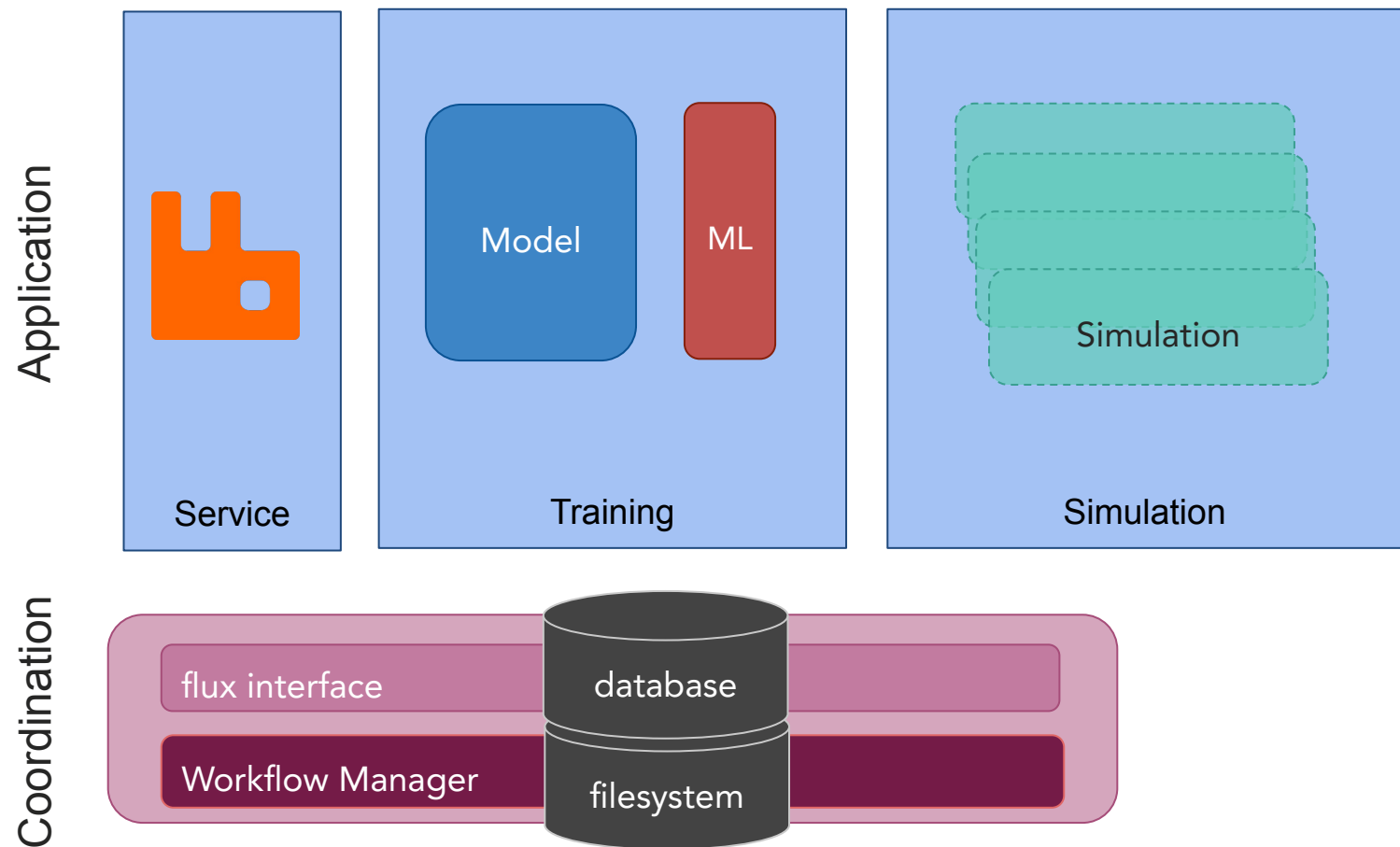
My HPC Job



The Science!

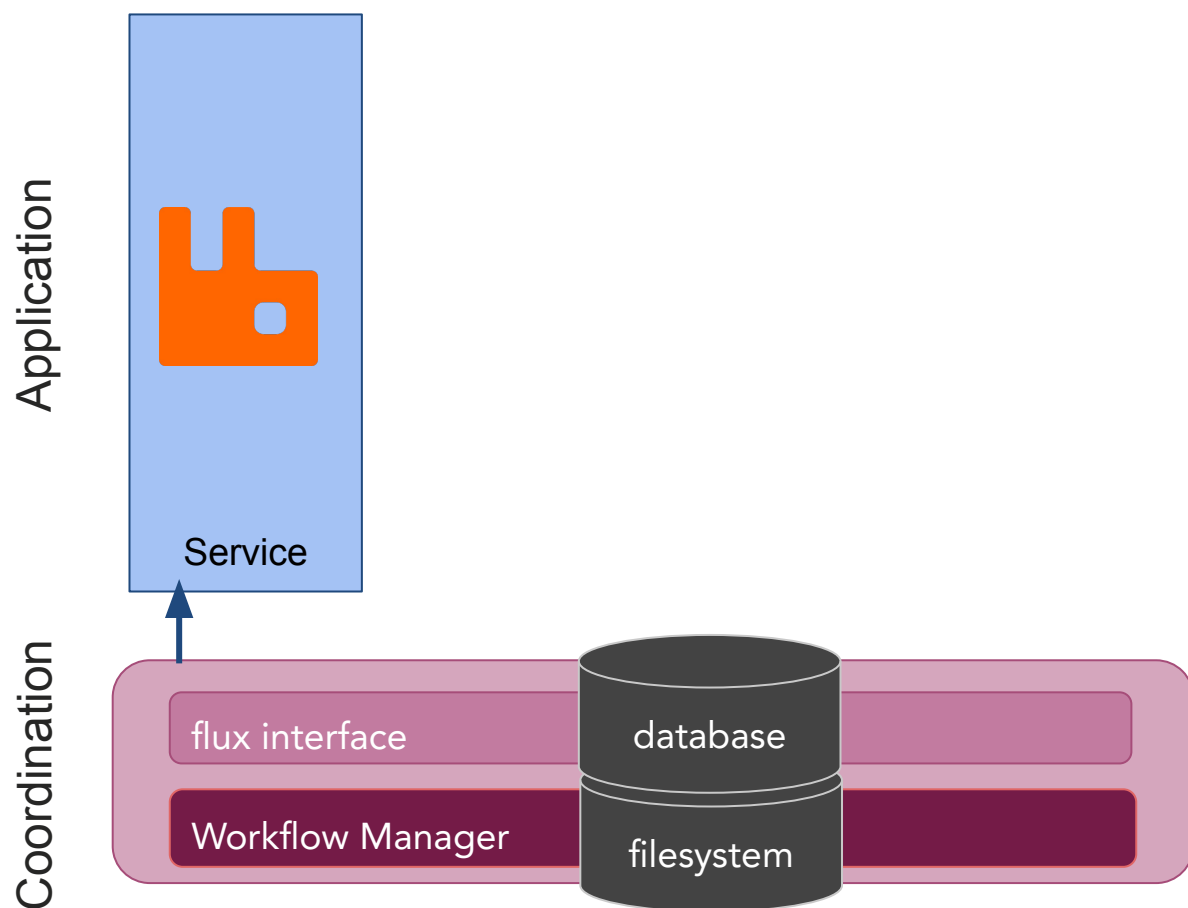
3. Flux APIs integrate with workflow managers

How might a workflow manager run this?



3. Flux APIs integrate with workflow managers

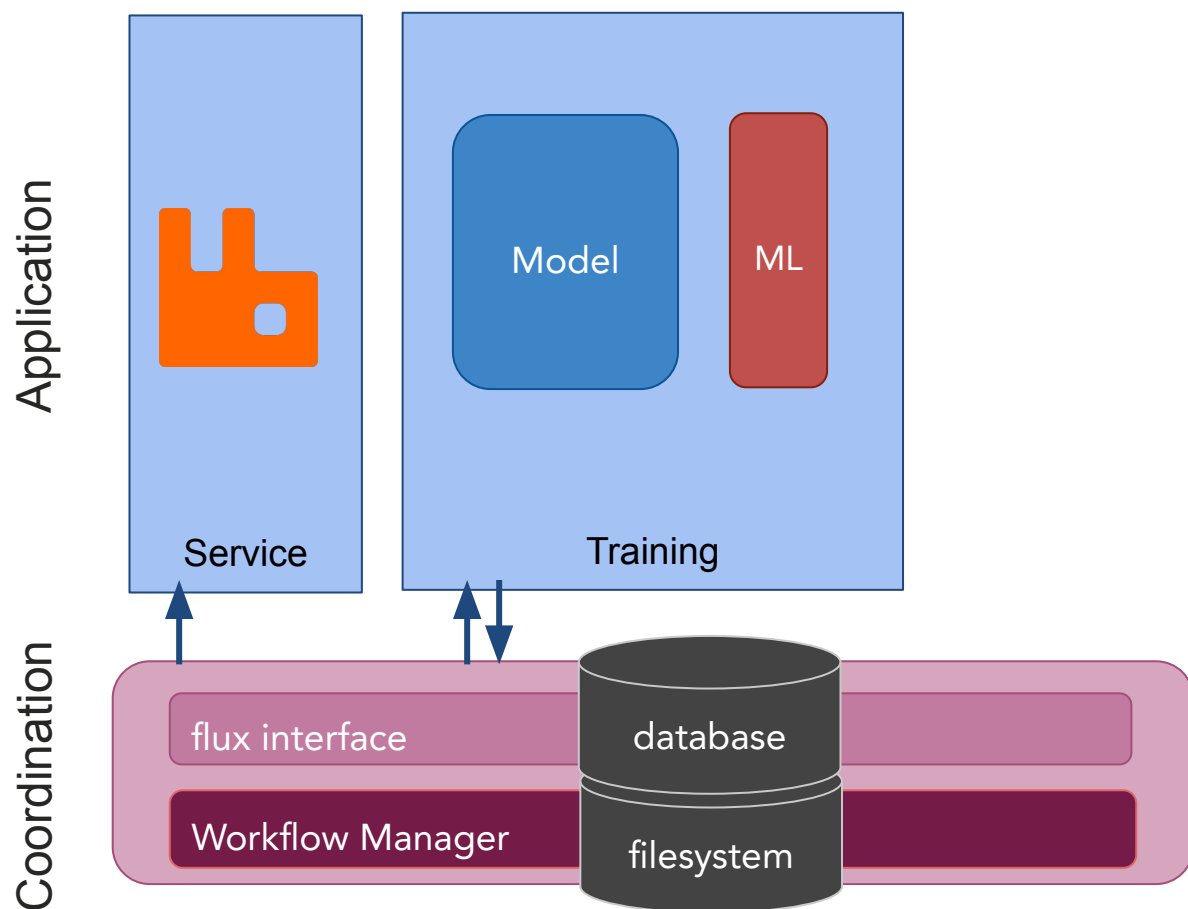
First submit a job for the service (not blocking)



```
flux submit -setattr=usernetes=true sh service.sh  
flux run -N4 -g1 python train.py  
flux submit -wait -N4 python simulation.py
```

3. Flux APIs integrate with workflow managers

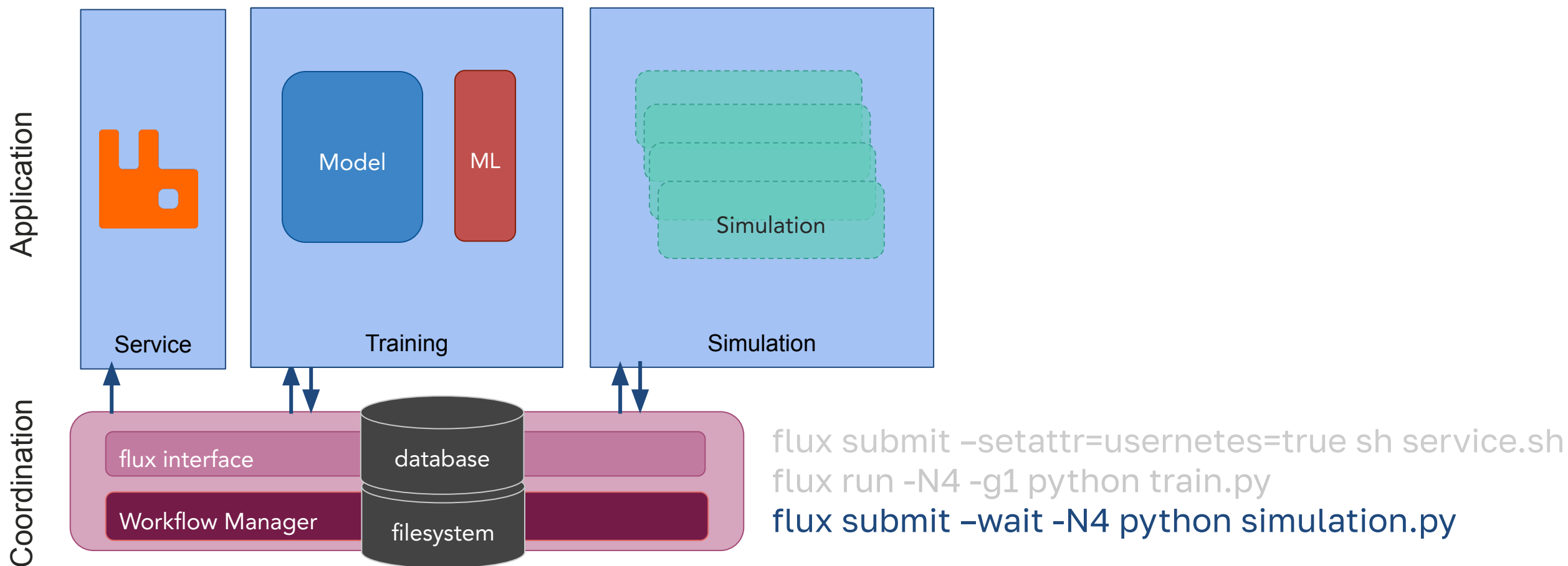
Next run training - we need to train *before* we run simulations, so block



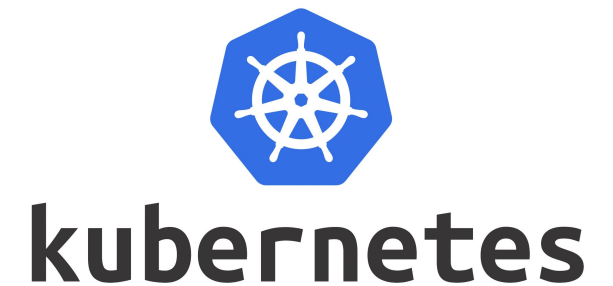
```
flux submit -setattr=usernetes=true sh service.sh  
flux run -N4 -g1 python train.py  
flux submit -wait -N4 python simulation.py
```

3. Flux APIs integrate with workflow managers

Finally, finish with simulations. Add "wait" so we wait until they finish!



Which workflow and orchestration tools support Flux?



Which workflow and orchestration tools support Flux?

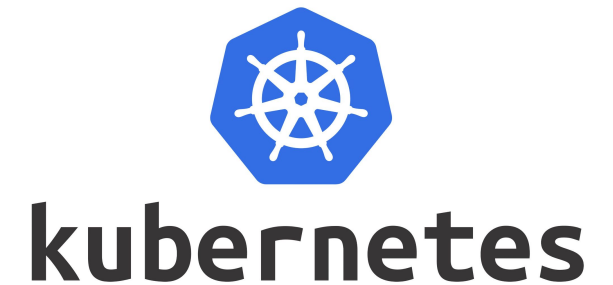
Are there others you'd like to see?



nextflow



m: maestro



Hands-On Demonstration



Log in to our AWS EKS cluster!

1. Go to tutorial.flux-framework.org
2. Create any username you want. The password is **“flux-tutorial-isc2025”**
3. You’re in!

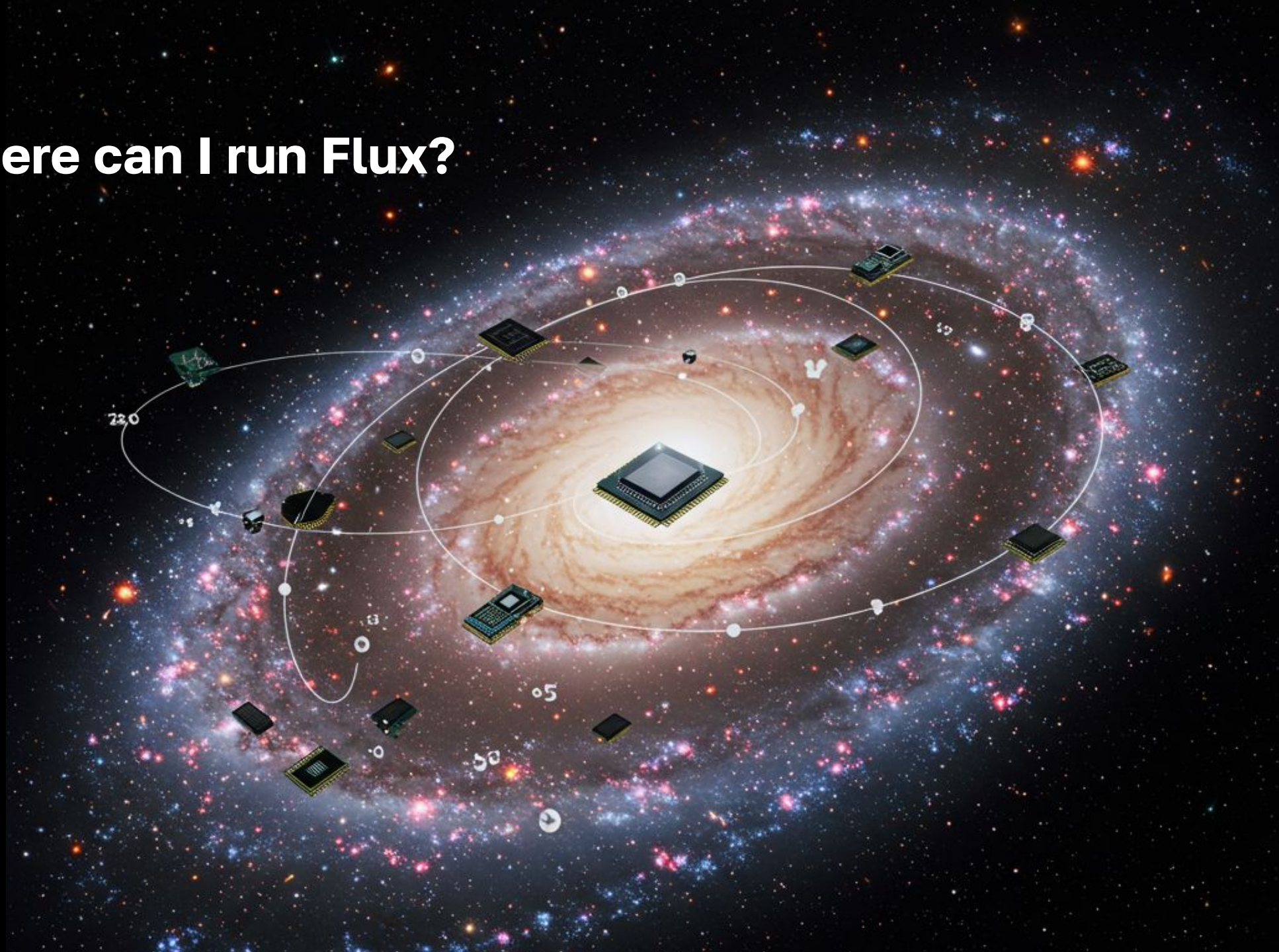
Quick Break / Informal Q&A Session

Part II

Flux Environments

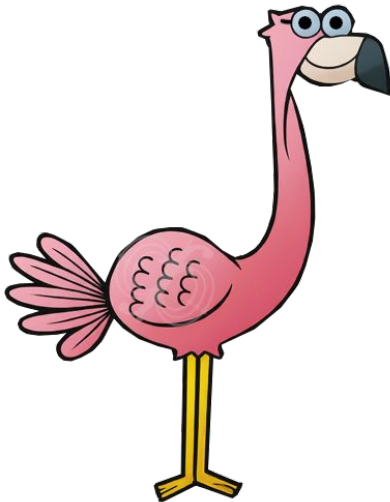


Where can I run Flux?



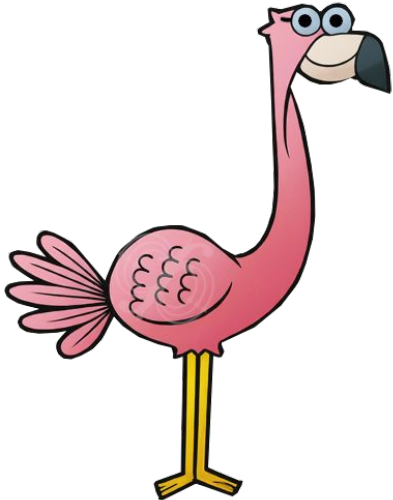
Where can I run Flux?

Hello, I'm Flux Bird, the pink bird that knows things.



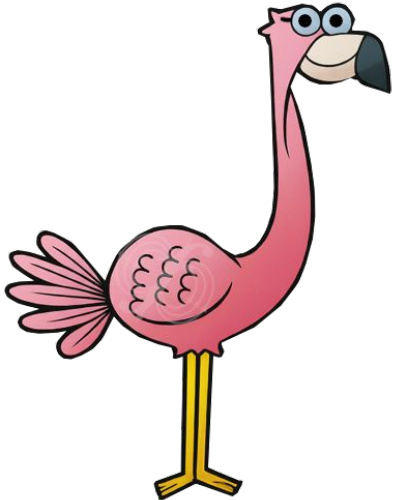
Where can I run Flux?

I think there is only one way to answer that question...

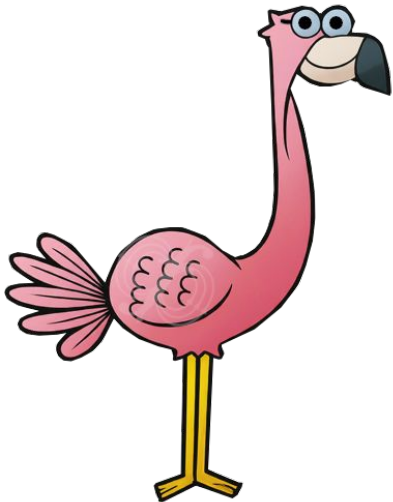


Where can I run Flux?

You can run it on a cluster,

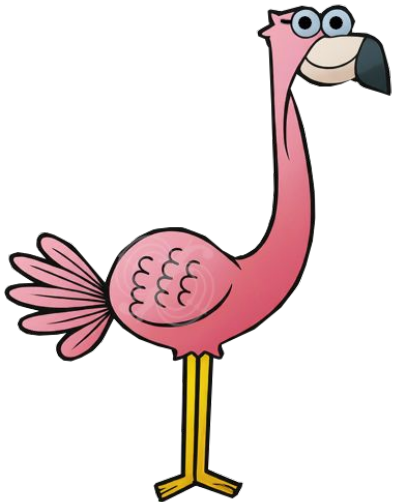


Where can I run Flux?



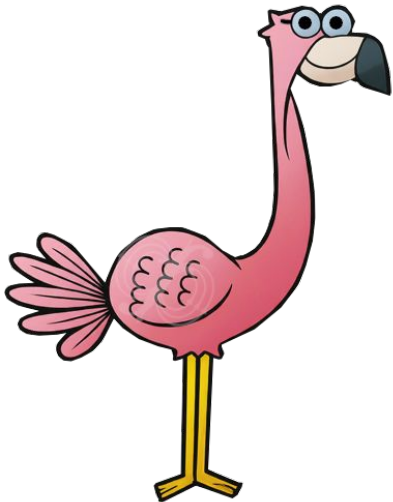
You can run it on a cluster,
You can run it alongside Lustre!

Where can I run Flux?



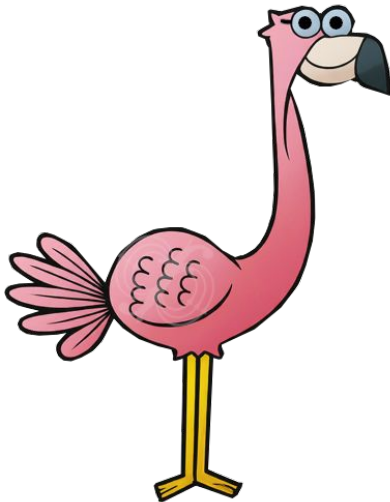
You can run it on a cluster,
You can run it alongside Lustre!
You can run it on a share...

Where can I run Flux?



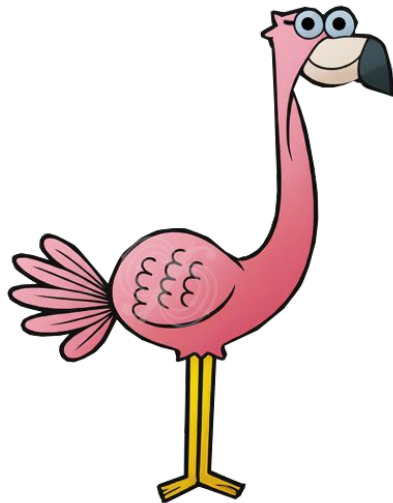
You can run it on a cluster,
You can run it alongside Lustre!
You can run it on a share...
You can run it anywhere!

Where can I run Flux?



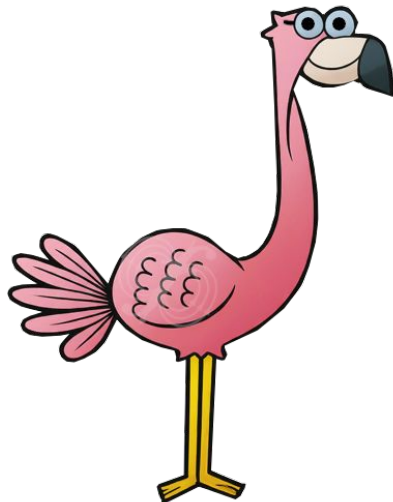
You can run it on a cluster,
You can run it alongside Lustre!
You can run it on a share...
You can run it anywhere!
You can run it in a container...

Where can I run Flux?



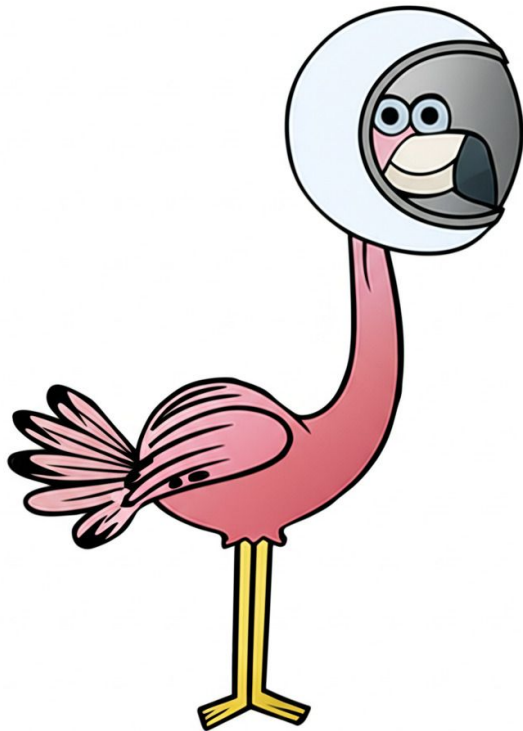
You can run it on a cluster,
You can run it alongside Lustre!
You can run it on a share...
You can run it anywhere!
You can run it in a container...
Flux is a portability layer!

Where can I run Flux?

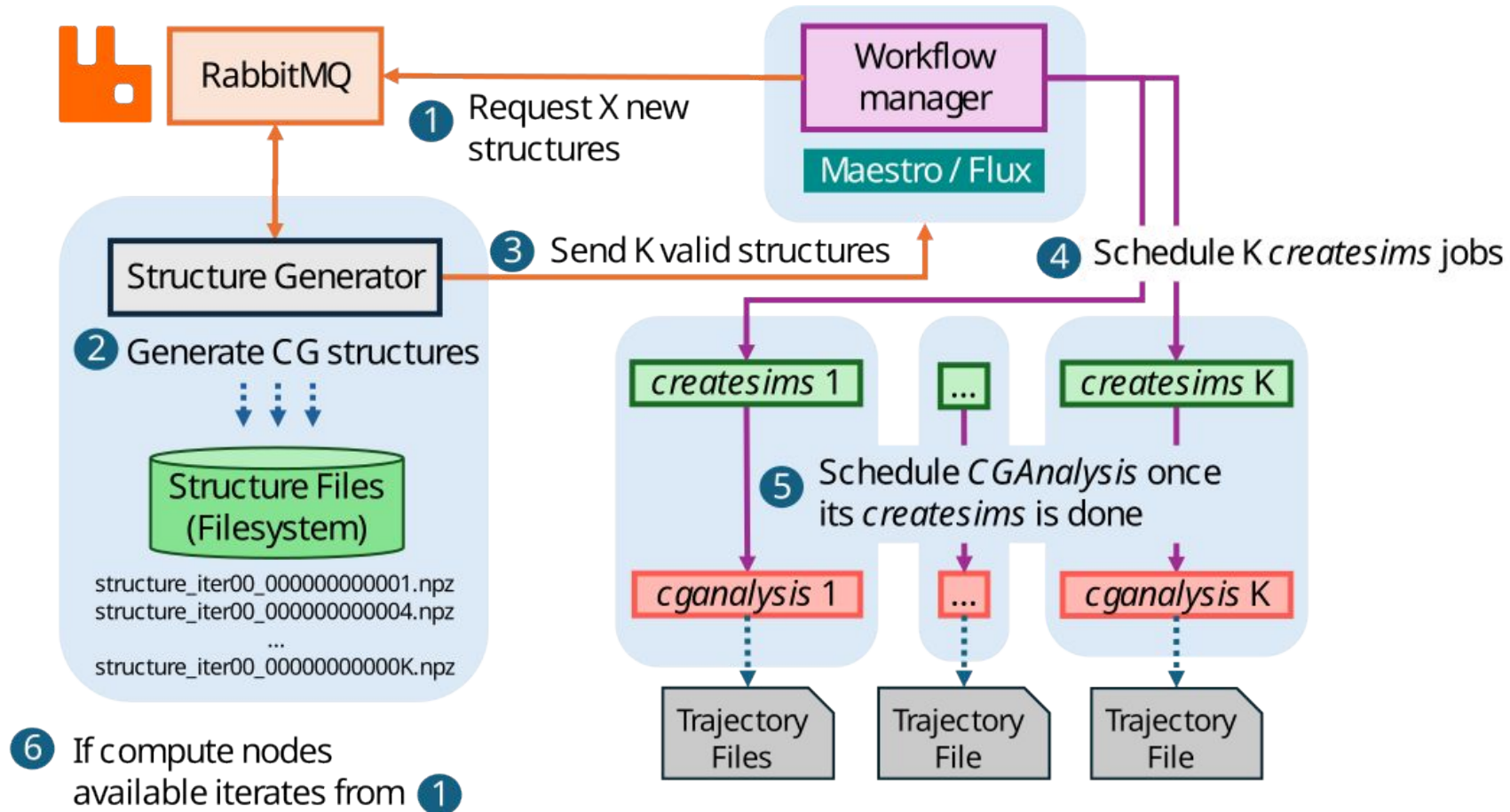


1. The system level scheduler of an HPC system
2. In user-space under another (e.g., Slurm)
3. As a single docker container
4. As an orchestrated set (e.g., docker-compose)
5. A full Flux cluster in Kubernetes (across clouds)
6. Virtual Machine setups (also across clouds)

But why do we need this portability?



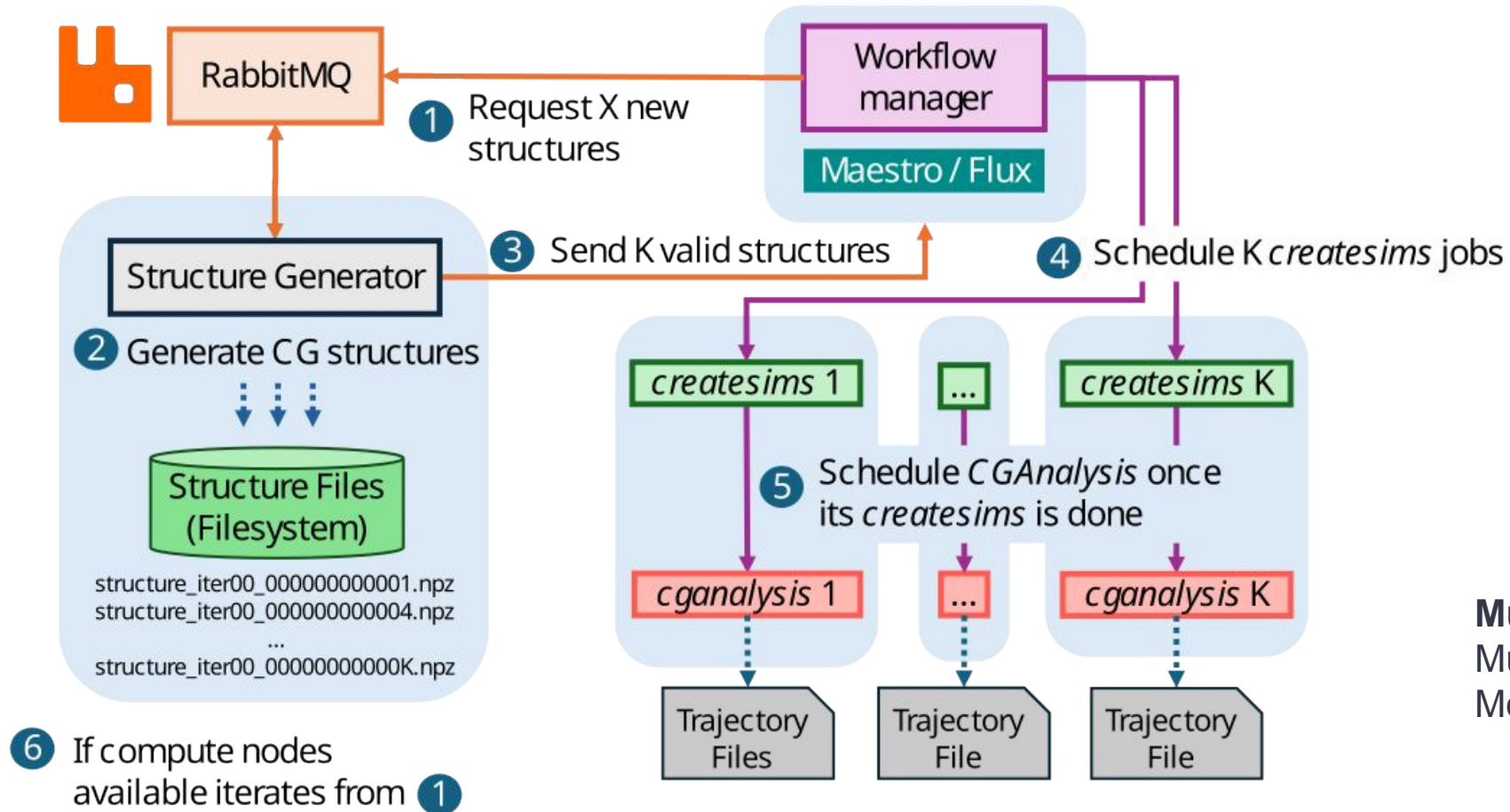
Emerging workflows have complex, heterogeneous components



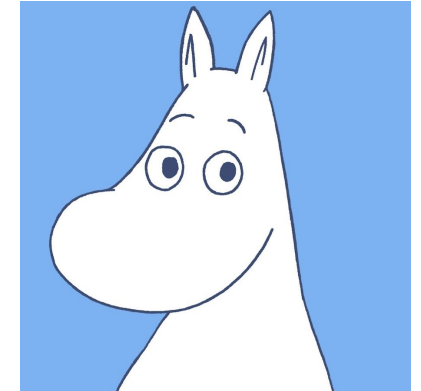
"MuMMI" models cancer-causing protein interactions - best paper SC19

"Mini MuMMI" (above) is a proxy application to it

Emerging workflows have complex, heterogeneous components



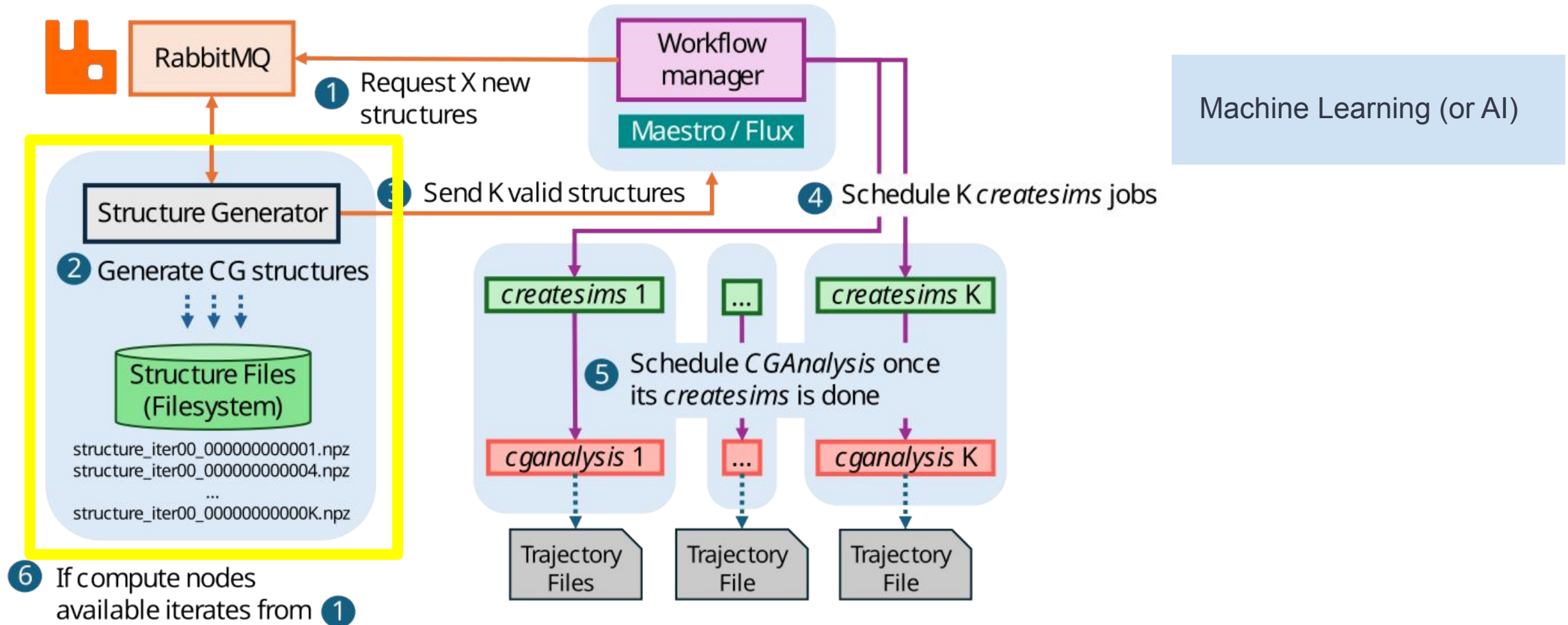
The Moomins



MuMMI:
Multiscale Machine-Learned
Modeling Infrastructure

"MuMMI" models cancer-causing protein interactions - best paper SC19
"Mini MuMMI" (above) is a proxy application to it

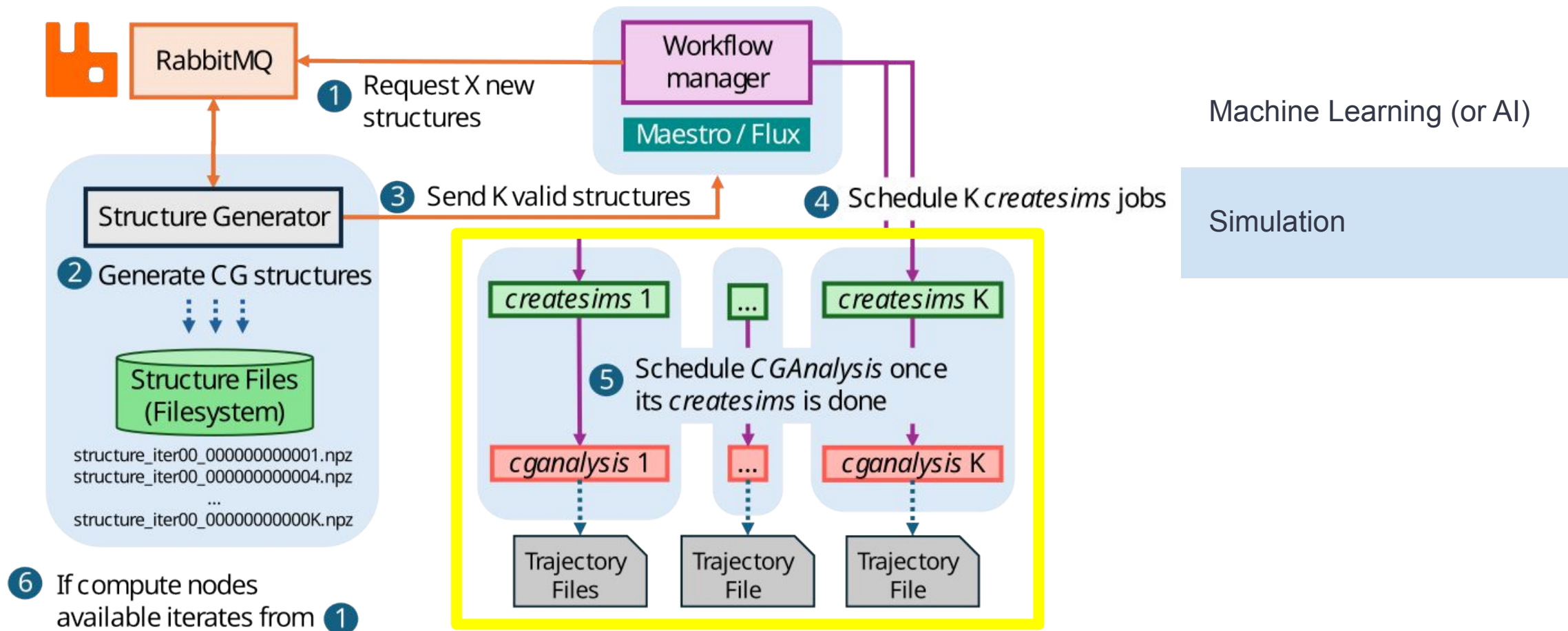
Emerging workflows have complex, heterogeneous components



"MuMMI" models cancer-causing protein interactions - best paper SC19

"Mini MuMMI" (above) is a proxy application to it

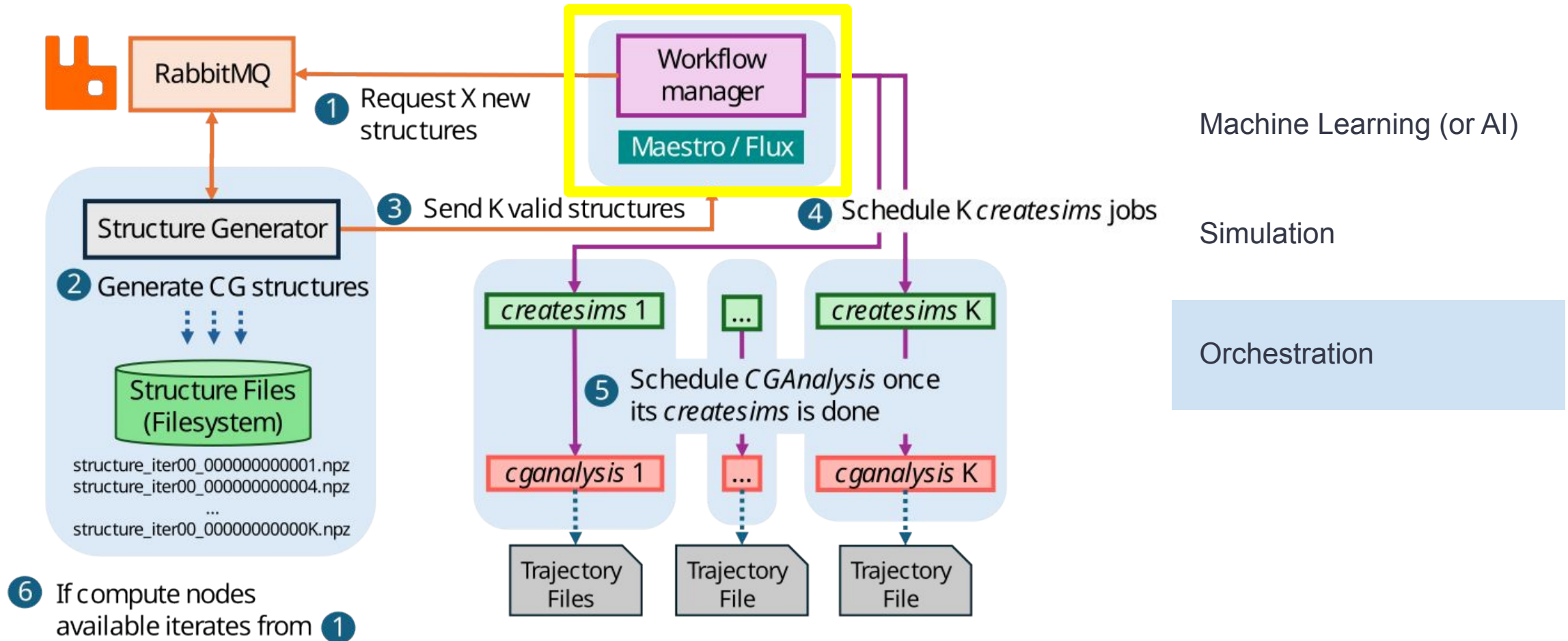
Emerging workflows have complex, heterogeneous components



"MuMMI" models cancer-causing protein interactions - best paper SC19

"Mini MuMMI" (above) is a proxy application to it

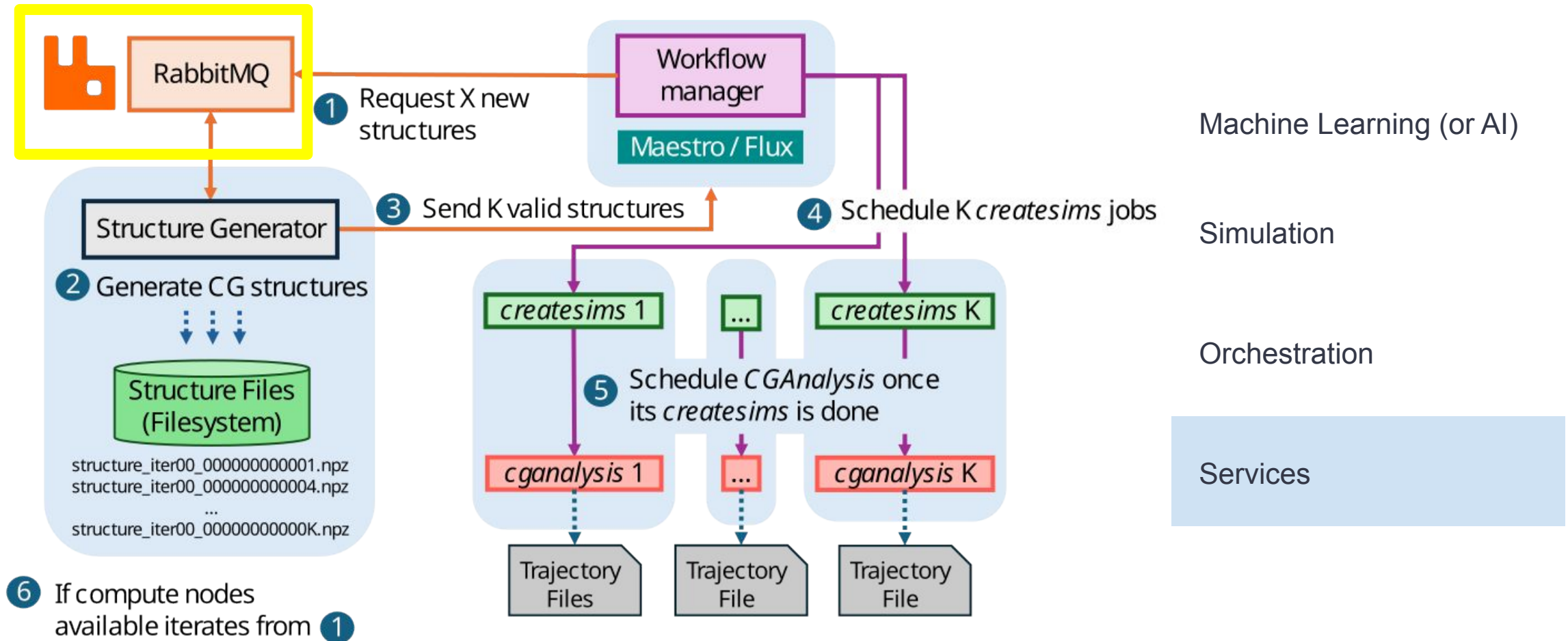
Emerging workflows have complex, heterogeneous components



"MuMMI" models cancer-causing protein interactions - best paper SC19

"Mini MuMMI" (above) is a proxy application to it

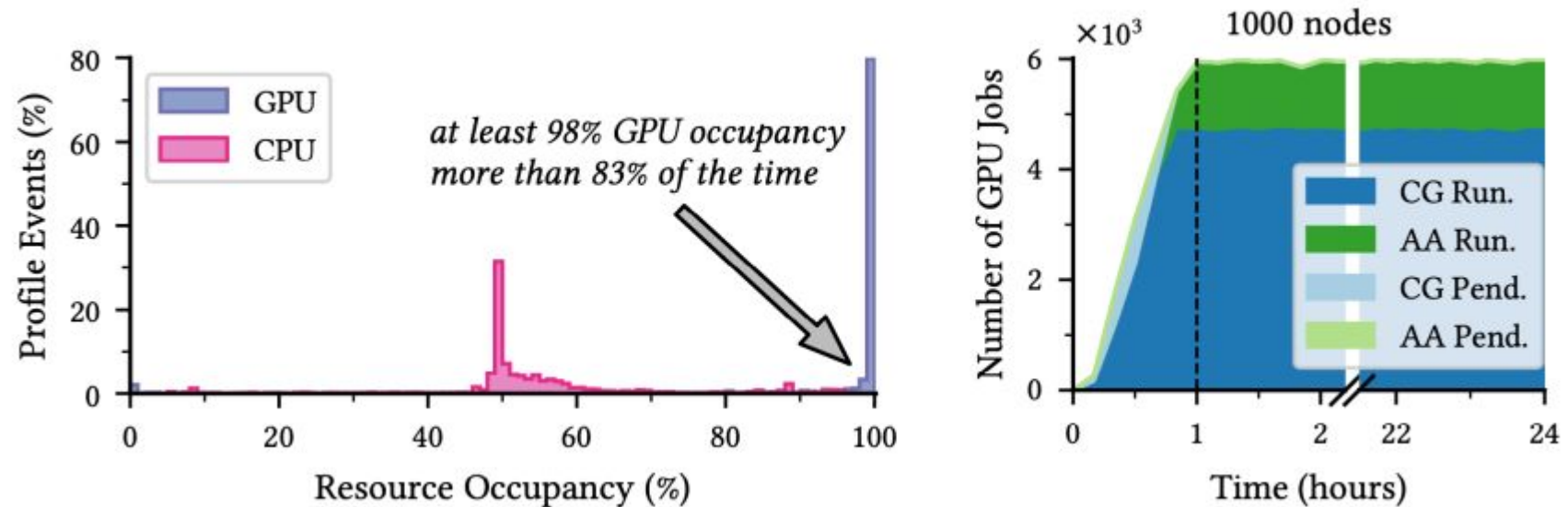
Emerging workflows have complex, heterogeneous components



"MuMMI" models cancer-causing protein interactions - best paper SC19

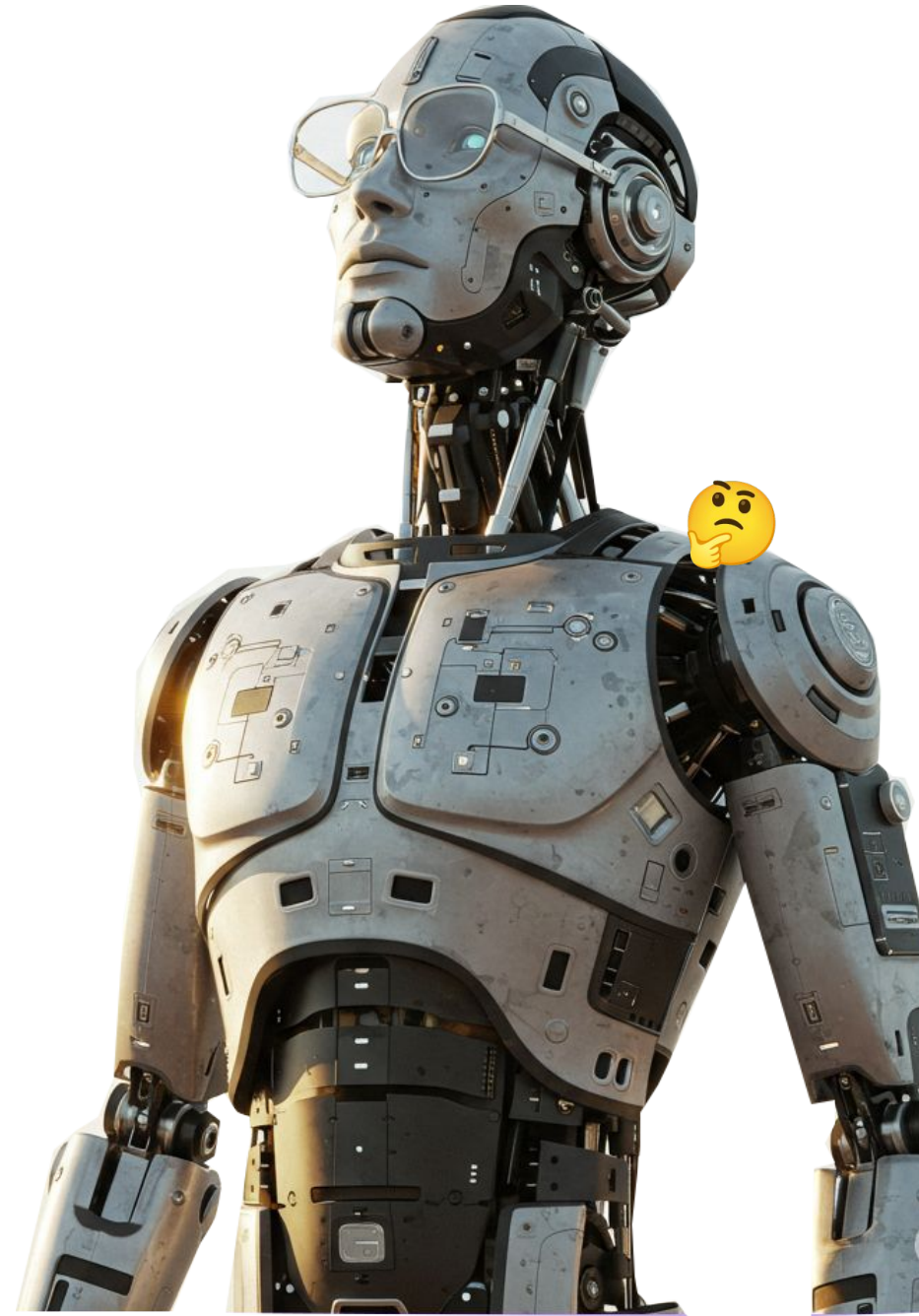
"Mini MuMMI" (above) is a proxy application to it

Traditional HPC workload managers struggle with orchestration



To run MuMMI, a DAT was required to request "all the possible resources MuMMI could possibly need, even when there was an initial ramp-up time that left most resources dormant."

**WHAT DO WE WANT THE
FUTURE TO LOOK LIKE?**



What do we want the future to look like?

- (a) **Portability**: I should be able to run my workloads and easily move between cloud and on-premises
 - Based on where they are best suited to run
 - Where resources are available (e.g., GPUs)

What do we want the future to look like?

- **(a) Portability:** I should be able to run my workloads and easily move between cloud and on-premises
 - Based on where they are best suited to run
 - Where resources are available (e.g., GPUs)



What do we want the future to look like?

- **(a) Portability:** I should be able to run my workloads and easily move between cloud and on-premises
 - Based on where they are best suited to run
 - Where resources are available (e.g., GPUs)

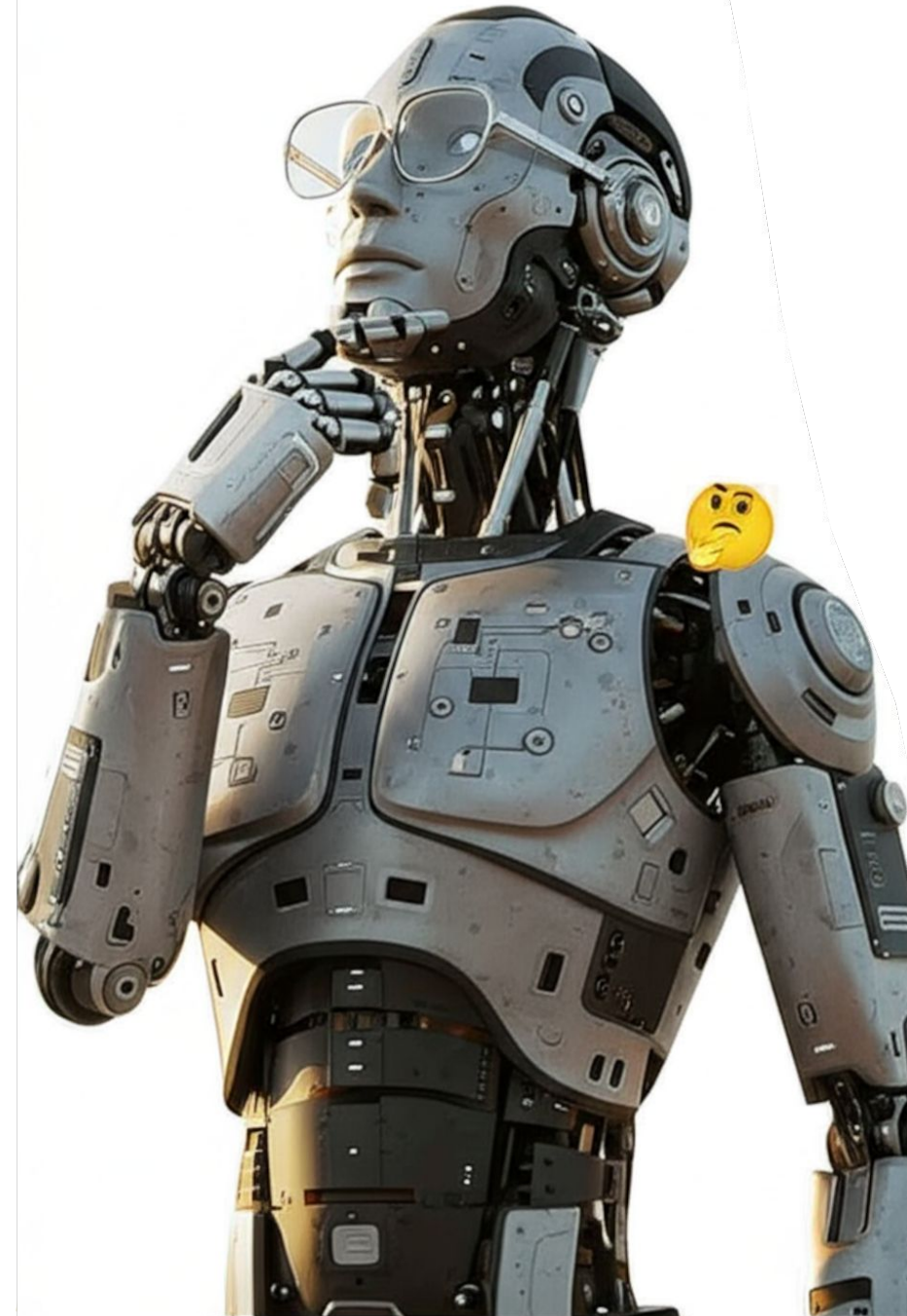


What do we want the future to look like?

- (a) **Portability:** I should be able to run my workloads and easily move between cloud and on-premises
 - Based on where they are best suited to run
 - Where resources are available (e.g., GPUs)



**PORTABILITY IS A METRIC OF
PERFORMANCE**

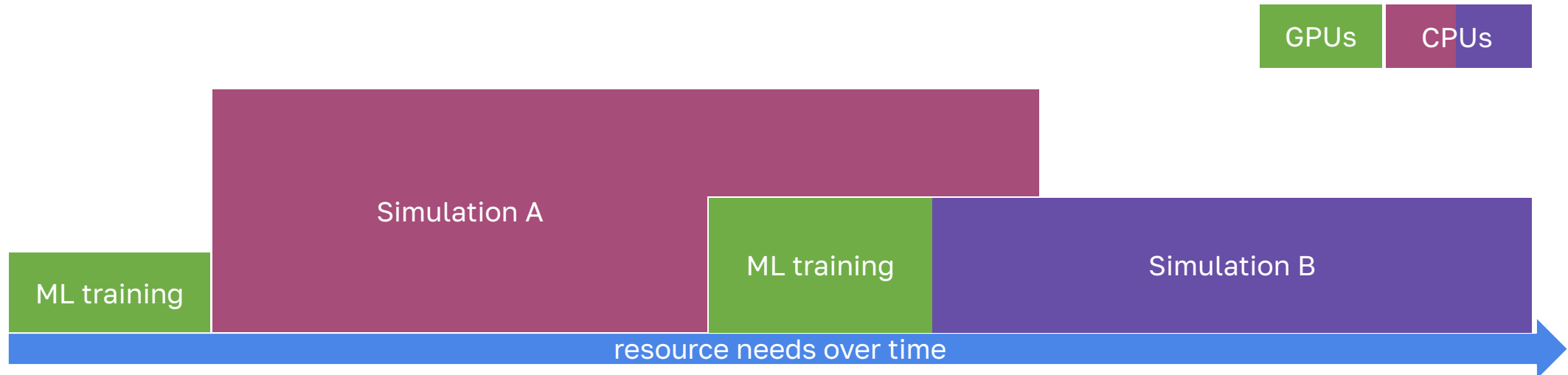


What do we want the future to look like?

- (a) **Portability**: I should be able to run my workloads and easily move between cloud and on-premises
 - Based on where they are best suited to run
 - Where resources are available (e.g., GPUs)
- (b) **Elasticity**: I want my workload manager to be elastic – scaling up and down based on workload needs

What do we want the future to look like?

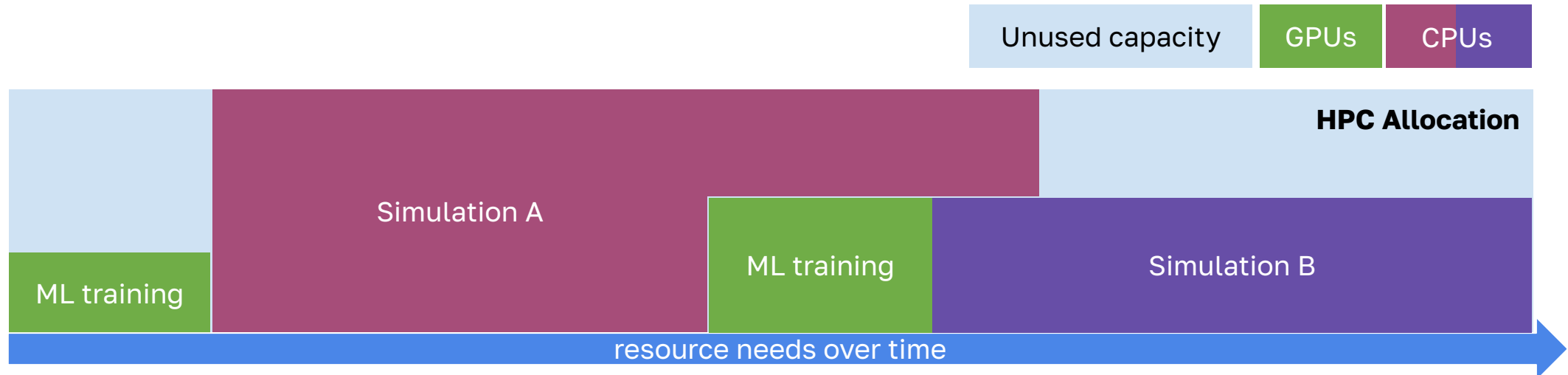
- (a) **Portability:** I should be able to run my workloads and easily move between cloud and on-premises
 - Based on where they are best suited to run
 - Where resources are available (e.g., GPUs)
- (b) **Elasticity:** I want my workload manager to be elastic – scaling up and down based on workload needs



Elasticity affording acquiring and releasing resources based on workload needs

What do we want the future to look like?

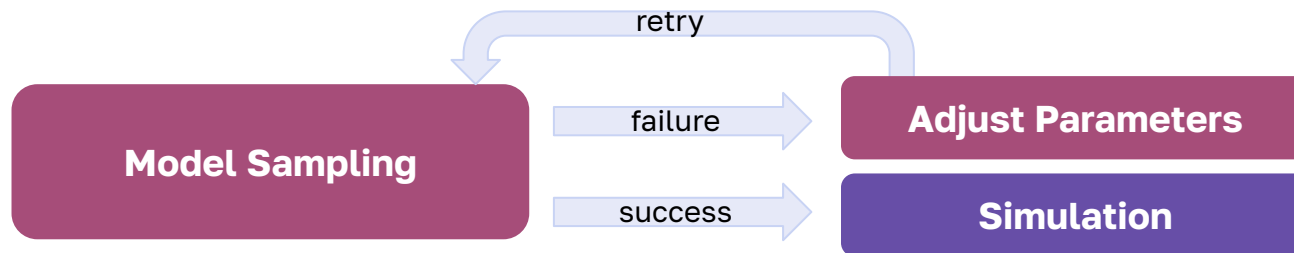
- **(a) Portability:** I should be able to run my workloads and easily move between cloud and on-premises
 - Based on where they are best suited to run
 - Where resources are available (e.g., GPUs)
- **(b) Elasticity:** I want my workload manager to be elastic – scaling up and down based on workload needs



A traditional approach with a static allocation and underutilization of resources

What do we want the future to look like?

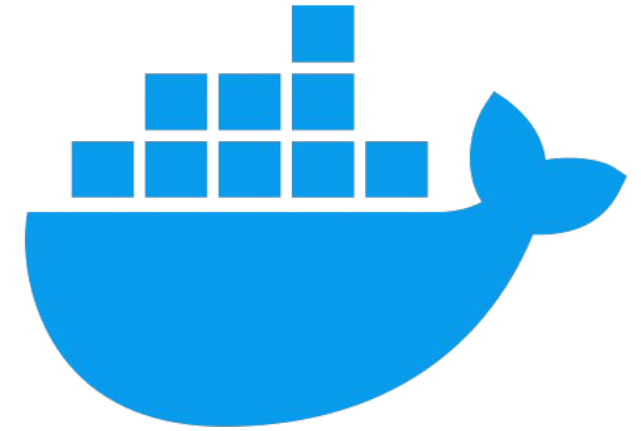
- **(a) Portability:** I should be able to run my workloads and easily move between cloud and on-premises
 - Based on where they are best suited to run
 - Where resources are available (e.g., GPUs)
- **(b) Elasticity:** I want my workload manager to be elastic – scaling up and down based on workload needs
- **(c) Dynamism:** I want my orchestration tool to respond to state
 - Traditional, static DAGs cannot account for uncertainty and loops



Environment 1: Containers

As a single docker container

```
$ docker run -it fluxrm/flux-sched:jammy
```

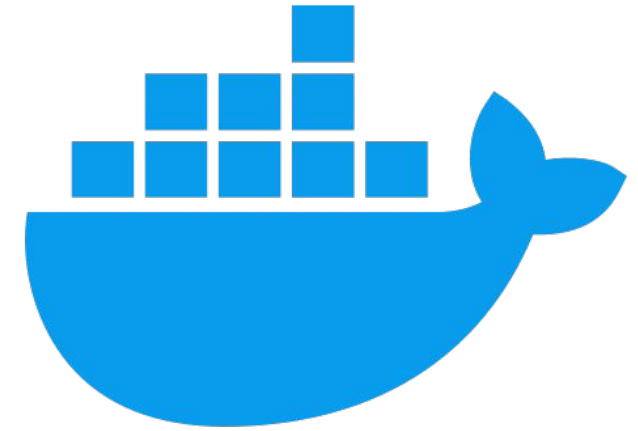


As a single docker container

```
$ docker run -it fluxrm/flux-sched:jammy
```

```
f(s=1,d=0) fluxuser@544ee10f63d1:~$ flux resource list
```

STATE	NNODES	NCORES	NGPUS	NODELIST
free	1	8	0	544ee10f63d1
allocated	0	0	0	
down	0	0	0	



As a single docker container

```
$ docker run -it fluxrm/flux-sched:jammy
```

```
f(s=1,d=0) fluxuser@544ee10f63d1:~$ flux resource list
```

STATE	NNODES	NCORES	NGPUS	ODELIST
free	1	8	0	544ee10f63d1
allocated	0	0	0	
down	0	0	0	

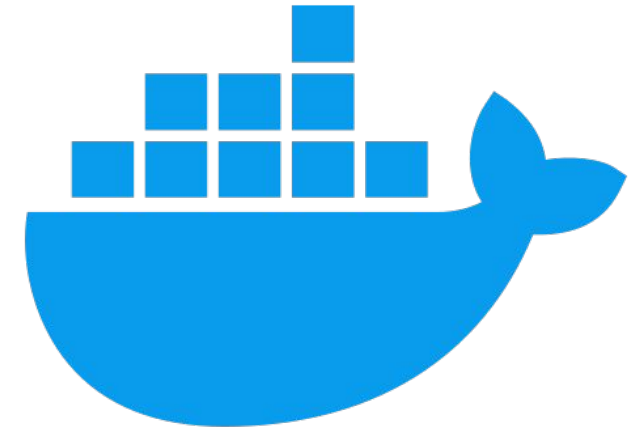
```
f(s=1,d=0) fluxuser@544ee10f63d1:~$ flux run -n 4 hostname
```

```
544ee10f63d1
```

```
544ee10f63d1
```

```
544ee10f63d1
```

```
544ee10f63d1
```



As an orchestrated set with docker-compose

```
$ docker compose up -d
```

```
$ docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED
rabbitmq	rabbitmq:3-management	"docker-entrypoint.s..."	rabbit	7 seconds ago
replicas-node-1	replicas-node	"/bin/sh -c '/bin/ba..."	node	7 seconds ago
replicas-node-2	replicas-node	"/bin/sh -c '/bin/ba..."	node	7 seconds ago
replicas-node-3	replicas-node	"/bin/sh -c '/bin/ba..."	node	7 seconds ago
replicas-node-4	replicas-node	"/bin/sh -c '/bin/ba..."	node	7 seconds ago

```
$ docker compose logs node -f
```

```
replicas-node-2 | broker.info[1]: rc1.0: running /etc/flux/rc1.d/01-sched-fluxion
replicas-node-2 | broker.info[1]: rc1.0: running /etc/flux/rc1.d/02-cron
replicas-node-2 | broker.info[1]: rc1.0: /etc/flux/rc1 Exited (rc=0) 0.2s
replicas-node-2 | broker.info[1]: rc1-success: init->quorum 0.178977s
replicas-node-2 | broker.info[1]: quorum-full: quorum->run 0.53695s
```

```
$ docker exec -it replicas-node-1 bash
```

```
fluxuser@2db537b6ba16:~$ flux resource list
```

STATE	NNODES	NCORES	NODELIST
free	4	4	replicas-node-[1-4]
allocated	0	0	
down	0	0	



<https://github.com/rse-ops/flux-compose>



As a single docker container

```
$ docker run -it fluxrm/flux-sched:jammy
```

```
f(s=1,d=0) fluxuser@544ee10f63d1:~$ flux resource list
```

STATE	NNODES	NCORES	NGPUS	ODELIST
free	1	8	0	544ee10f63d1
allocated	0	0	0	
down	0	0	0	

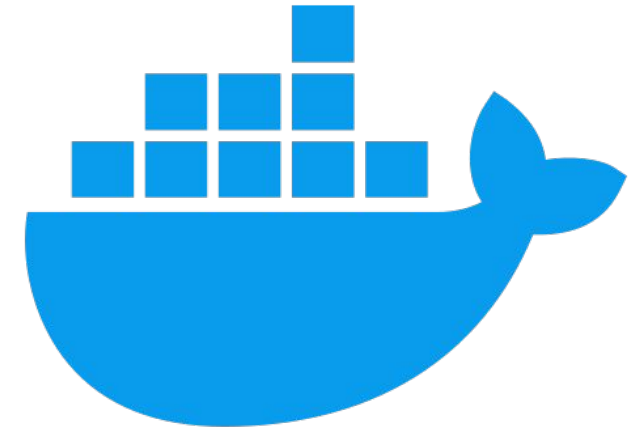
```
f(s=1,d=0) fluxuser@544ee10f63d1:~$ flux run -n 4 hostname
```

```
544ee10f63d1
```

```
544ee10f63d1
```

```
544ee10f63d1
```

```
544ee10f63d1
```

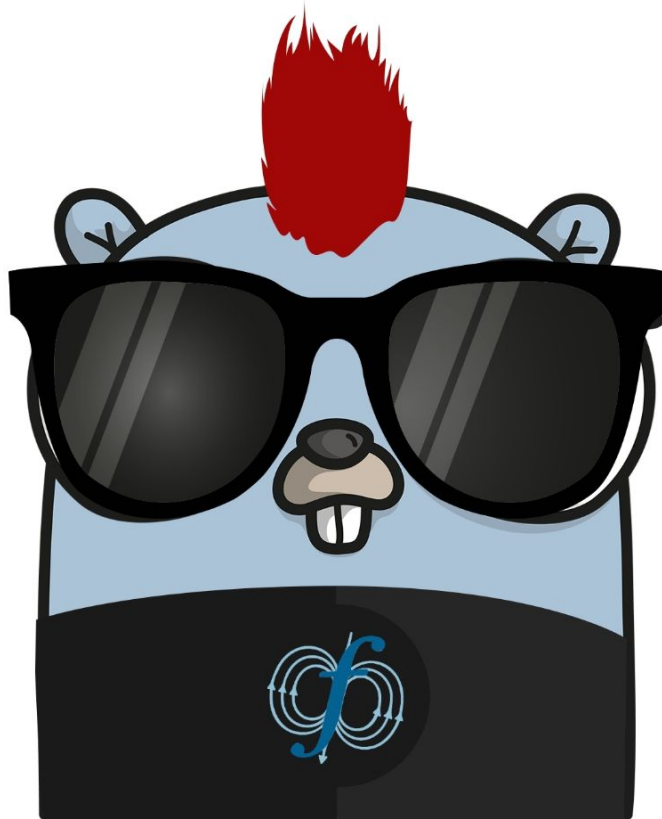


flux start --test-size=4

Environment 2: Kubernetes

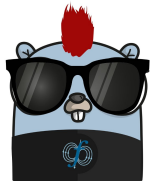
As a complete multi-node cluster in Kubernetes

The Flux Operator



We can map Flux components into containers and Kubernetes abstractions to implement the entirety of Flux inside of Kubernetes via an *operator*, a controller that knows how to manage objects in a Kubernetes cluster.

The Flux Operator: Flux inside of Kubernetes



MiniCluster Definition

container: <application>
command: <command>
size: <nodes>

Reconcile

Kubernetes

follower

leader

follower

follower

follower

follower

follower

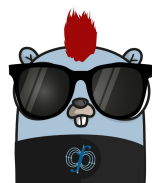
follower

follower

Flux MiniCluster



The Flux Operator: Flux inside of Kubernetes



MiniCluster Definition

container: <application>
command: <command>
size: <nodes>

Reconcile

Kubernetes

follower

leader

follower

follower

follower

follower

follower

follower

follower

Flux MiniCluster

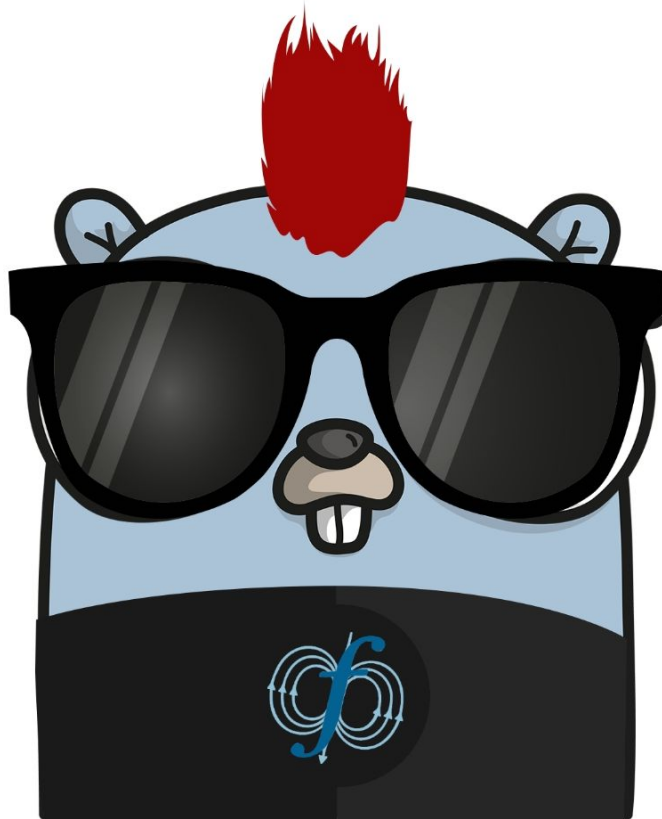
Did you know!

The Flux Operator has enabled work for two Kubecon talks, several publications running HPC applications on cloud (e.g., ensembles with MPI), and was deployed across Kubernetes environments in Azure, AWS, and Google Cloud for each of GPU and CPU for ~14 applications for a recent performance study.



As a complete multi-node cluster in Kubernetes

The Flux Operator



```
kind create cluster --config ./kind-config.yaml  
kubectl apply -f flux-cluster.yaml  
kubectl exec -it flux-sample-0-xxxx bash
```

```
source /mnt/flux/flux-view.sh  
flux proxy $fluxsocket bash  
flux resource list
```

STATE	NNODES	NCORES	NGPUS	NODELIST
free	4	160	0	flux-sample-[0-3]
allocated	0	0	0	
down	0	0	0	

<https://github.com/flux-framework/flux-operator>

Helm Charts for HPC Experiments

AMG2023

```
helm dependency update amg2023/
helm install \
  --set experiment.nodes=16 \
  --set minicluster.size=16 \
  --set minicluster.tasks=1408 \
  --set minicluster.save_logs=true \
  --set amg.problem_size="256 256 128" \
  --set amg.processor_topology="8 16 11" \
  --set experiment.iterations=5 \
  --set experiment.tasks=1408 \
  amg amg2023/

time kubectl wait --for=condition=ready pod -l job-name=amg --timeout=600s
pod=$(kubectl get pods -o json | jq -r .items[0].metadata.name)
kubectl logs ${pod} -f |& tee ./logs/amg.out
helm uninstall amg
```

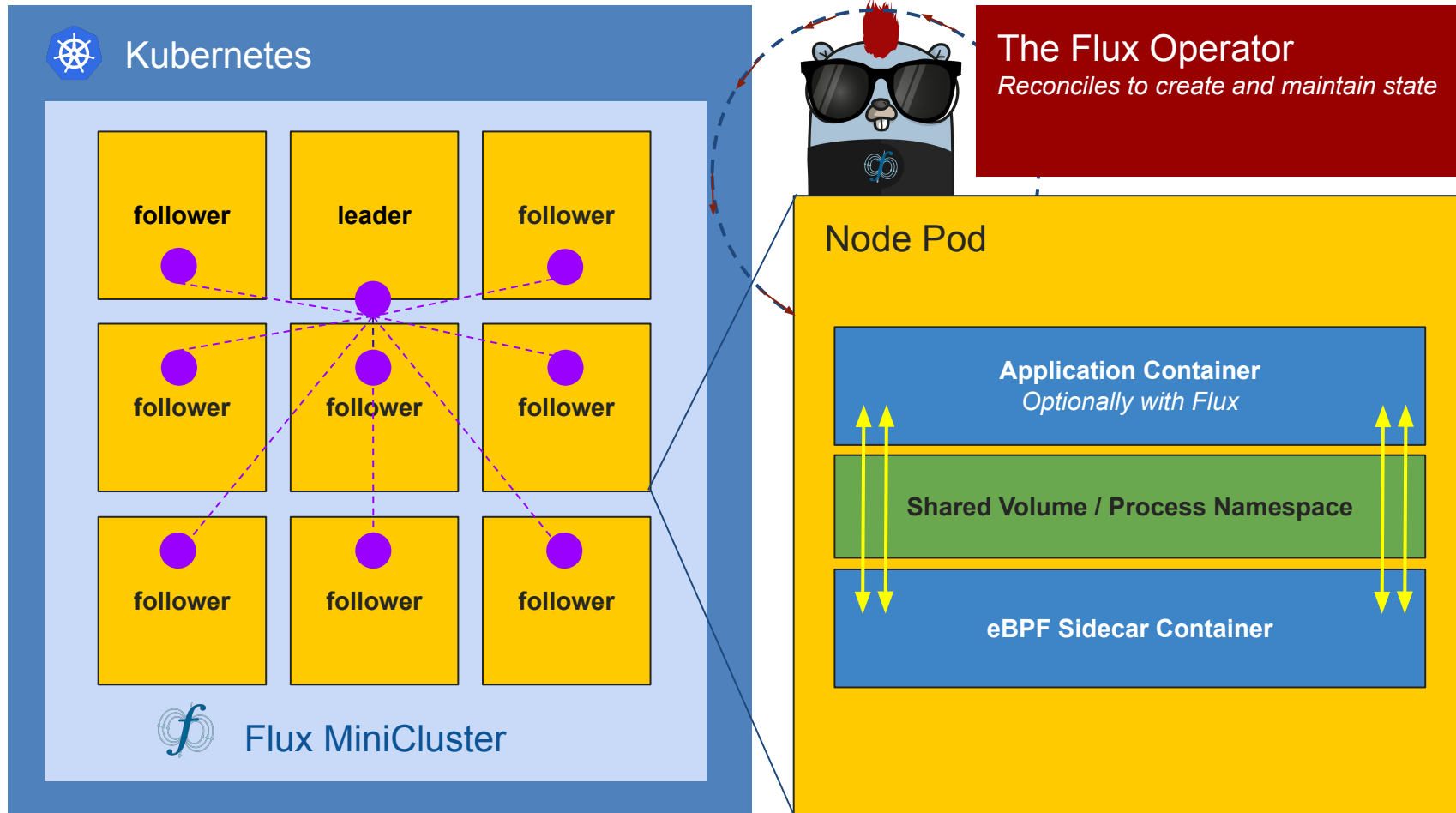
Applications

AMG2023	mt-gemm
LAMMPS	miniFE
Kripke	Quicksilver
Laghos	Single-node benchmark
Stream	
OSU Benchmarks	mfem
Mixbench	... ~30 total!



Collect metrics with the eXtended Berkeley Packet Filter

eBPF



Collect metrics with the eXtended Berkeley Packet Filter

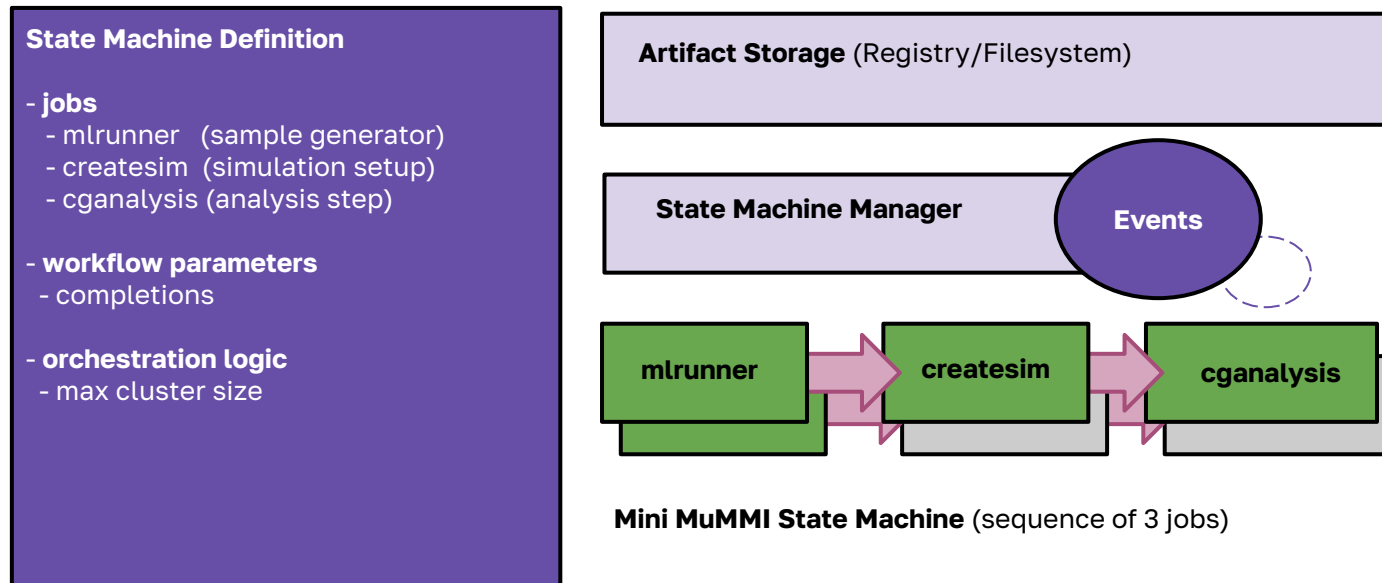
eBPF

```
helm install \
  --set experiment.nodes=64 \
  --set minicluster.size=64 \
  --set minicluster.tasks=5632 \
  --set experiment.tasks=5632 \
  --set minicluster.save_logs=true \
  --set lammps.x=32 \
  --set lammps.y=16 \
  --set lammps.z=16 \
  --set experiment.monitor="tcp|cpu|open_close|futex|shmem" \
  --set experiment.iterations=3 \
  lammps ./lammps-reax
```

Modeling MuMMI as a state machine with event-driven orchestration

✓ Portability ✓ Elasticity ✓ Dynamism

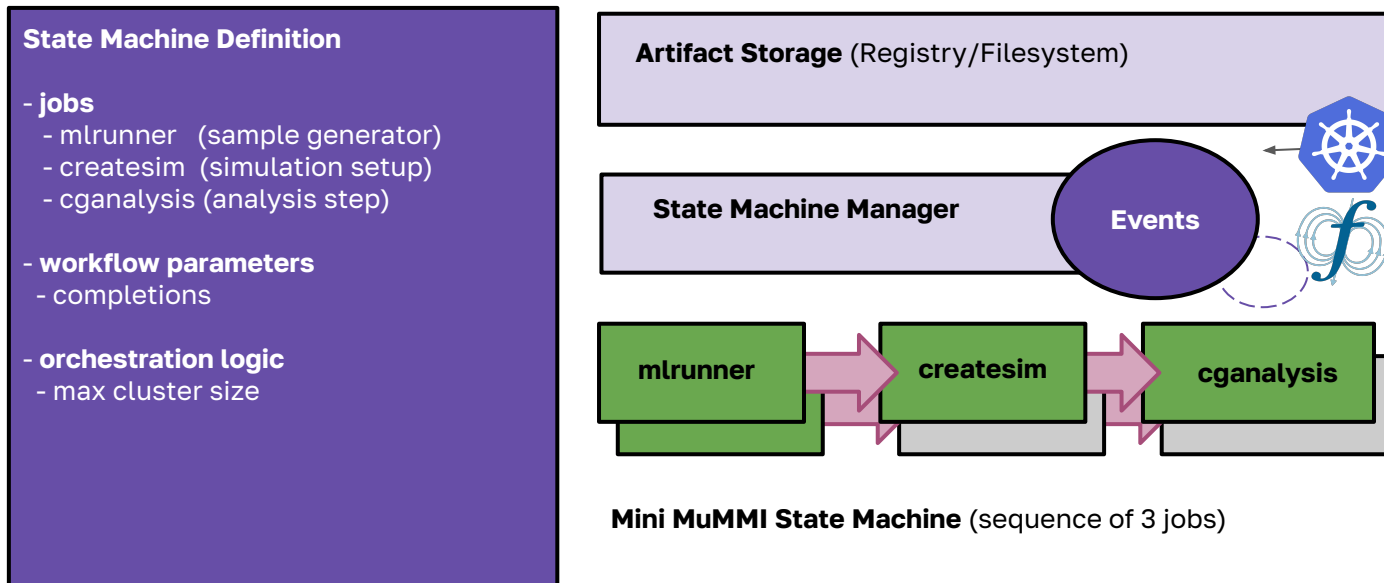
Goal-oriented workflows where each job is an independent, dynamic unit



Modeling MuMMI as a state machine with event-driven orchestration

✓ Portability ✓ Elasticity ✓ Dynamism

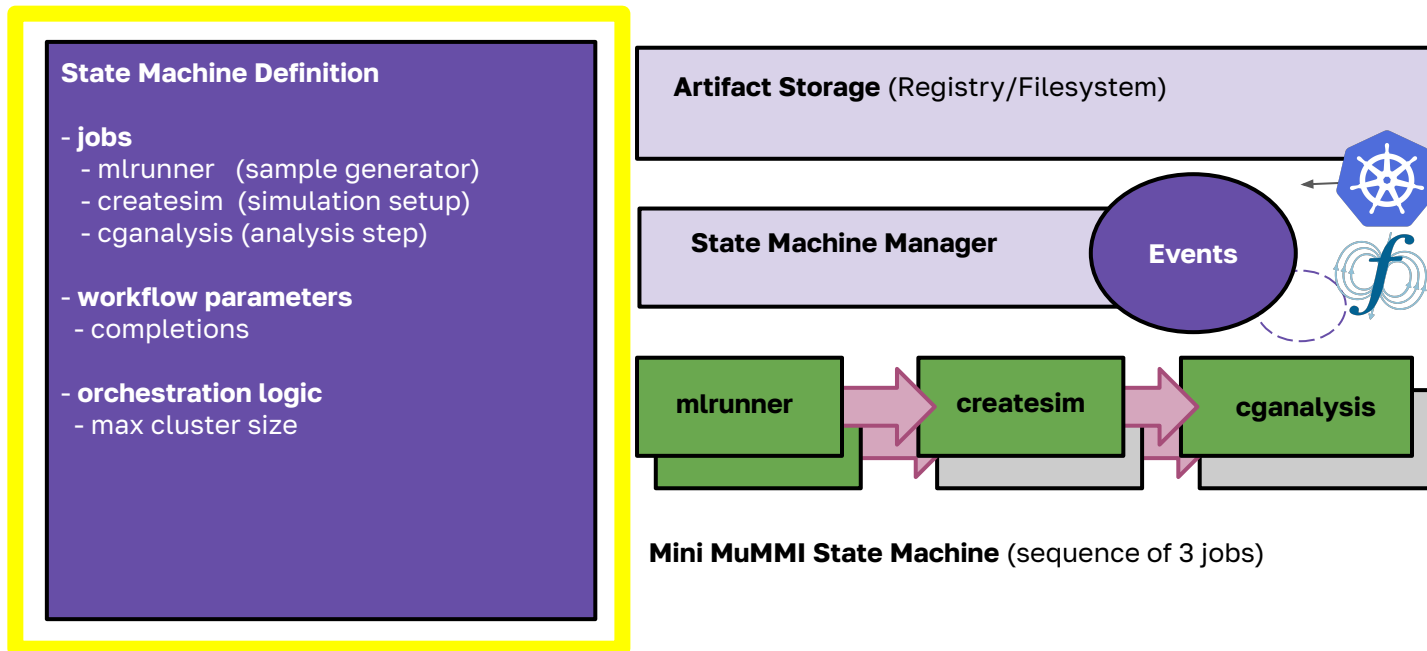
We can run the same state machines with Flux or Kubernetes equivalently!



Modeling MuMMI as a state machine with event-driven orchestration

✓ Portability ✓ Elasticity ✓ Dynamism

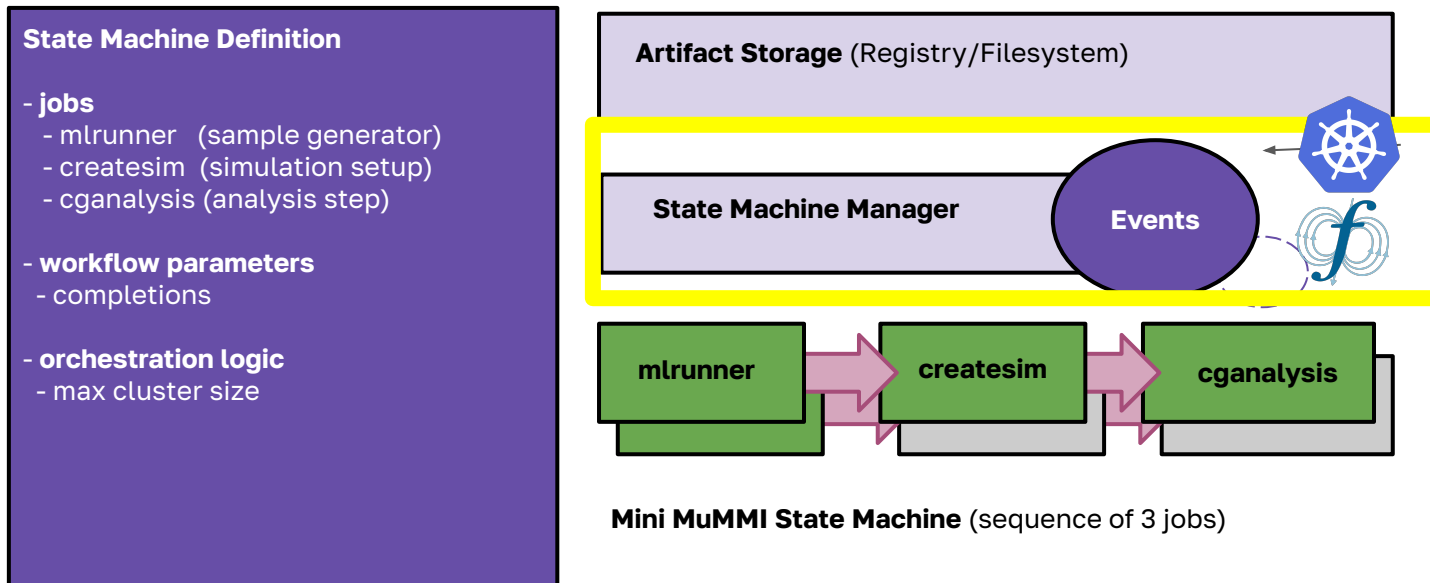
We can run the same state machines with Flux or Kubernetes equivalently!



Modeling MuMMI as a state machine with event-driven orchestration

✓ Portability ✓ Elasticity ✓ Dynamism

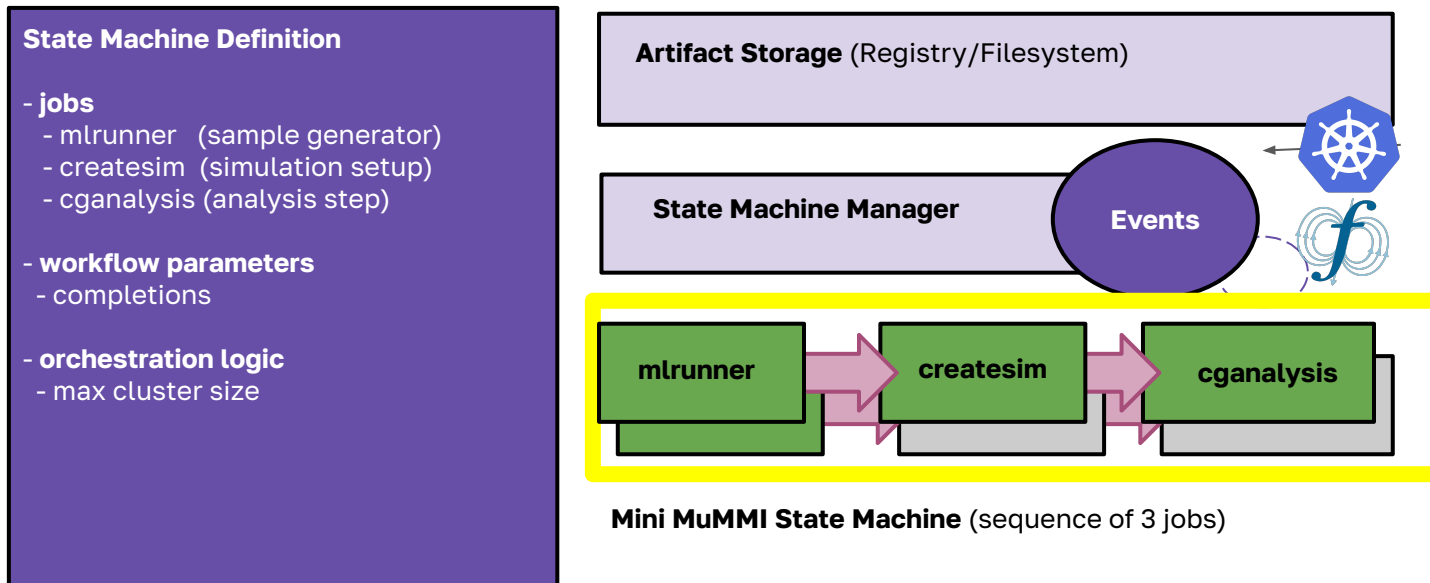
We can run the same state machines with Flux or Kubernetes equivalently!



Modeling MuMMI as a state machine with event-driven orchestration

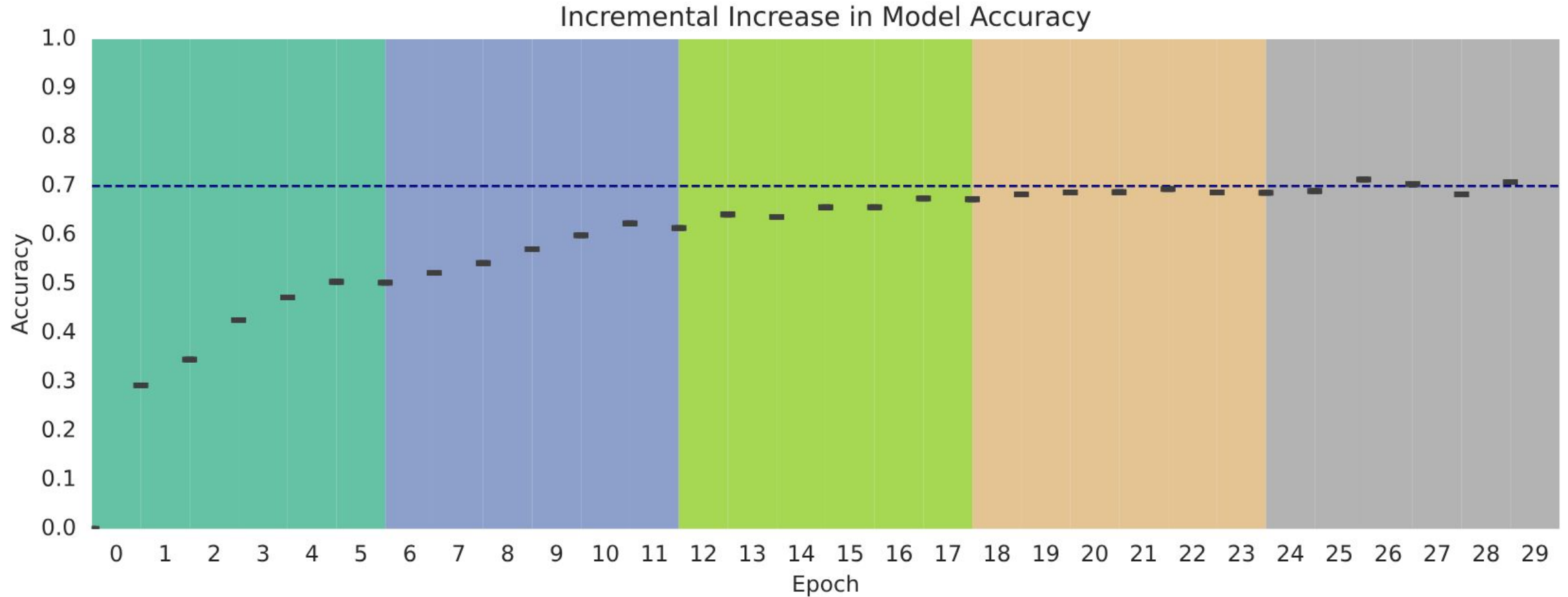
✓ Portability ✓ Elasticity ✓ Dynamism

We can run the same state machines with Flux or Kubernetes equivalently!



State Machine orchestration with metrics - streaming ML models

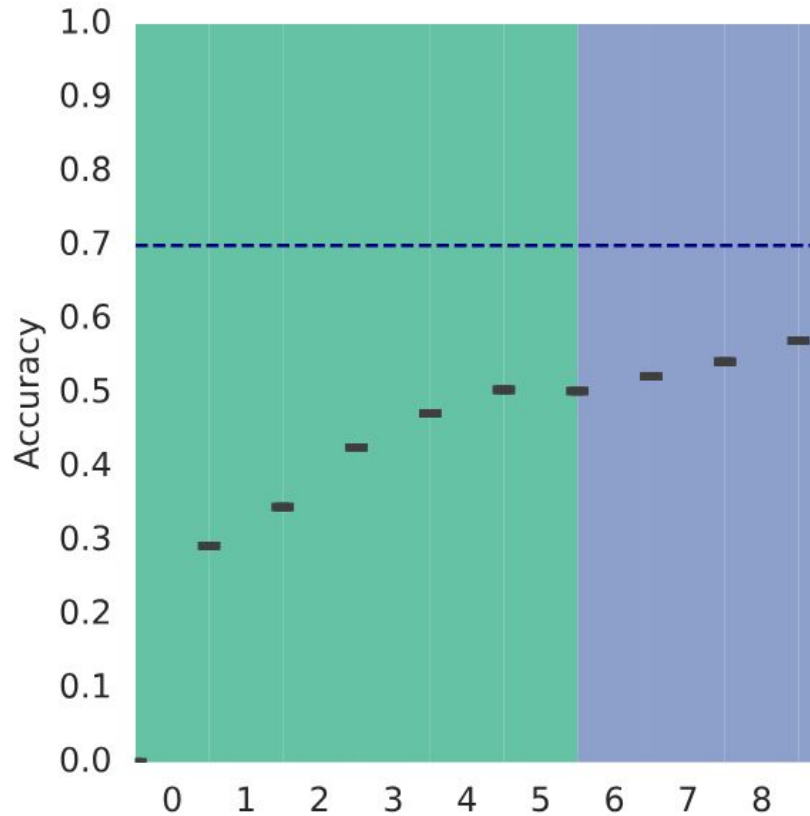
"Keep training this model until you reach this metric of goodness."



Sochat, Pottier, Milroy 2025 (under review)

State Machine orchestration with metrics - streaming ML models

"Keep training this model until you reach this metric of goodness."

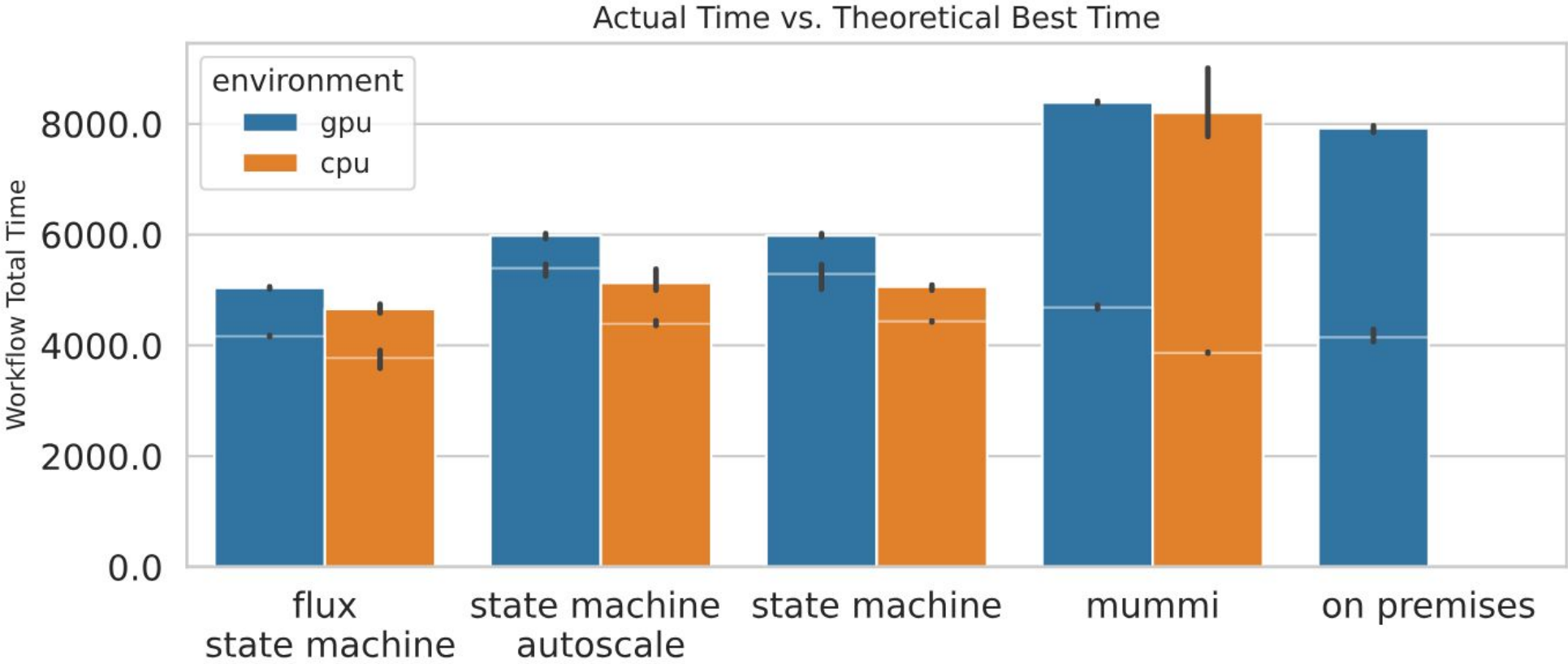


```
apiVersion: state-machine.converged-computing.org/v1alpha1
kind: StateMachine
metadata:
  name: state-machine
spec:
  manager:
    interactive: true
    pullPolicy: Always
  workflow:
    completed: 1
    events:
      # Finish the workflow when accuracy good enough
      # This is a workflow action
      - metric: max.pytorch.accuracy
        when: ">= 0.7"
        action: finish-workflow
      # Repeat the step (new job) if accuracy not good enough, or not yet defined
      # The model starts fluctuating (accuracy) around 0.6
      - metric: max.pytorch.accuracy
        when: "< 0.7"
        action: repeat
```

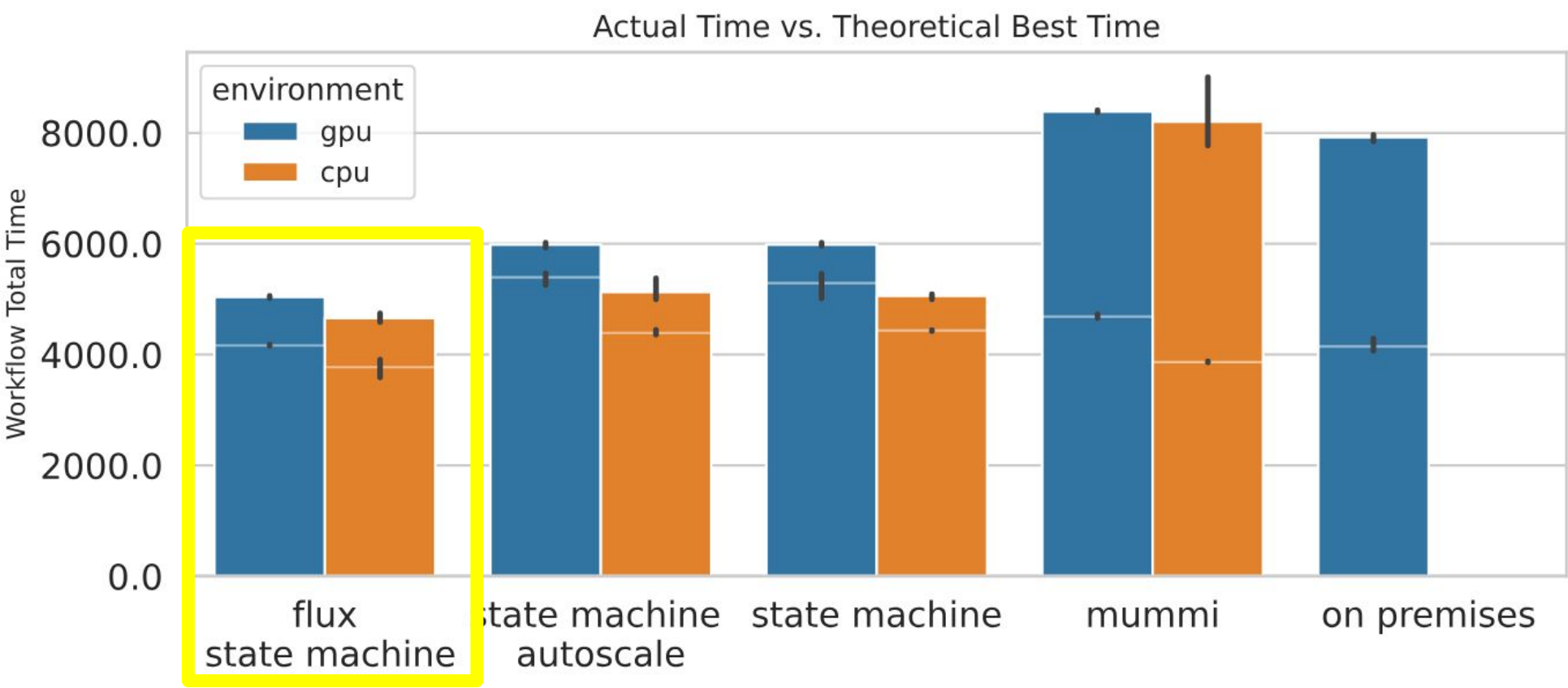
Metrics can be job, queue, or parsed directly from output logs

```
# Event parsing. This returns the last model accuracy, which reflects the last epoch run
events:
script: |
    def parse_log(log):
        import re
        # Find the last line with reported accuracy (each epoch is reported)
        log = [x for x in log.split('\n') if "Accuracy" in x][-1]
        match = re.search('Accuracy: (?P<accuracy>.*)', log)
        accuracy = float(match.groupdict()['accuracy'])
        return {"accuracy": accuracy}
```

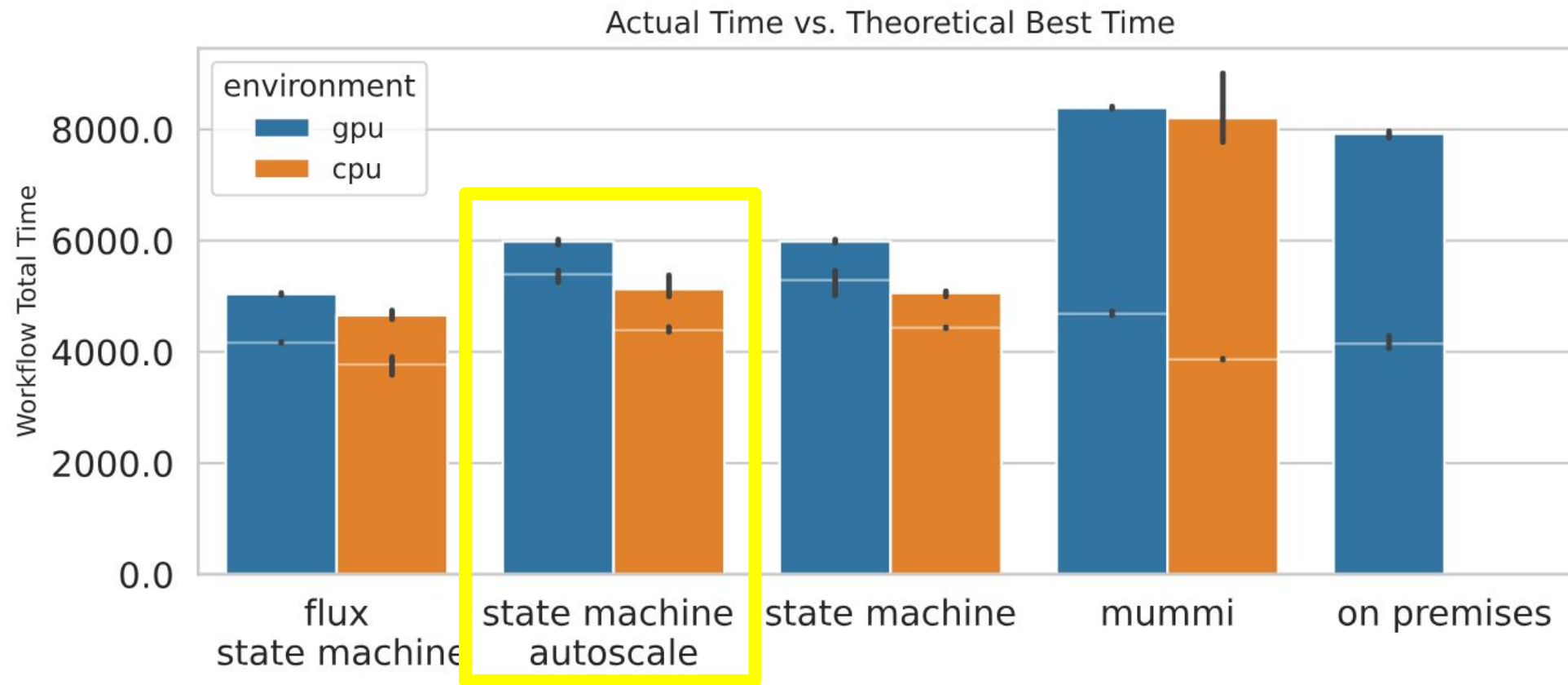
State Machine orchestration improves upon time to completion



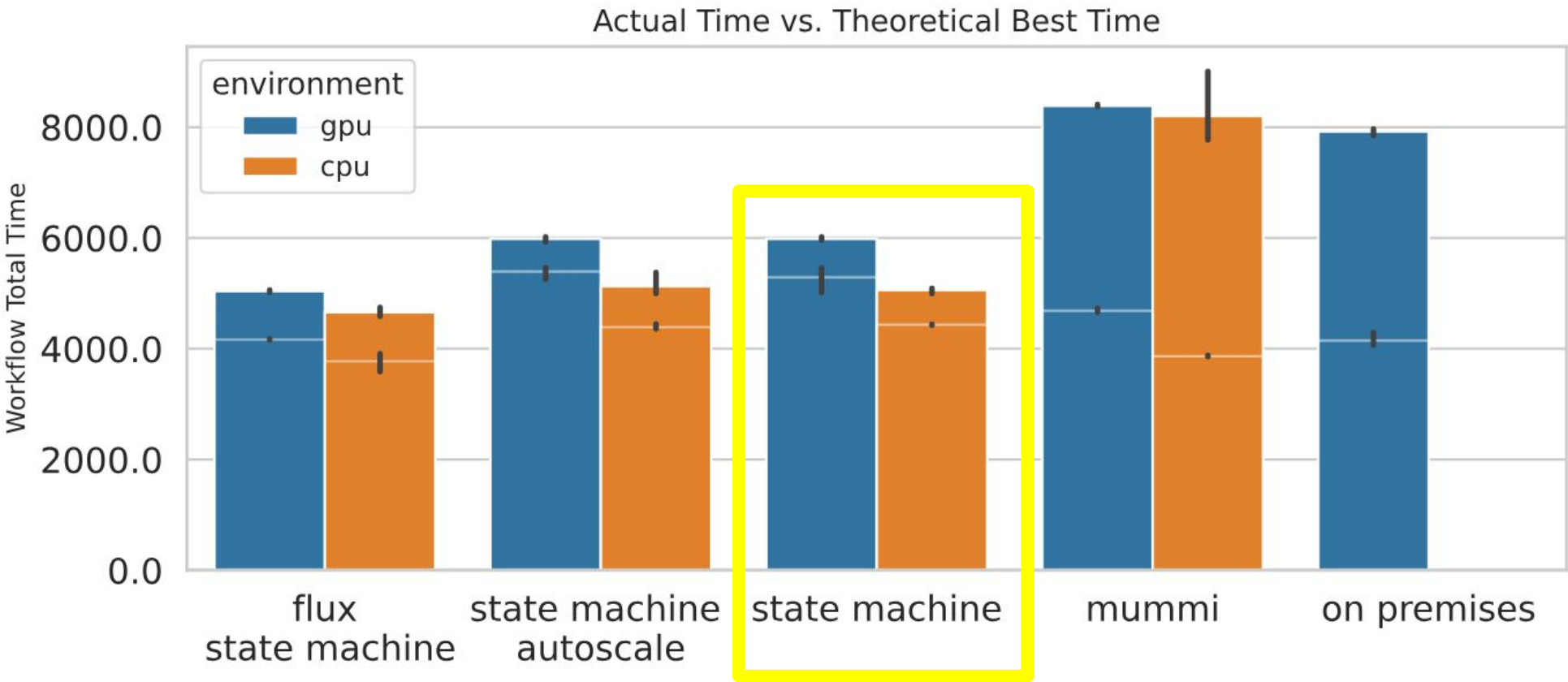
State Machine orchestration improves upon time to completion



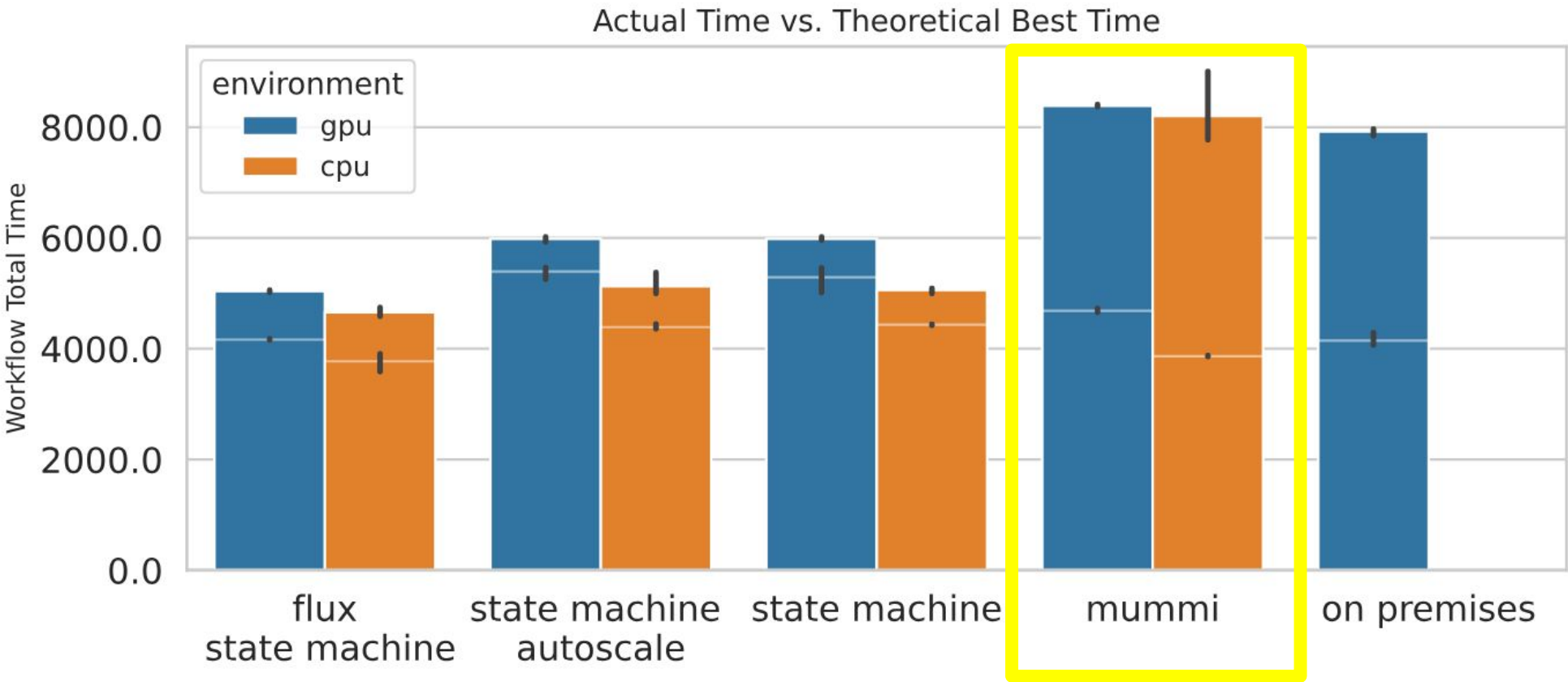
State Machine orchestration improves upon time to completion



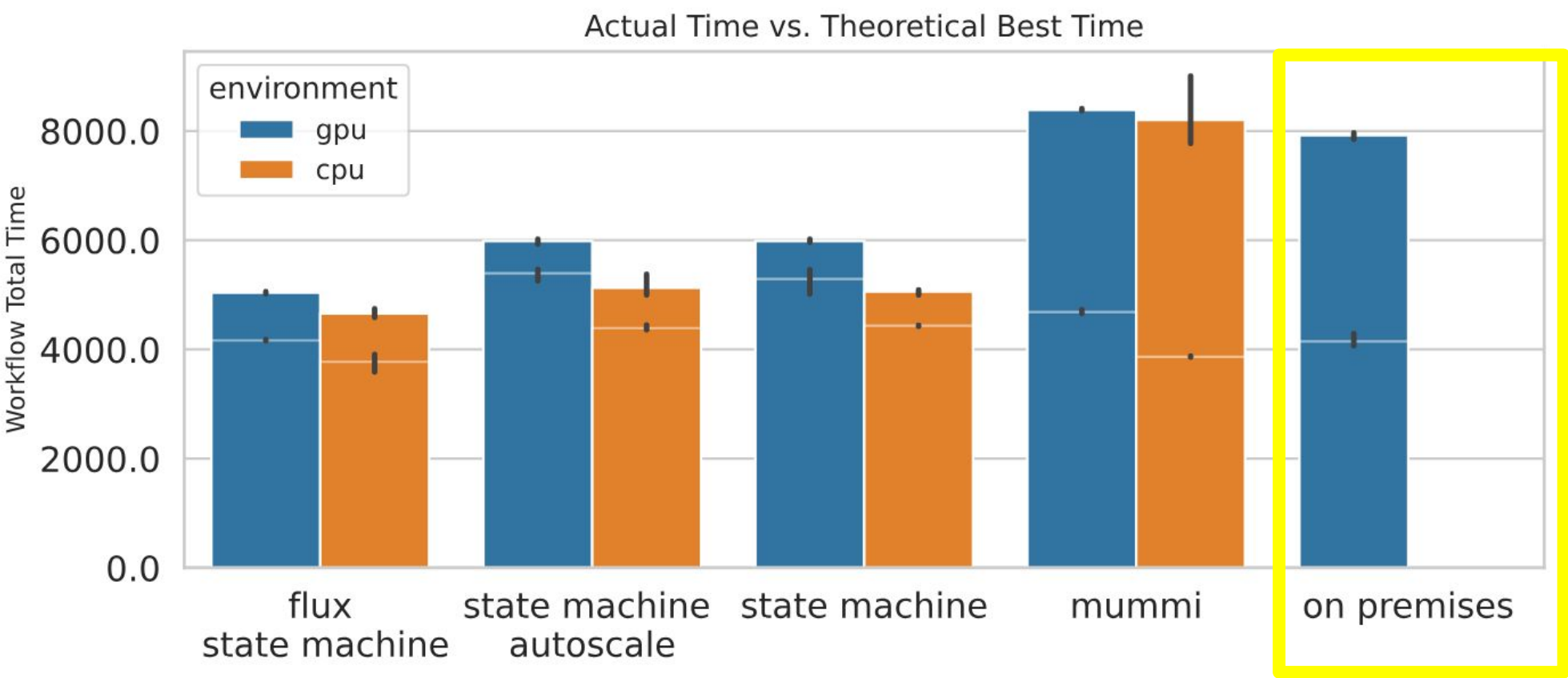
State Machine orchestration improves upon time to completion



State Machine orchestration improves upon time to completion

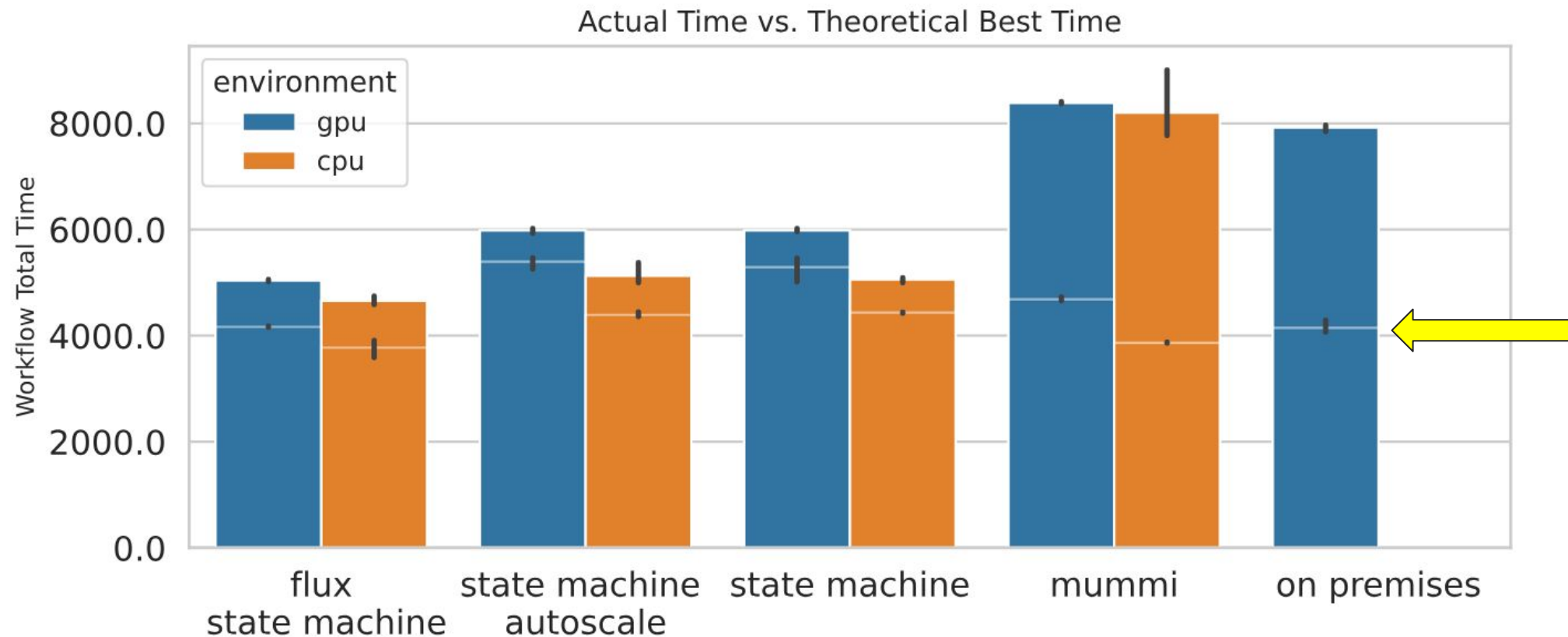


State Machine orchestration improves upon time to completion

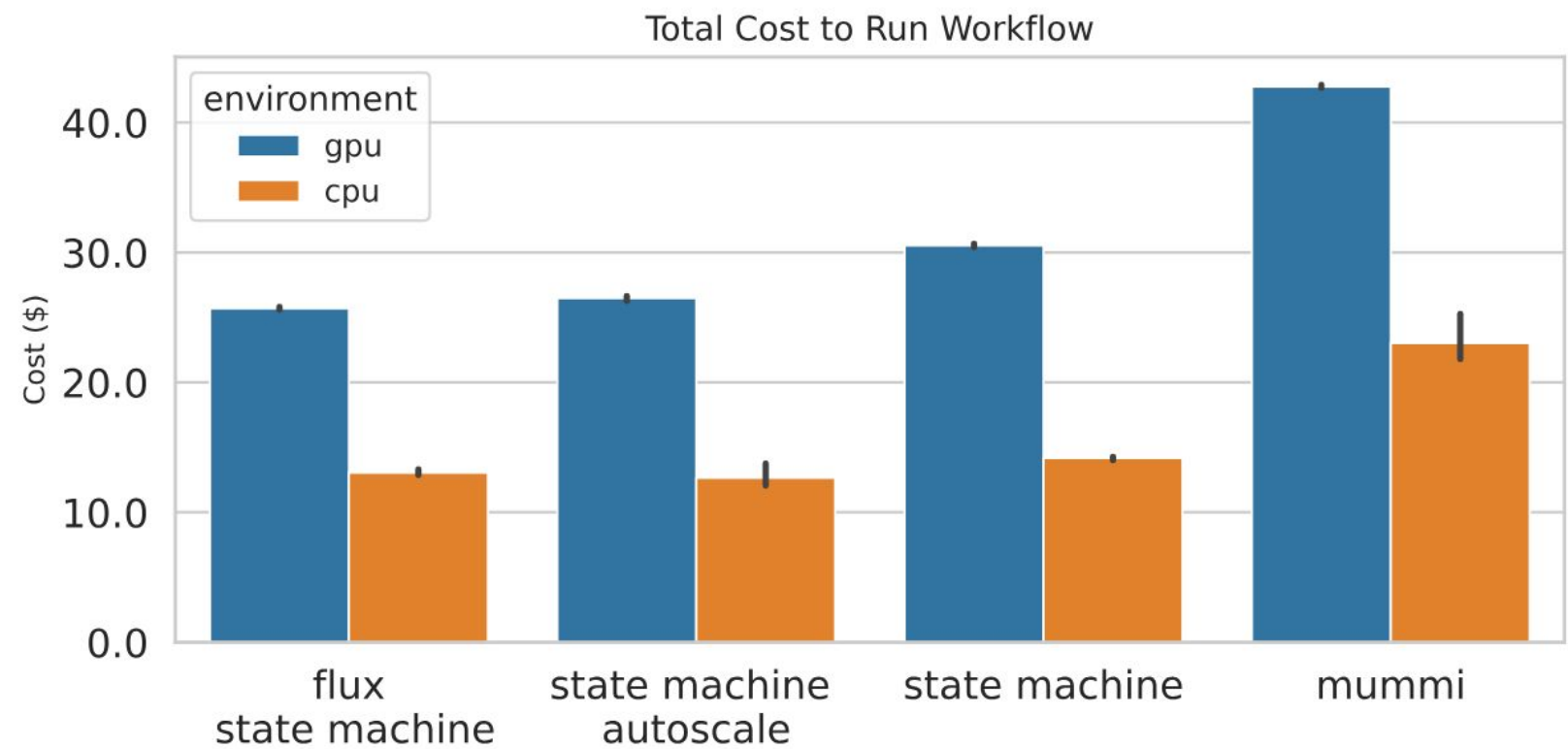


State Machine orchestration improves upon time to completion

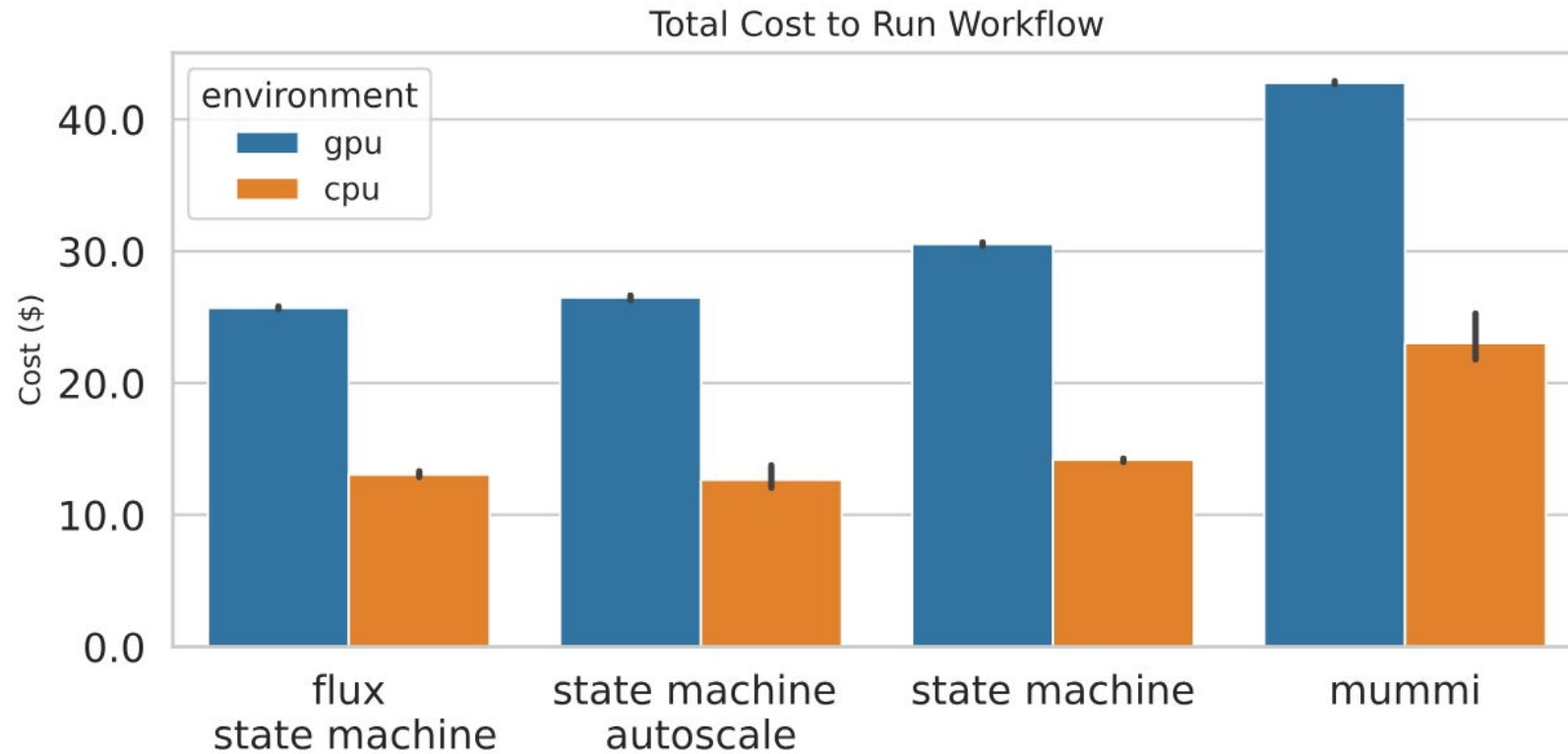
Theoretical best time: sum of all jobs, assuming orchestration removed



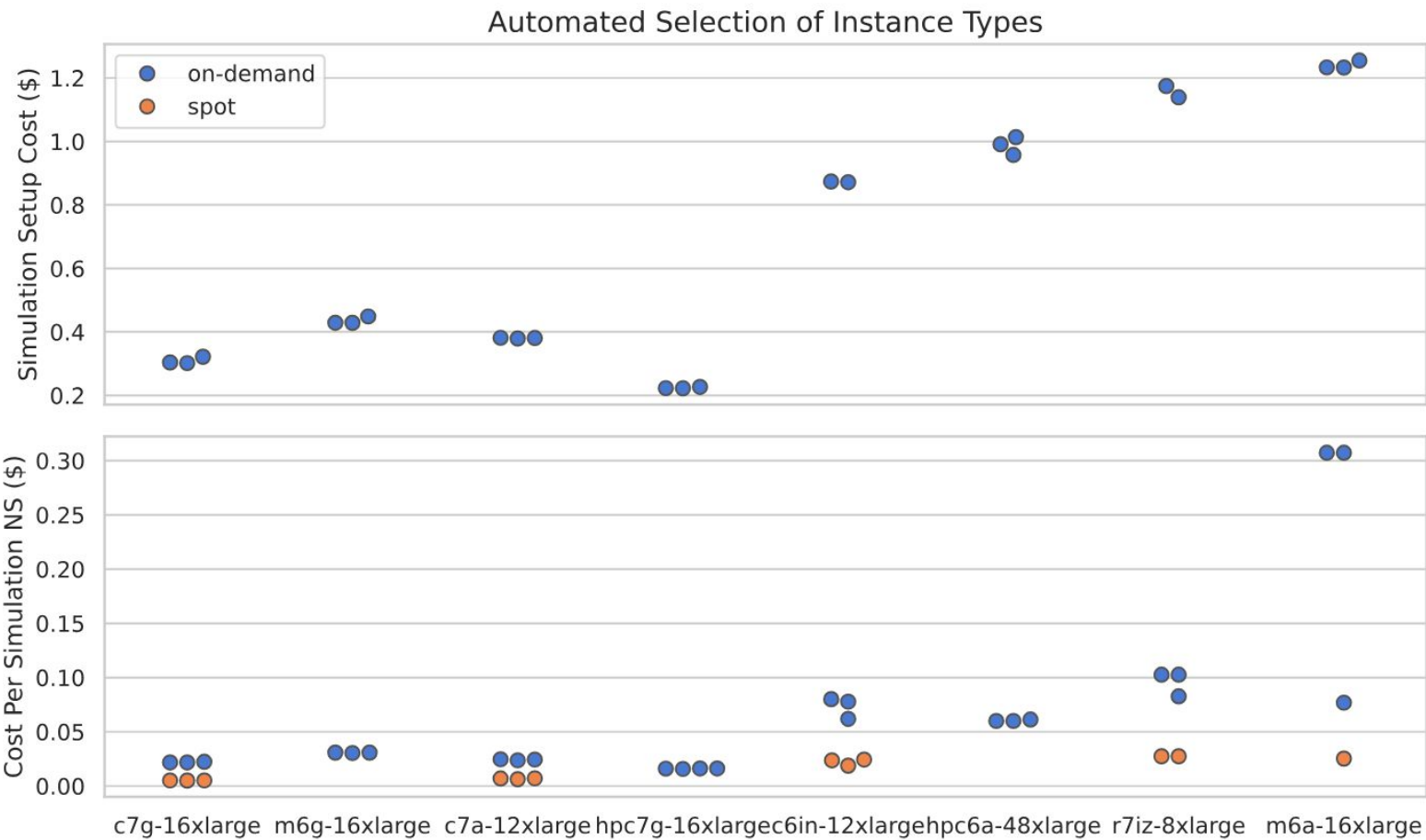
State Machine orchestration improves upon total cost



State Machine orchestration improves upon total cost



State machine workflows can help with instance selection too



Environment 3: Cloud Virtual Machines

Virtual machine setups across clouds

Builds with packer, orchestration with Terraform



converged-computing/flux-tutorials

Virtual machine setups across clouds

Builds with packer, orchestration with Terraform



Deployed on EC2
Elastic Fabric Adapter "EFA"
HPC family of instances



converged-computing/flux-tutorials

Virtual machine setups across clouds

Builds with packer, orchestration with Terraform



Deployed on EC2
Elastic Fabric Adapter "EFA"
HPC family of instances



Deployed as a VM set
Infiniband fabric
Standard HB120 v3



converged-computing/flux-tutorials

Virtual machine setups across clouds

Builds with packer, orchestration with Terraform



Deployed on EC2
Elastic Fabric Adapter "EFA"
HPC family of instances



Deployed as a VM set
Infiniband fabric
Standard HB120 v3



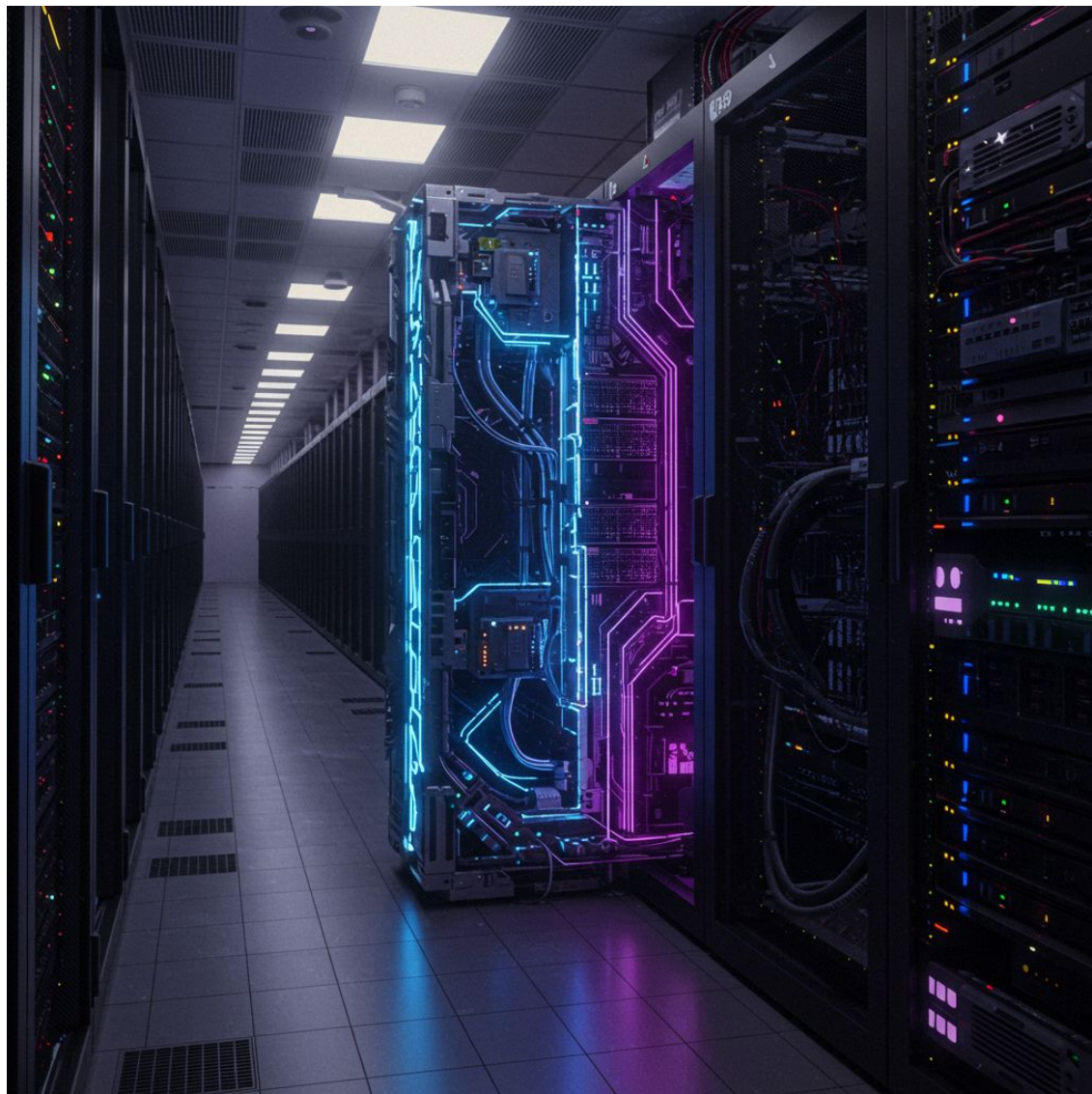
Deployed on Compute Engine
Tier I Networking (Premium)



converged-computing/flux-tutorials

Environment 4: Traditional HPC

System-level scheduler of an HPC System



Flux is the system scheduler on systems in the TOP500:

- El Capitan (#1, LLNL)
- Tuolumine (Top 10, LLNL)
- El Dorado (Top 50, Sandia)
- Rzadams (Top 50, LLNL)
- Tioga (Top 500, LLNL)
- Tenaya (Top 500)

Smaller clusters for user feedback

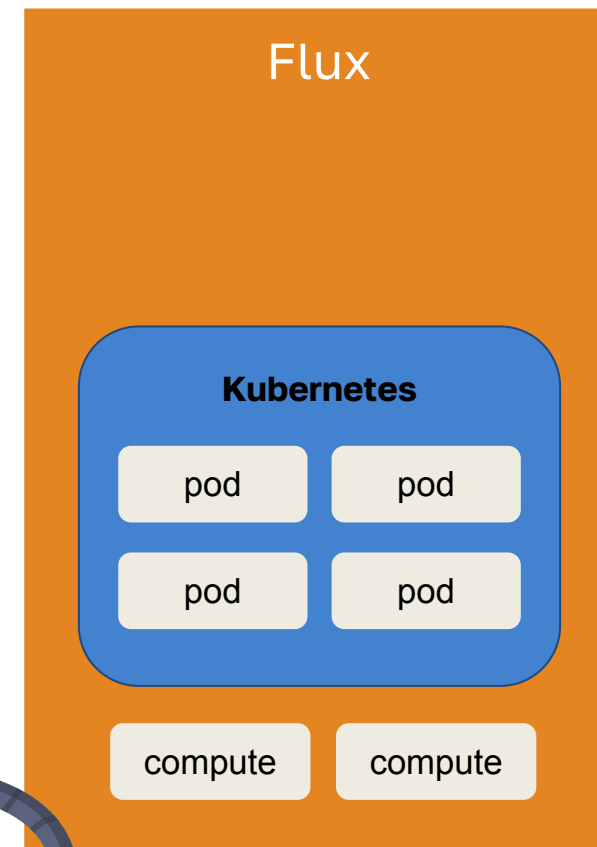
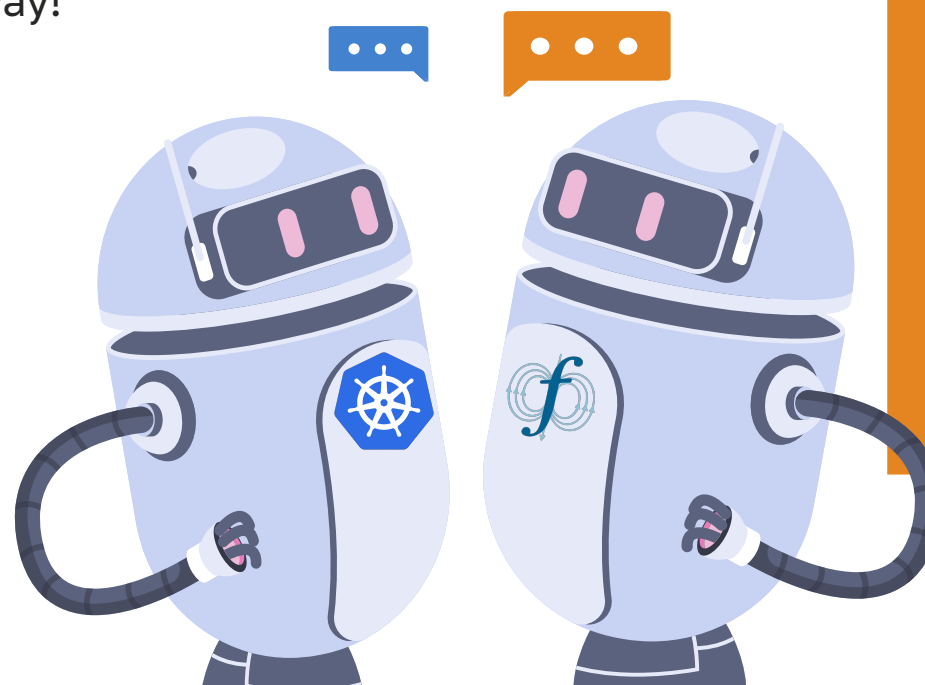
- Hetchy, Fluke, Elmerfudd

User-space Kubernetes alongside traditional HPC

"The Bare Metal Bros"

Deploy Kubernetes alongside Flux to get the best of both worlds - HPC simulations alongside services. We have achieved equal performance between bare metal and Kubernetes for environments, including:

- AWS using the Elastic Fabric Adapter
- Azure using Infiniband
- Production on-premises underway!



V. Sochat, D. Fox, and D. Milroy, "HPC Alongside User-space Kubernetes," 2024.
<https://arxiv.org/abs/2406.06995>

A user requests Usernetes via a batch job or allocation



I want to use Flux and Kubernetes (kubectl) interactively

flux alloc --setattr=usernetes=true bash

I want to run work that combines simulation with Kubernetes services

flux batch --setattr=usernetes=true batch.sh

A user requests Usernetes via a batch job or allocation



"The user-space Kubernetes cluster is created for the lifecycle of the allocation, and torn down upon completion."

Within the instance, you have Flux and Kubernetes

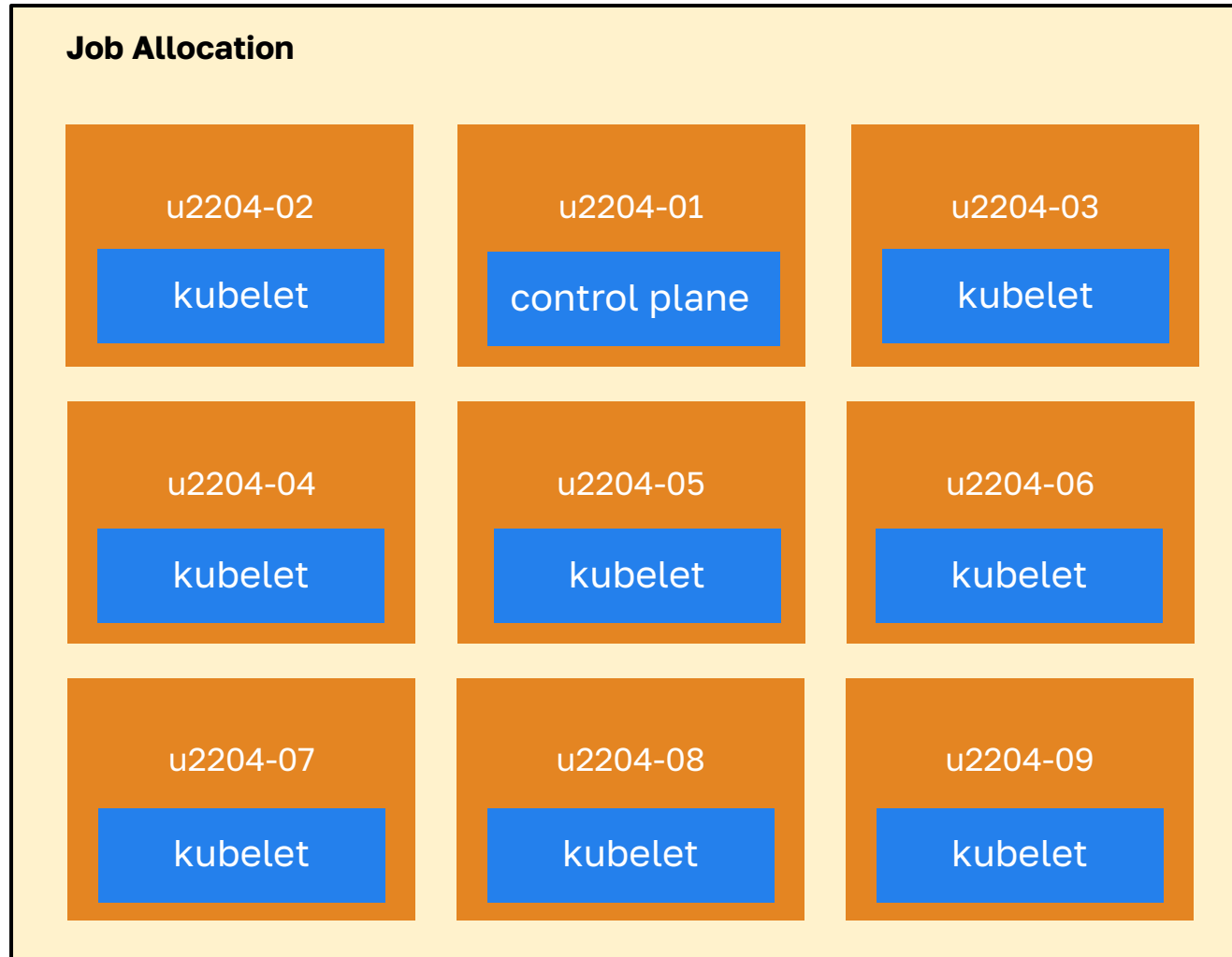
\$ kubectl get nodes

NAME	STATUS	ROLES	AGE	VERSION
u7s-u2204-01	Ready	control-plane	2m44s	v1.29.0
u7s-u2204-02	Ready	<none>	2m8s	v1.29.0
u7s-u2204-03	Ready	<none>	2m3s	v1.29.0
u7s-u2204-04	Ready	<none>	2m5s	v1.29.0
u7s-u2204-05	Ready	<none>	2m10s	v1.29.0
u7s-u2204-06	Ready	<none>	2m11s	v1.29.0
u7s-u2204-07	Ready	<none>	2m3s	v1.29.0
u7s-u2204-08	Ready	<none>	2m1s	v1.29.0
u7s-u2204-09	Ready	<none>	2m8s	v1.29.0

\$ flux resource list

STATE	NNODES	NCORES	NGPUS	NODELIST
free	9	72	0	u2204-[01-09]
allocated	0	0	0	
down	0	0	0	

Job Setup with Flux and User-Space Kubernetes

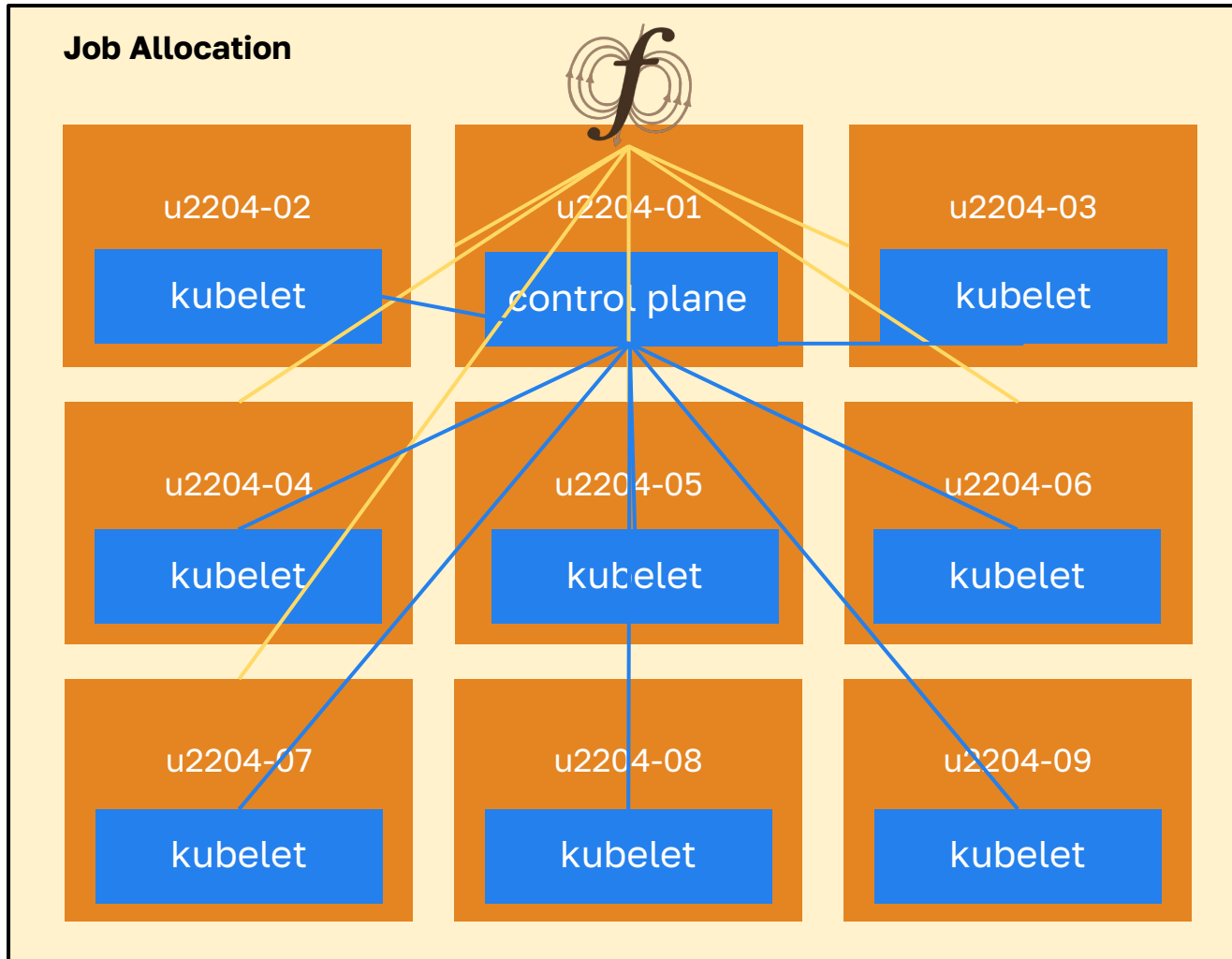


Usernetes 

Flux 

- rootless docker, **podman**, nerdctl
- docker-compose deployment
- Cluster configuration: kubeadm
- CRI: containerd
- OCI: runc
- CNI: Flannel

TBON (Flux) and Kubernetes CNI



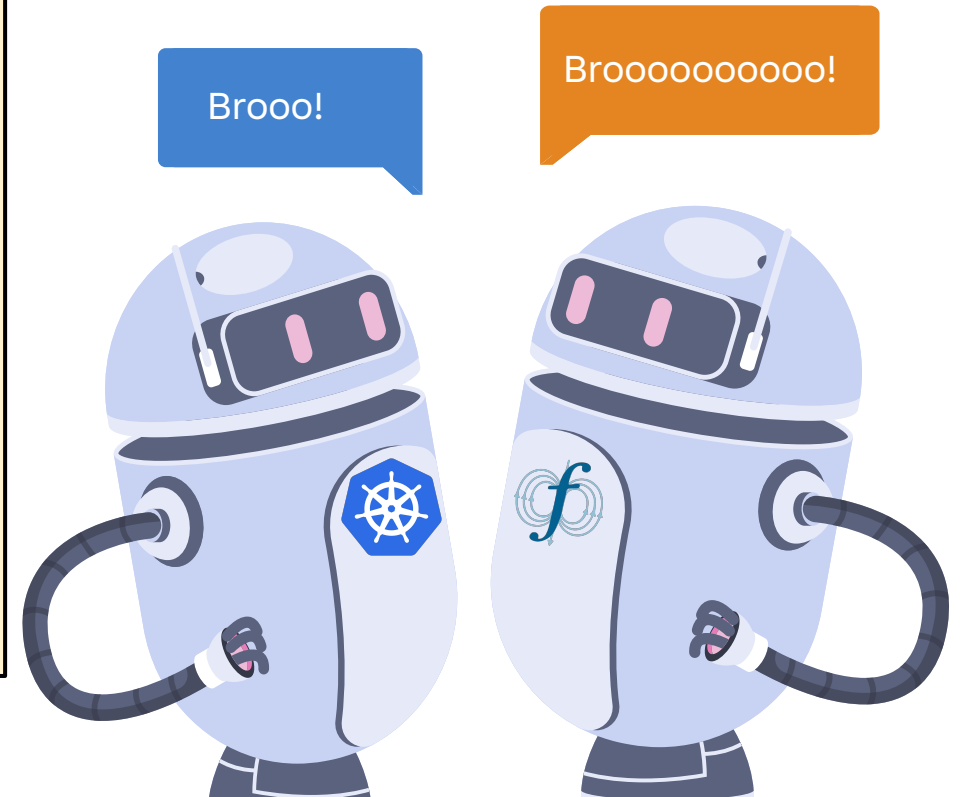
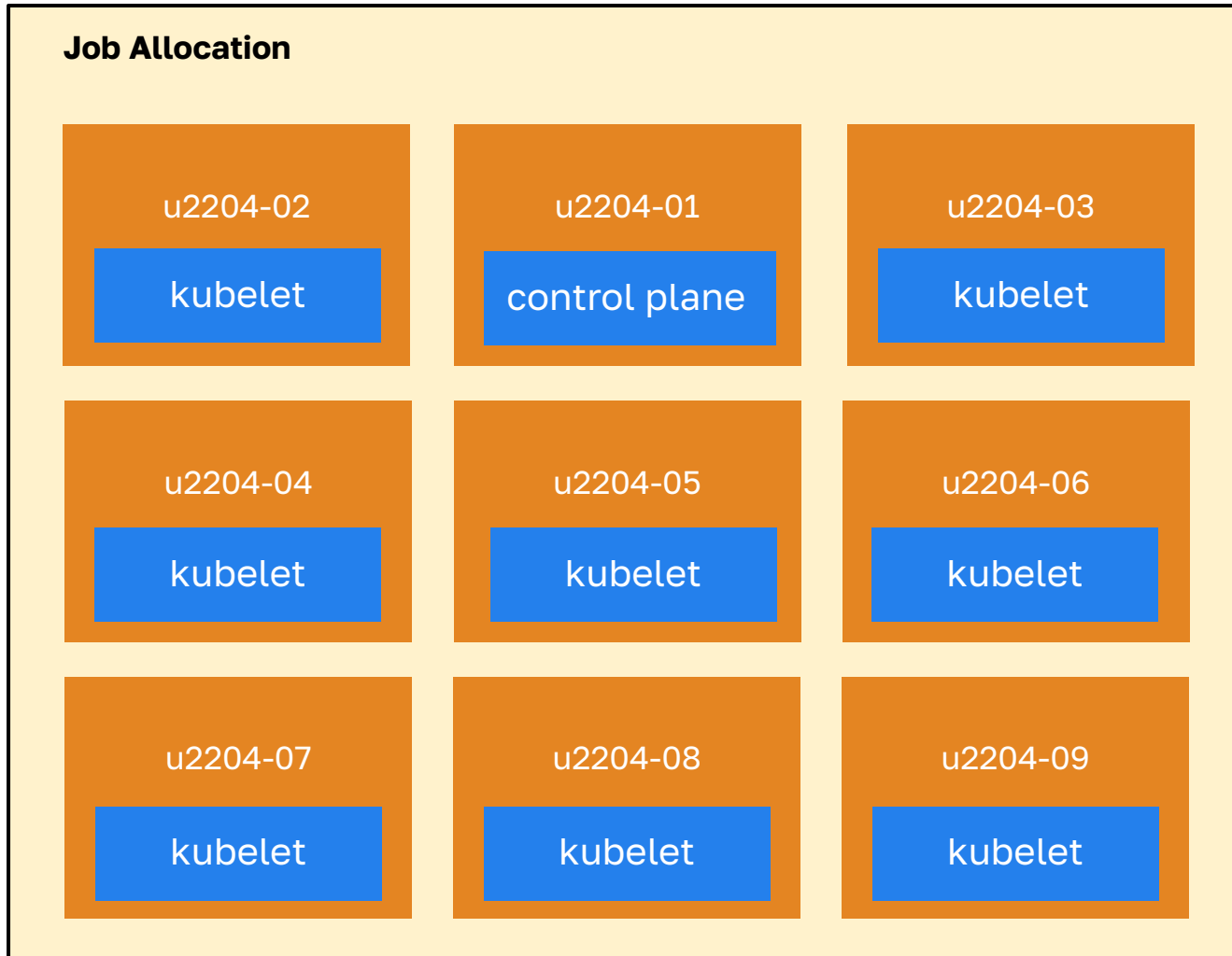
Usernetes



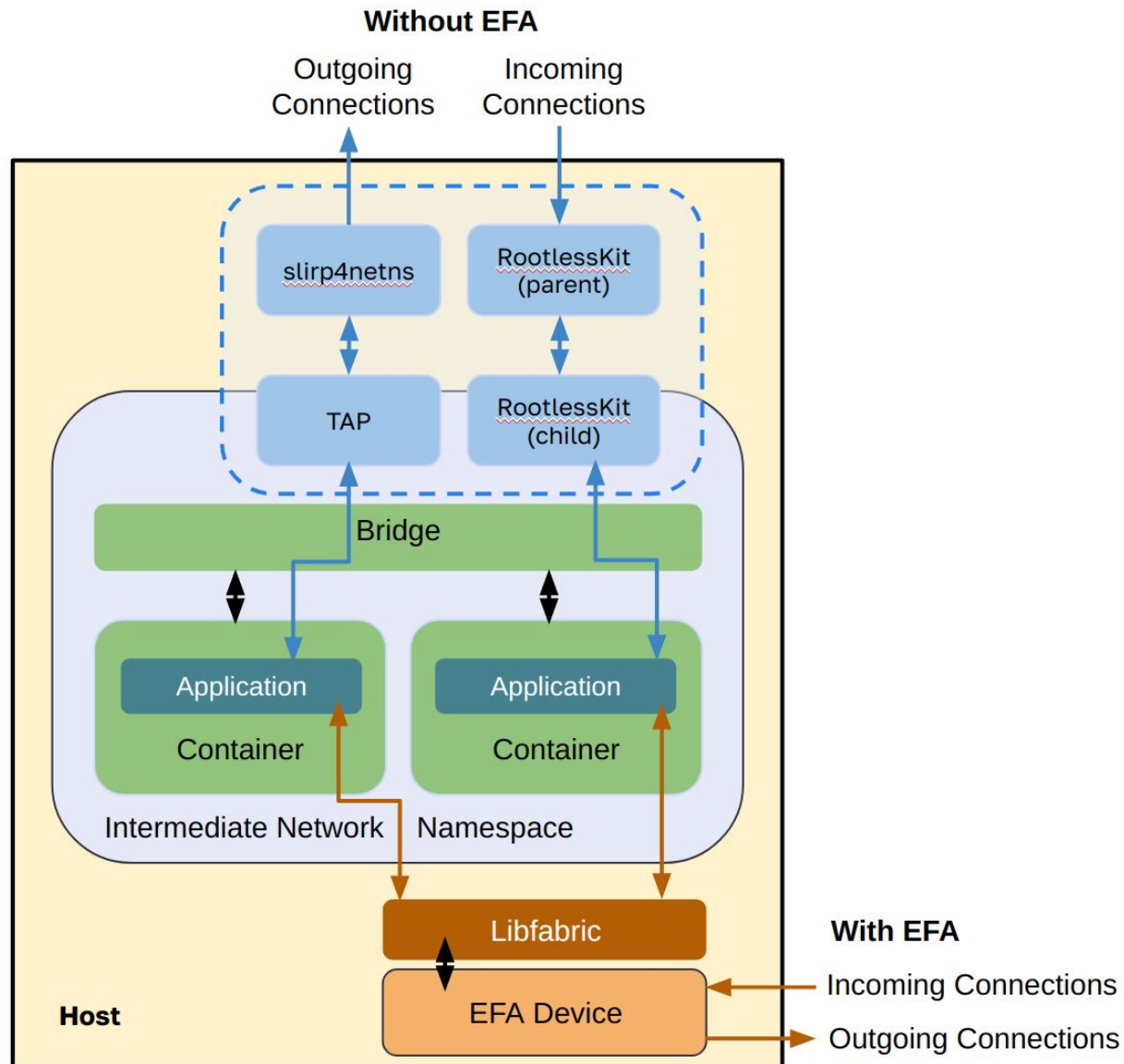
Flux



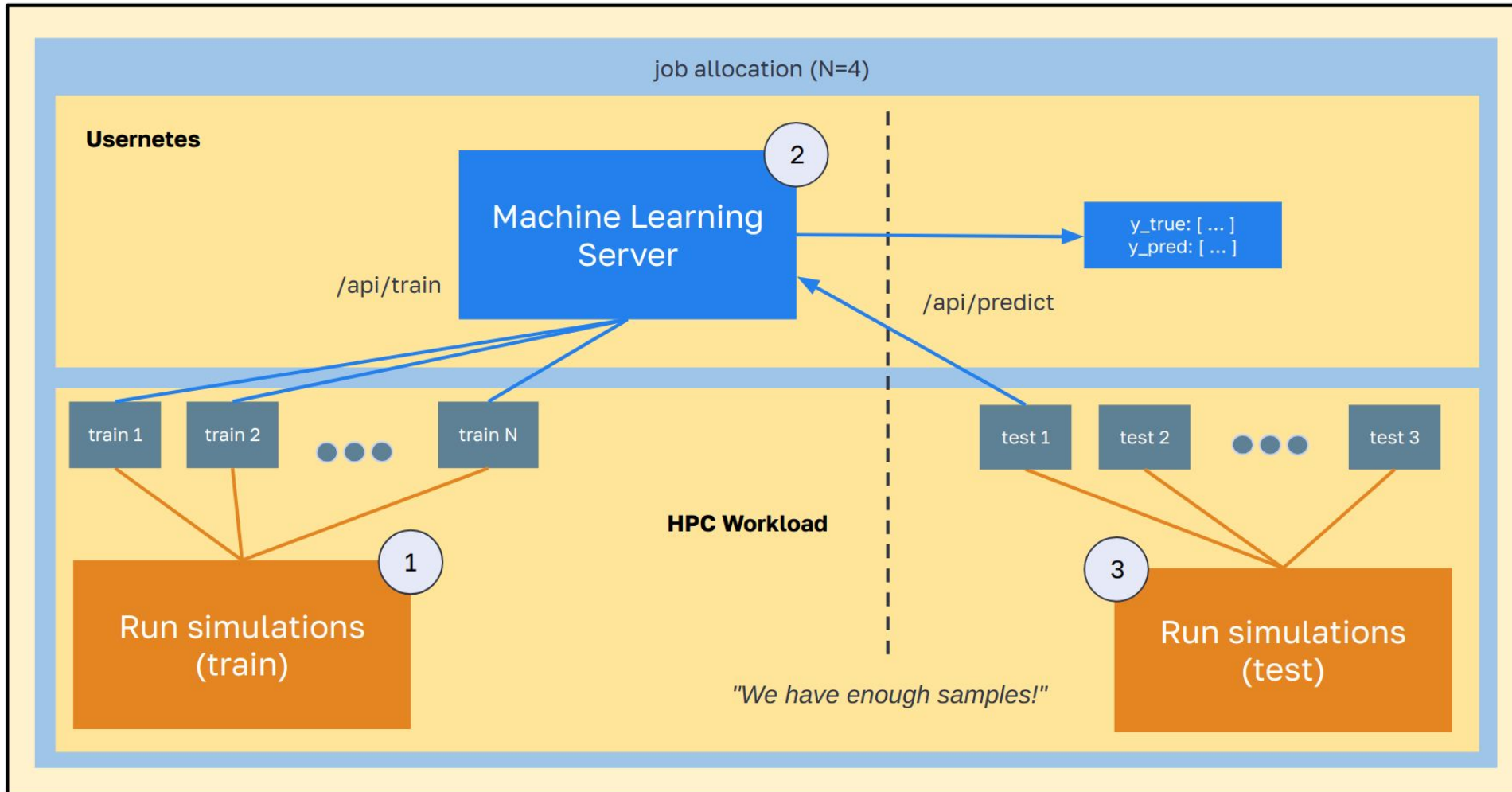
The "Bare Metal Bros"



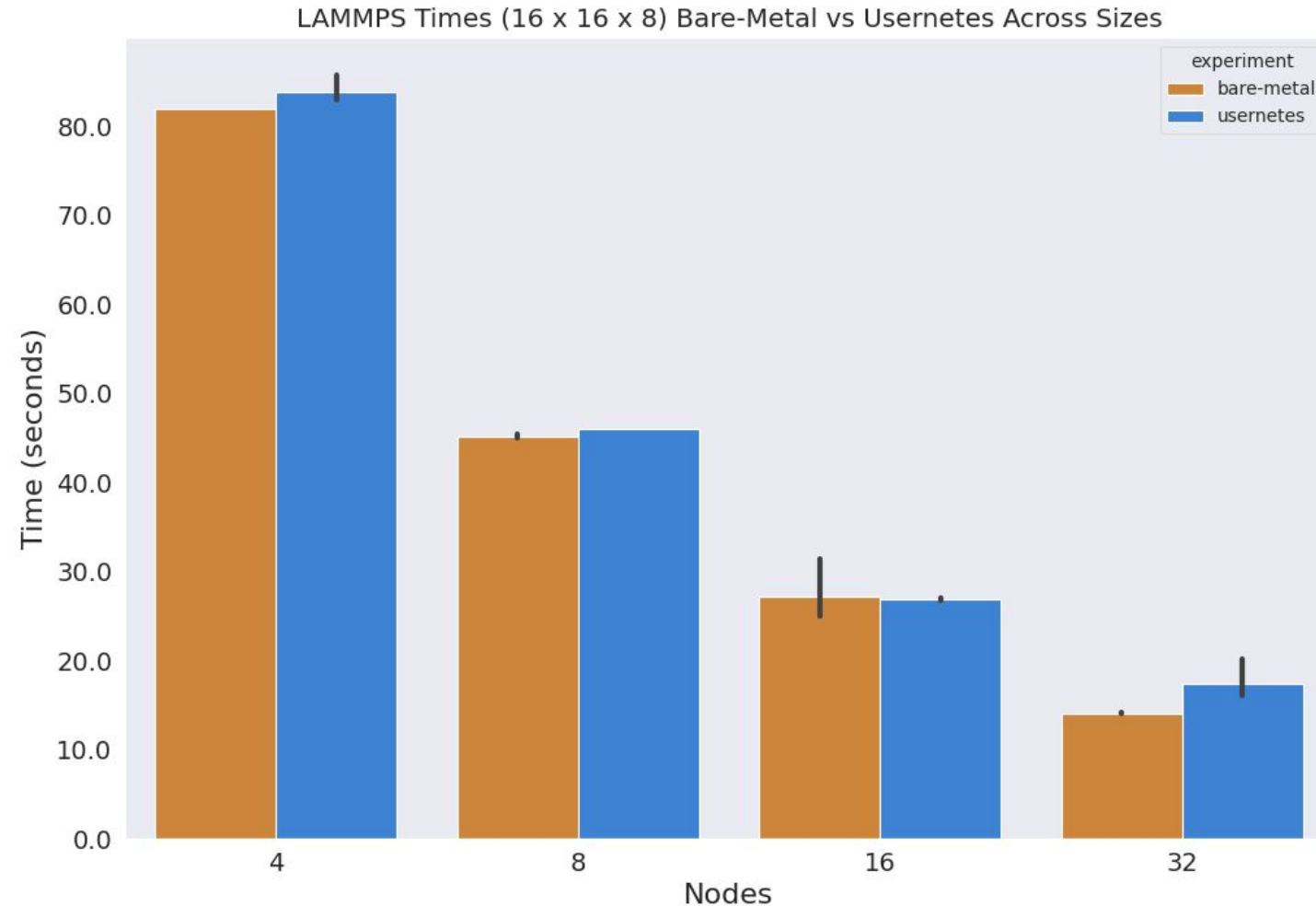
Elastic Fabric Adapter (EFA) on AWS bypasses slirp4netns



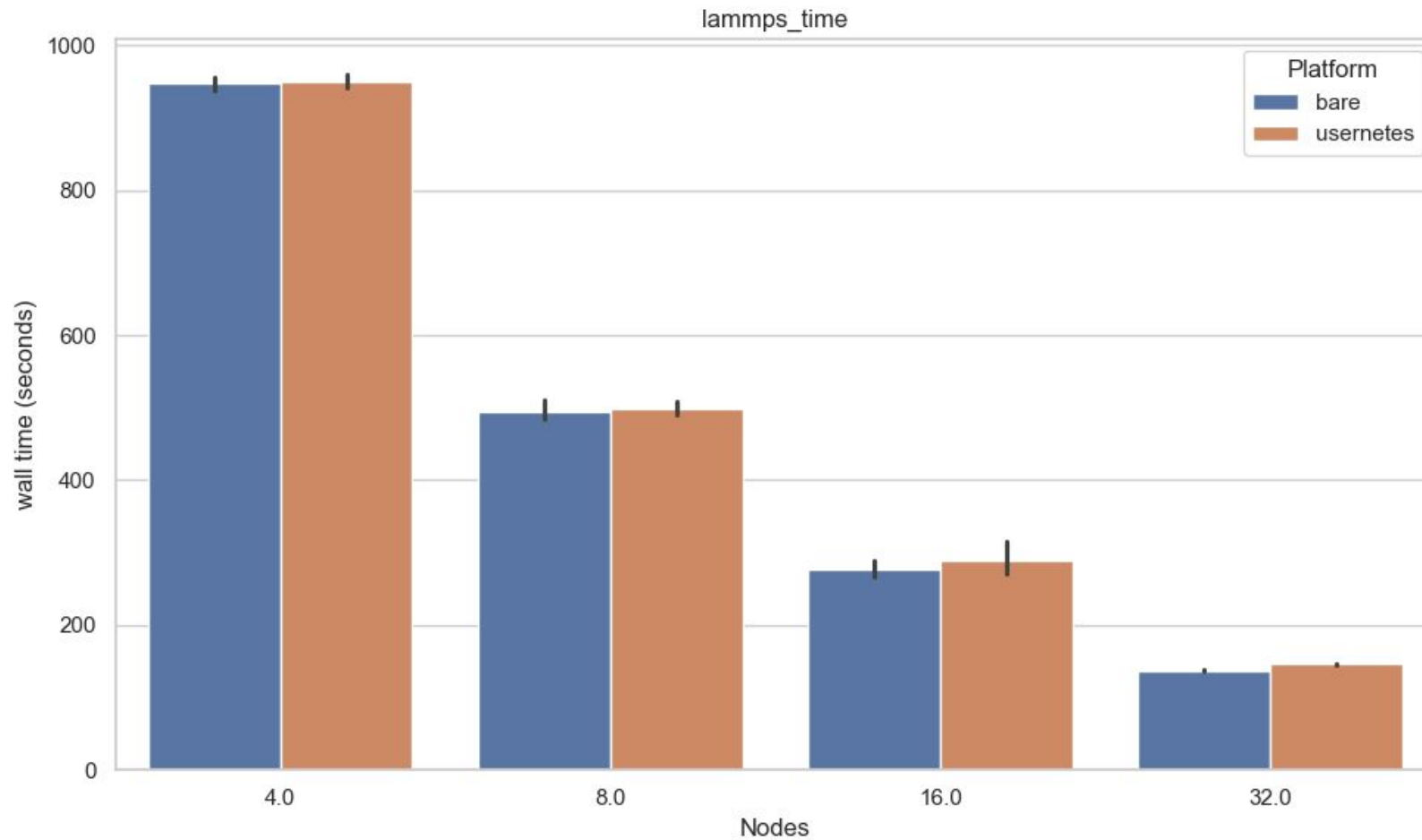
Orchestrated simulation with ML Server



Elastic Fabric Adapter (EFA) on AWS bypasses slirp4netns

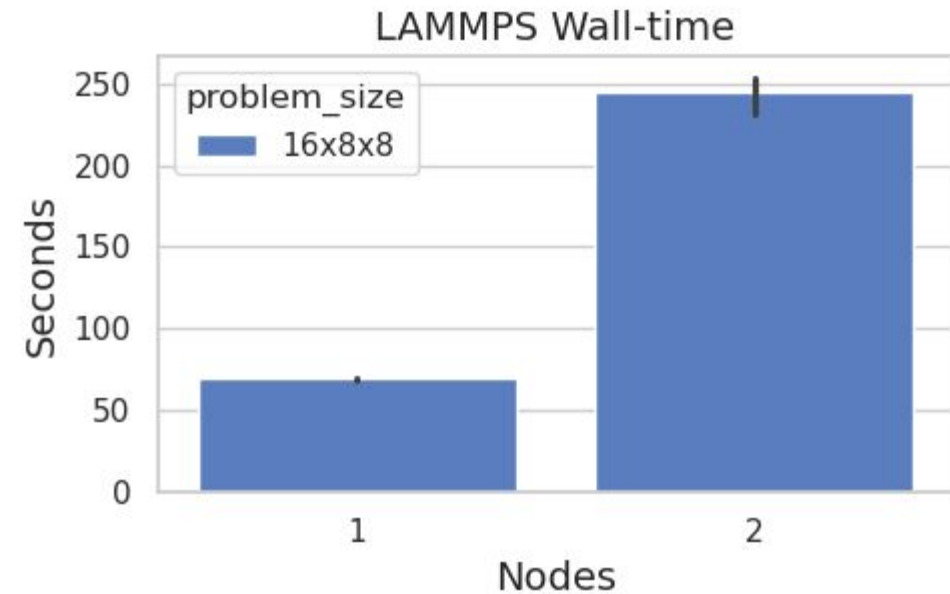
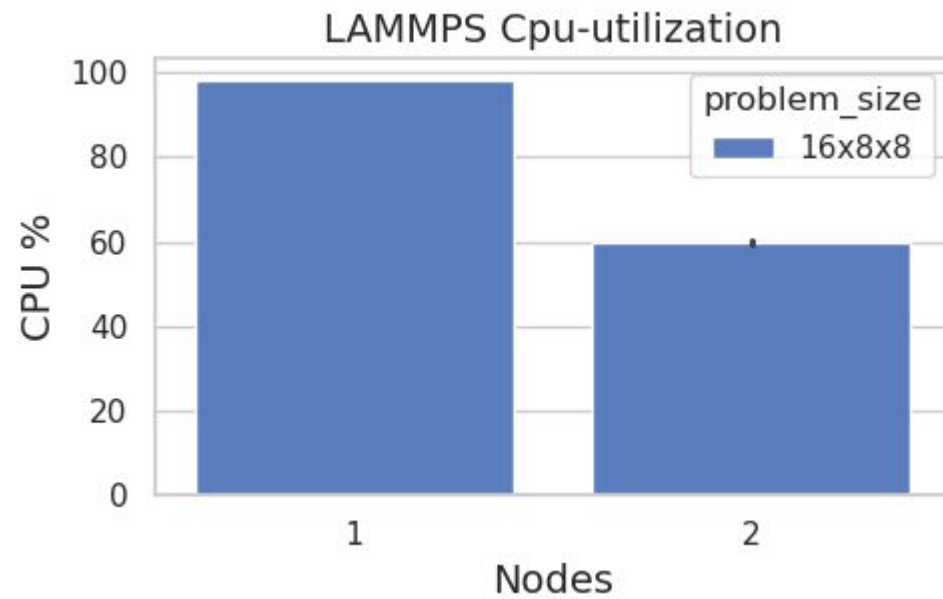


Infiniband Azure (LAMMPS) also bypasses slirp4netns!



Bare Metal - Hello World!

No good fabric yet

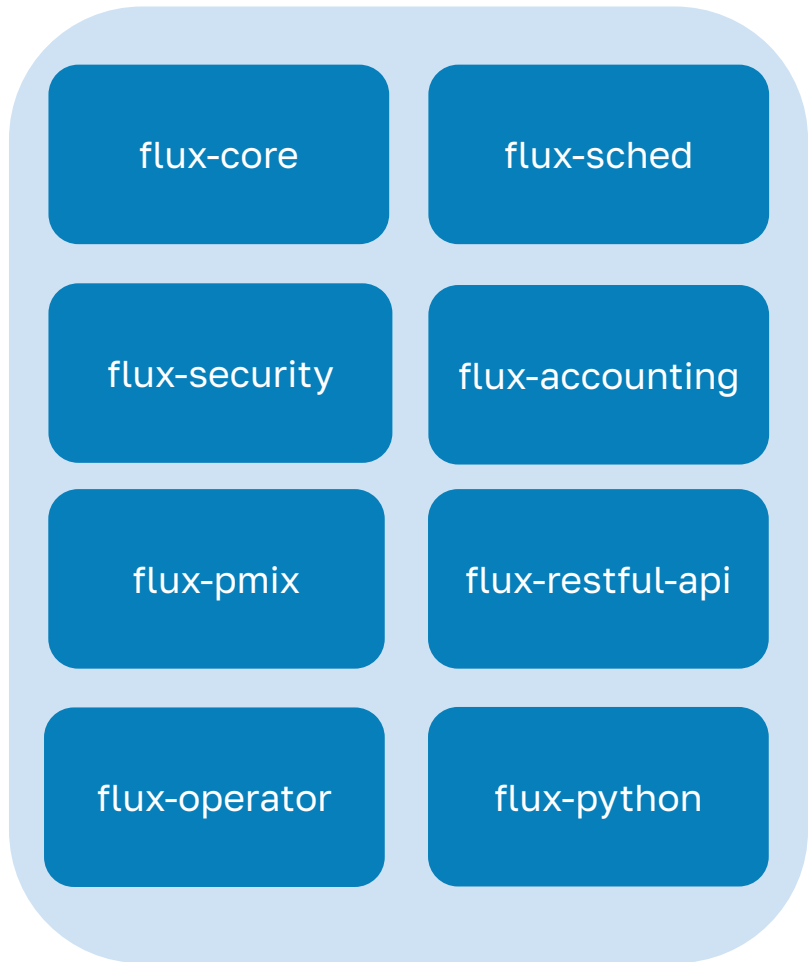


Part III

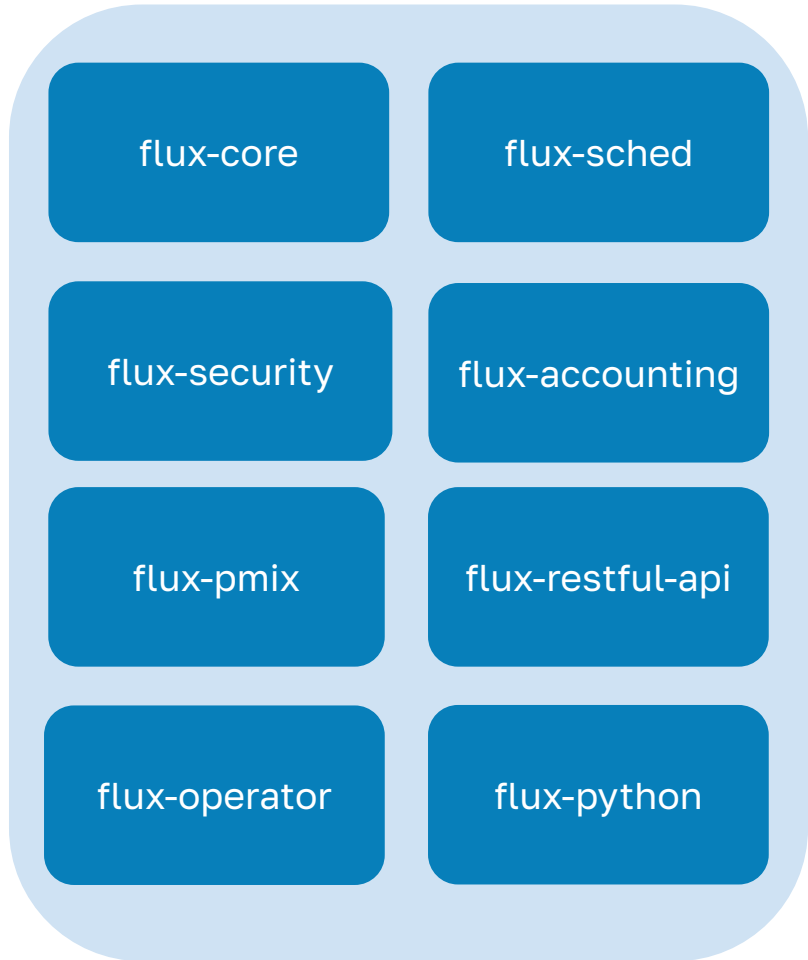
Flux Community



All of Flux is Open Source

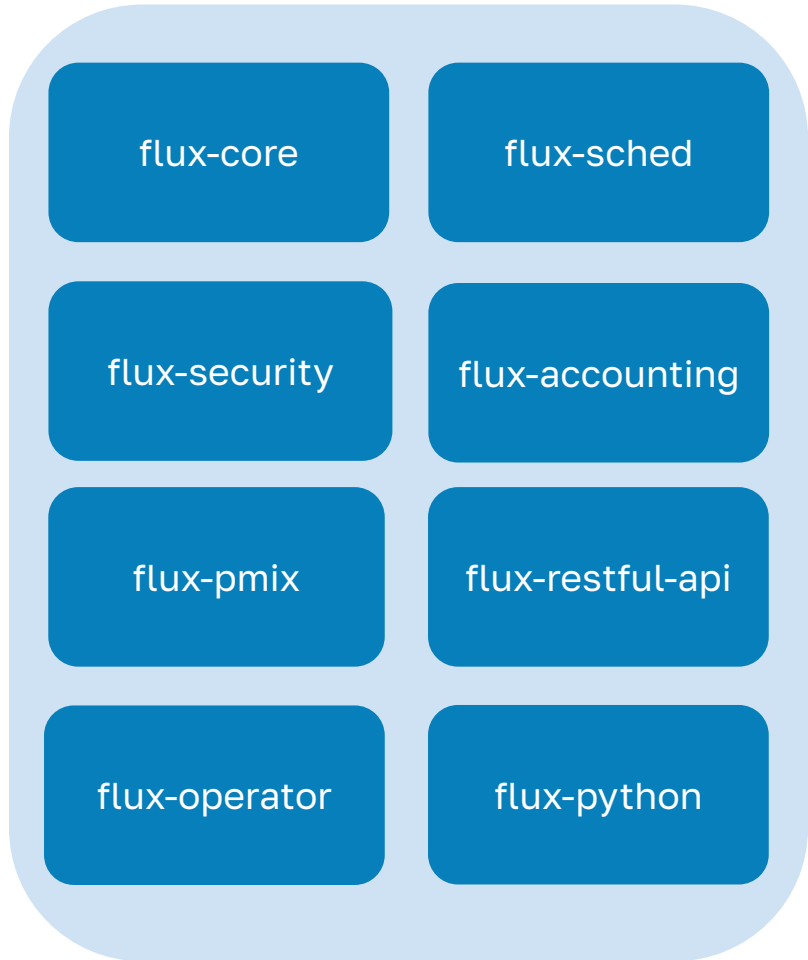


All of Flux is Open Source



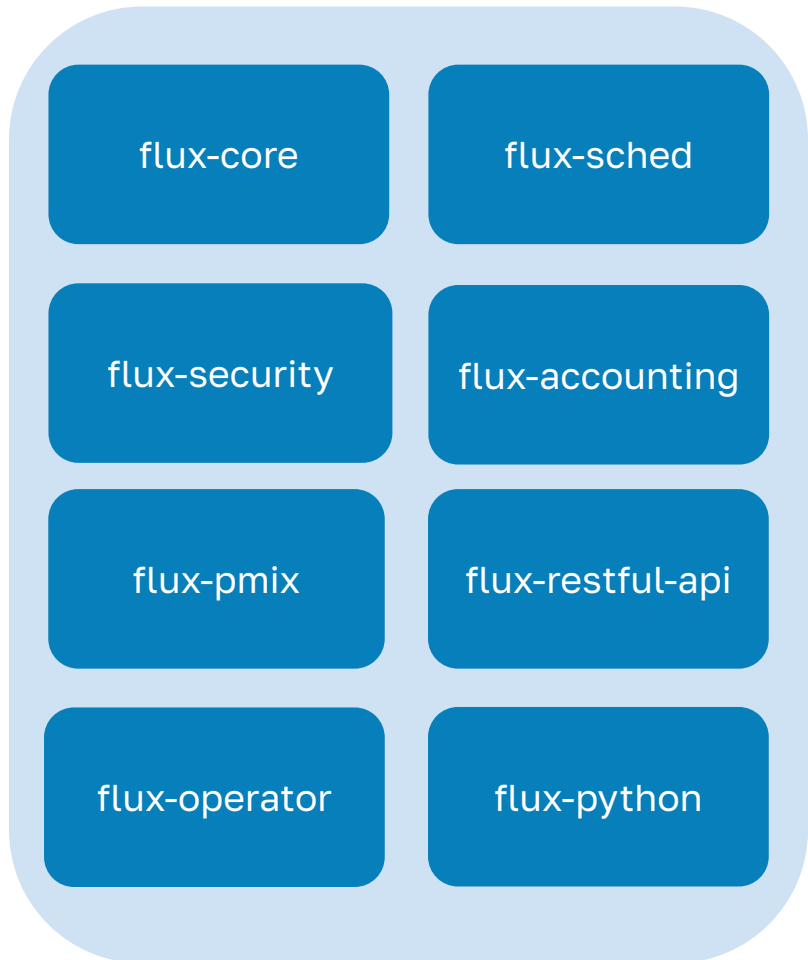
- Open issues for questions or bugs
 - If unsure, post to flux-core

All of Flux is Open Source



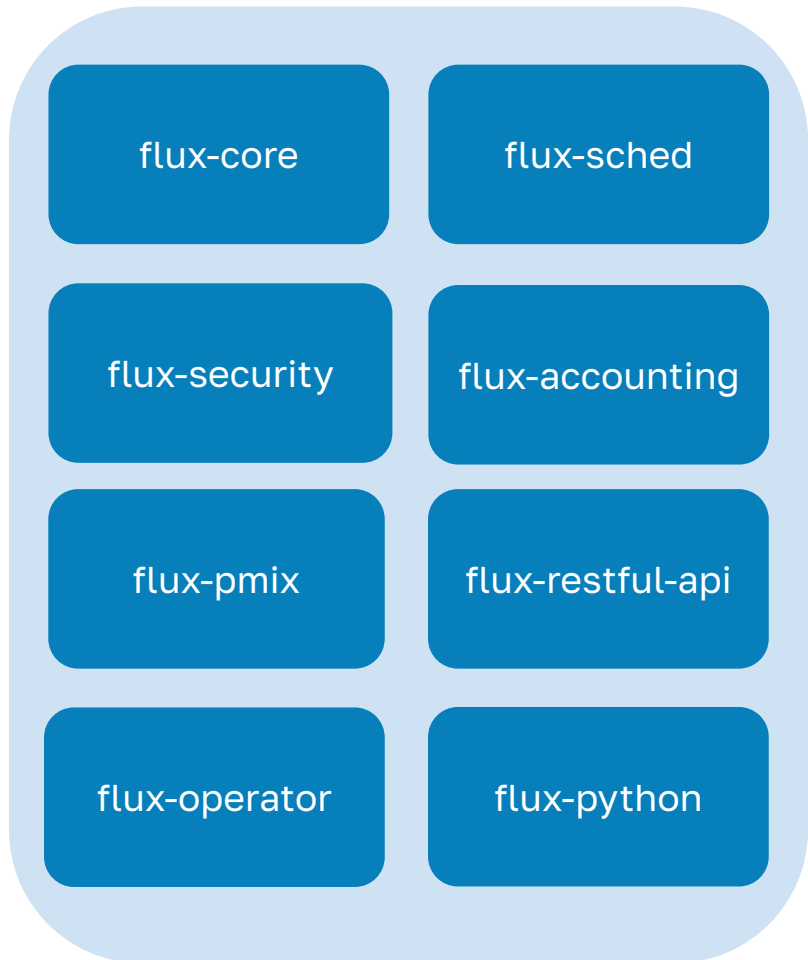
- Open issues for questions or bugs
 - If unsure, post to flux-core
- Discussion board (flux-core)

All of Flux is Open Source



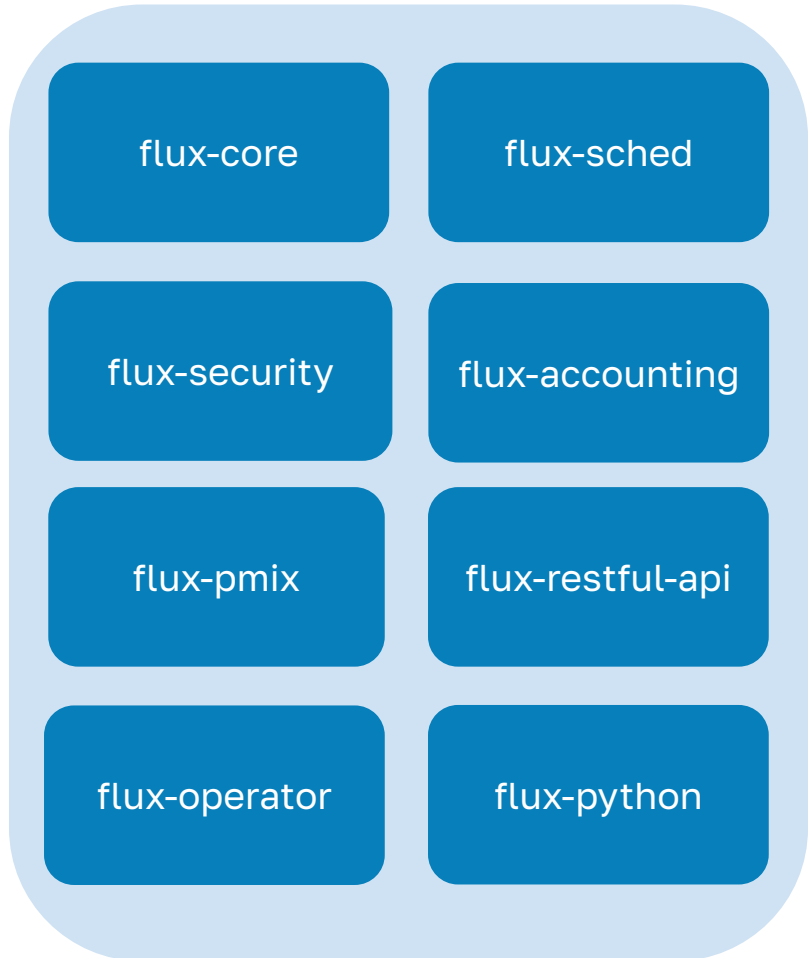
- Open issues for questions or bugs
 - If unsure, post to flux-core
- Discussion board (flux-core)
- flux-discuss@lists.llnl.gov

All of Flux is Open Source



- Open issues for questions or bugs
 - If unsure, post to flux-core
- Discussion board (flux-core)
- flux-discuss@lists.llnl.gov
- Slack: llnl-performance.llnl.gov
 - Ask us for an invite link

All of Flux is Open Source



- Open issues for questions or bugs
 - If unsure, post to flux-core
- Discussion board (flux-core)
- flux-discuss@lists.llnl.gov
- Slack: llnl-performance.llnl.gov
 - Ask us for an invite link
- Contributing guide:

<https://flux-framework.readthedocs.io/en/latest/contributing.html>

We actively pursue industry collaboration to advocate for and support science.



AWS:

Several Flux collaborations, SC25 Tutorial 🙌



Red Hat



IBM T.J. Watson Research Center / RedHat

Collaborated on Flux and scheduling



Google:

Collaborated on Flux Operator

Flux developers respond to needs of researcher users

- ✓ New of resource/job events journal consumer interfaces (Python API)
- ✓ Flux module tracing
- ✓ Scalability/reliability improvements to running on large systems
- ✓ Support for mustache templates in environment variables
- ✓ Custom resource module policies (flux-sched)
- ✓ Support for grow/shrink of resource graph and jobs (flux-sched)
- ✓ Enhanced support for batch job resiliency (shrink resource set)
- ✓ QueueList class access to config, limits, status, counts (Python API)

Flux Community Survey 🎉

<https://converged-computing.org/community-survey-2025>

Thank you!

<https://converged-computing.org/community-survey-2025>

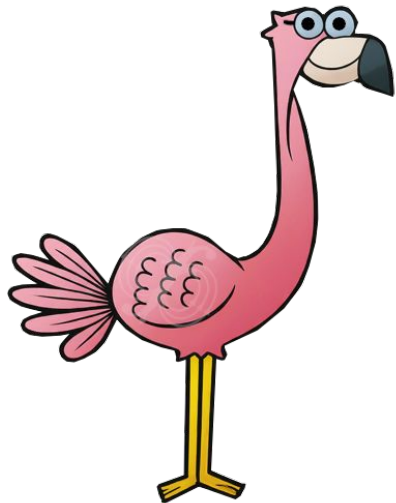
<https://flux-framework.org>

scogland1@llnl.gov | milroy1@llnl.gov | sochat1@llnl.gov | hobbs17@llnl.gov



This work was performed under the auspices of the U.S. Department of Energy by Lawrence Livermore National Laboratory under Contract DE-AC52-07NA27344.

Non-LLNL owned images were generated by Gemini unless otherwise attributed.



Astounding.

Flux developers respond to needs of researcher users

Last update: August 2024 (flux-core 0.65.0 and flux-sched 0.37.0)

- ✓ Interfaces for watching and querying jobs and queues
- ✓ Improved tools for file broadcast
- ✓ Utility for posting manual job events
- ✓ Scalability/reliability improvements to running on large systems
- ✓ Improvements to Flux user interfaces and human readability of logs
- ✓ More options for generating taskmaps
- ✓ Command **flux update** to change job duration after submit
- ✓ Housekeeping: epilog tasks decoupled from jobs